

中山大學
SUN YAT-SEN UNIVERSITY



国家超级计算广州中心
NATIONAL SUPERCOMPUTER CENTER IN GUANGZHOU

DCS290

Compilation Principle

编 译 原 理

第四章 语法分析

郑馥丹
zhengfd5@mail.sysu.edu.cn

CONTENTS

目 录

01 自顶向下分析 Top-Down Parsing

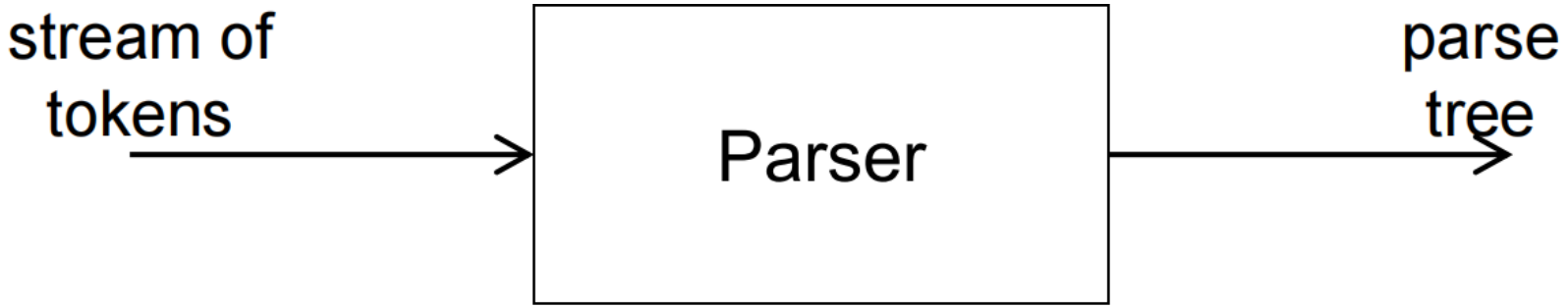
02 LL(1)分析 LL(1) Parsing

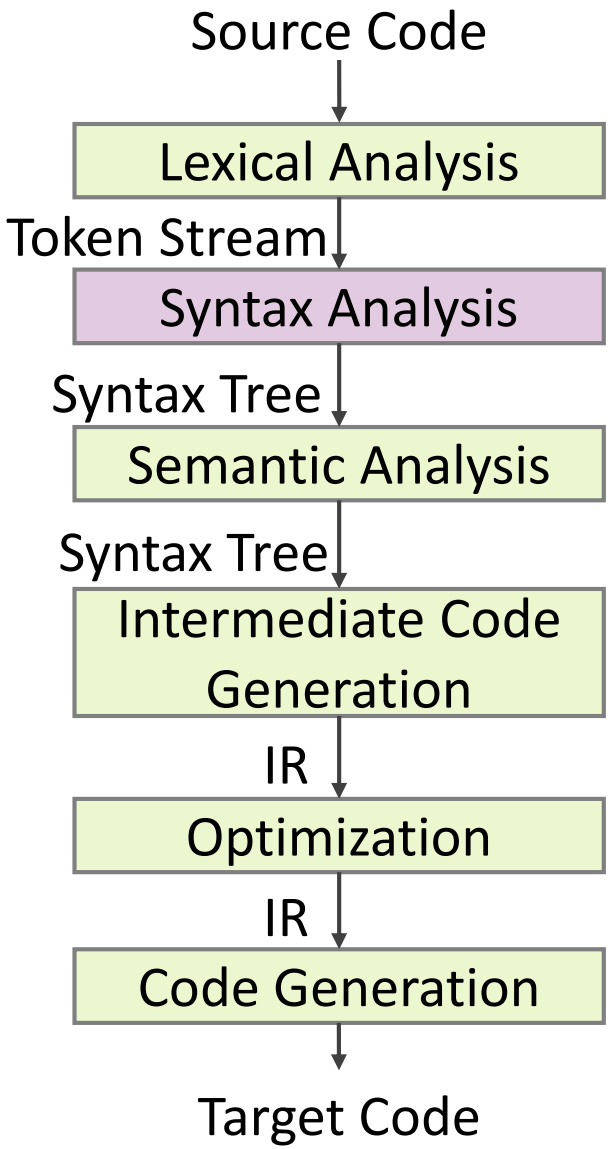
03 自底向上分析 Bottom-Up Parsing

04 LR分析 LR Parsing

1. 语法分析[Syntax Analysis]

- 解析源程序对应的token序列，生成语法分析结构（syntax tree, 语法分析树）





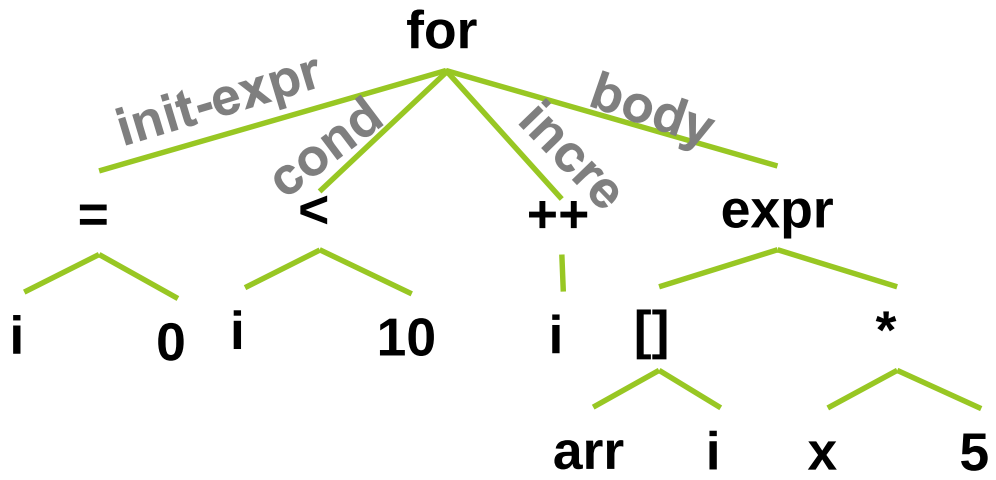
1. 语法分析[Syntax Analysis]

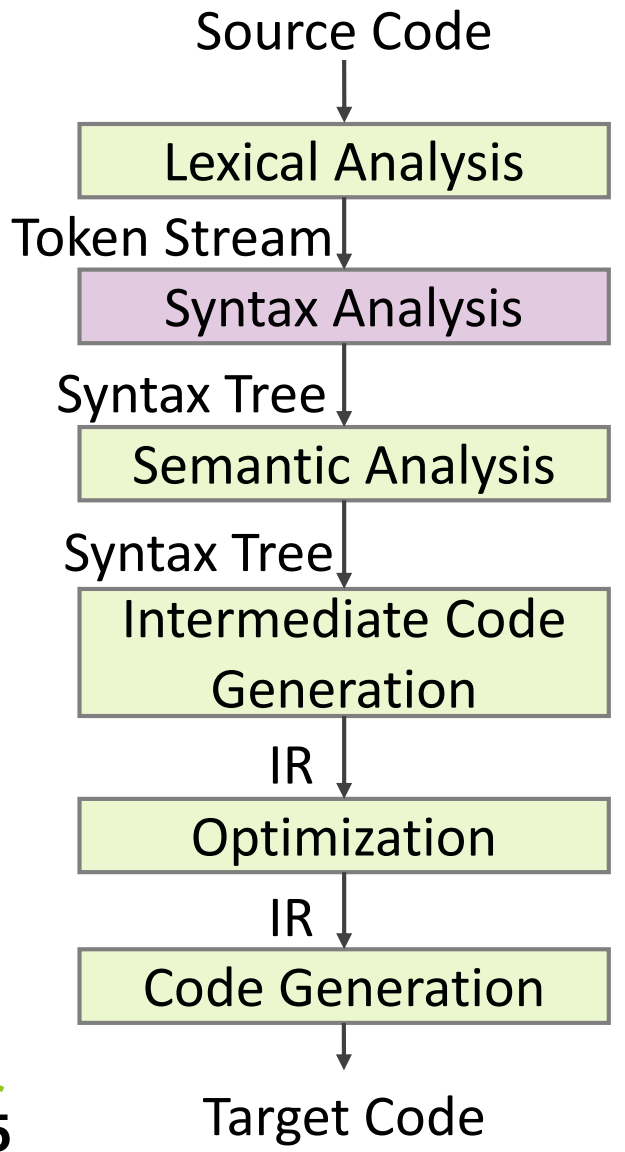
- 解析源程序对应的token序列，生成语法分析结构（syntax tree, 语法分析树）
  - 输入：单词流，输出：语法树
  - 输入程序是否符合语法规则？
    - ✓ x*+
    - ✓ a += 5;

```
void main()
{
    int arr[10], i, x = 1;

    for (i = 0; i < 10; i++)
        arr[i] = x * 5;
}
```

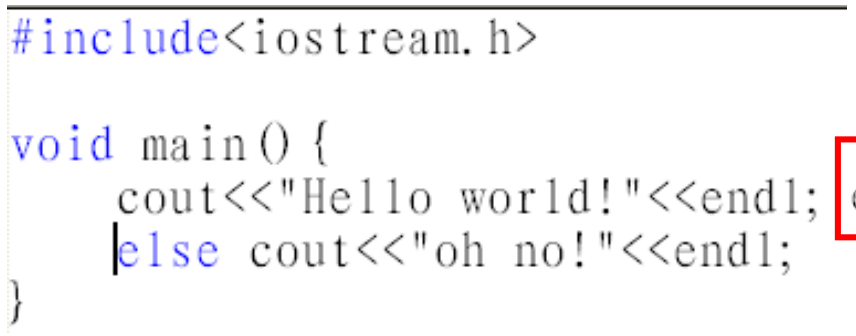
```
keyword(for)
symbol(()
id(i)
symbol(=)
num(0)
symbol(;)
id(i)
symbol(<)
num(10)
symbol(;)
id(i)
symbol(++)
symbol())
id(arr)
symbol([)
id(i)
symbol(])
symbol(=)
id(x)
symbol(*)
num(5)
symbol(;)
```





1. 语法分析[Syntax Analysis]

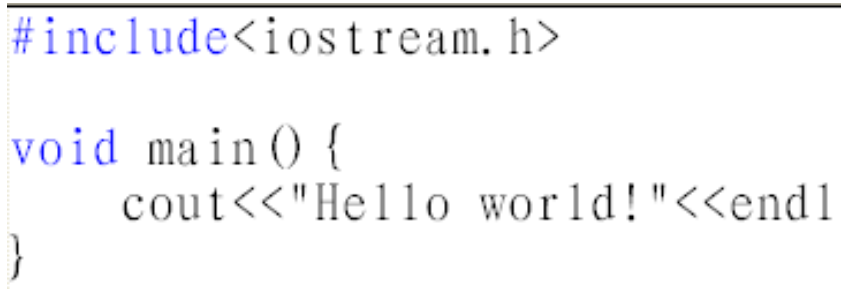
- 解析源程序对应的token序列，生成语法分析结构（syntax tree, 语法分析树）
  - 输入：单词流，输出：语法树
  - 输入程序是否符合语法规则？



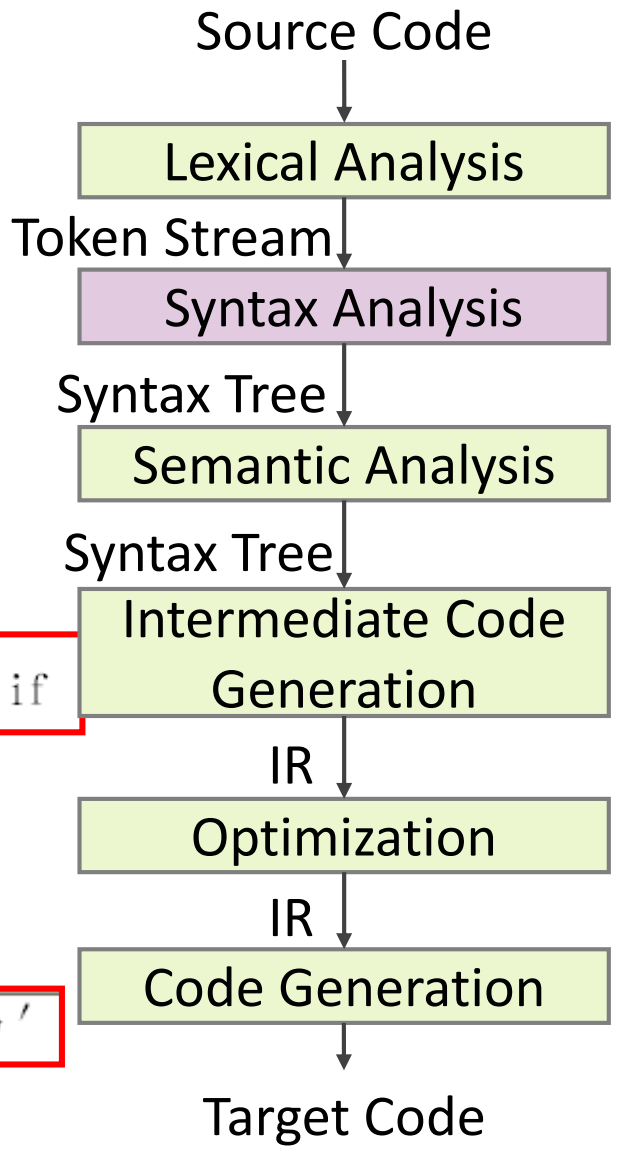
else没有匹配的if

```
error C2181: illegal else without matching if
```

表达式缺少分号结尾




```
syntax error : missing ';' before '}'
```



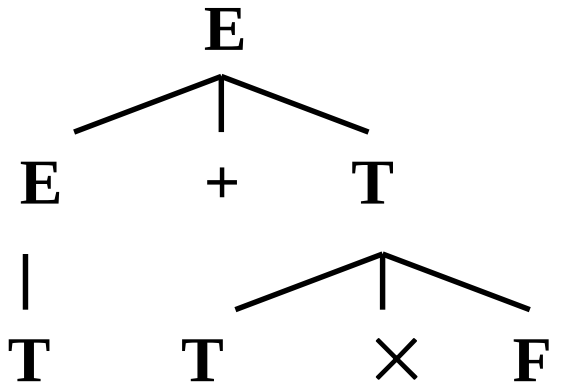
1. 语法分析[Syntax Analysis]

- 复习——准备工作
  - 上下文无关文法**[Context-Free Grammar，CFG]**:
    - ✓ 对任一产生式α→β，都有α∈V_N， β∈(V_N∪V_T)*

      例 文法G[S]：S→AB　A→BS|0　B→SA|1
  - 推导与语法树
    - ✓ 给定文G=(V_N,V_T,P,S)，对于G的任何句型都能构造与之关联的语法树

文法G：　E→E+T|T
T→T×F|F
F→(E)|i

E=>E+T　=>E+T×F　=>T+T×F

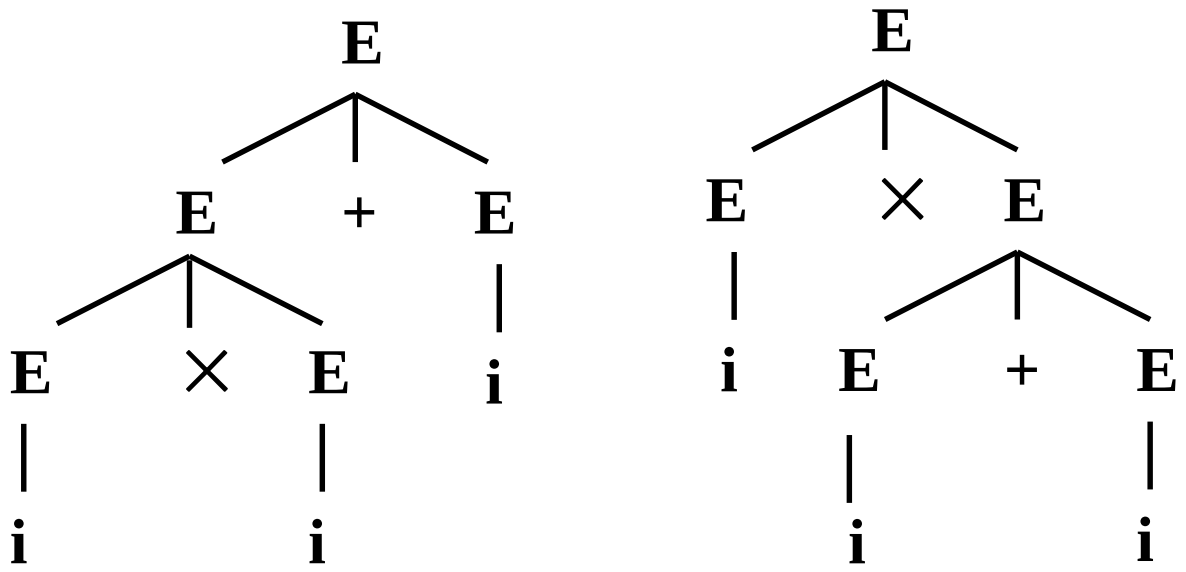


1. 语法分析[Syntax Analysis]

- 复习——准备工作
  - 二义性
    - ✓ 如果一个文法存在某个句子对应**两棵不同的语法树**，则说这个文法是二义的
    - ✓ 二义性文法存在某个句子，它有**两个不同的最左（右）推导**

文法G：E→E+E|E×E|（E）|i

句子 i×i+i 对应的语法树



存在两个不同的最左推导：

推导1：$\underline{E} \Rightarrow \underline{E}+E \Rightarrow \underline{E}\times E+E \Rightarrow i\times\underline{E}+E \Rightarrow i\times i+\underline{E} \Rightarrow i\times i+i$

推导2：$\underline{E} \Rightarrow \underline{E}\times E \Rightarrow i\times\underline{E} \Rightarrow i\times\underline{E}+E \Rightarrow i\times i+\underline{E} \Rightarrow i\times i+i$

1. 语法分析[Syntax Analysis]

- 复习——准备工作
  - 句型分析
    - ✓ **自顶向下分析法 (Top-Down parsing)**
      - **从文法的开始符号出发**，**反复使用文法的产生式**，寻找与输入符号串匹配的推导。
      - 将文法的**开始符号作为语法树的根**，**向下**逐步建立语法树，使语法树的末端结点符号串正好是输入符号串。
    - ✓ **自底向上分析法 (Bottom-Up parsing)**
      - **从输入符号串开始**，逐步进行**归约**，直至归约到文法的开始符号。
      - 从输入符号串开始，以它作为**语法树的末端结点符号串**，**自底向上**的构造语法树。

2. 自顶向下语法分析[Top-Down parsing]

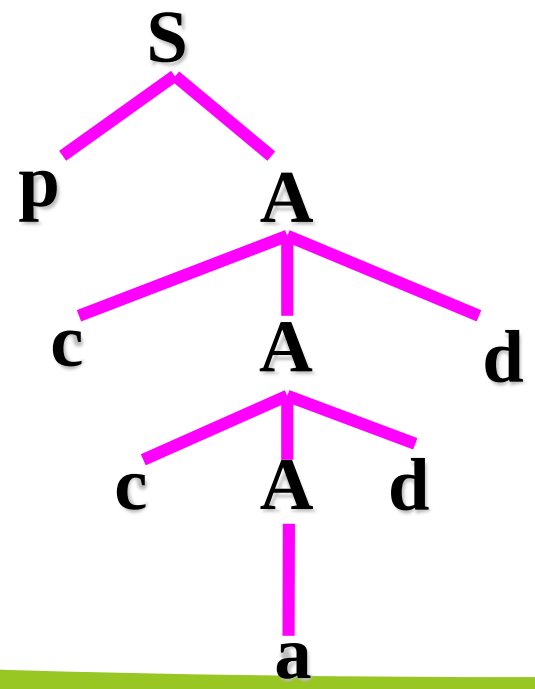
- 例：设有文法G[S]:

  S→pA|qB

  A→cAd|a

  B→dB|b

若输入串W=pccadd。自顶向下的推导过程为:



该推导过程是确定的！

原因:

(1) 每个产生式的右部由终结符开头;

(2) 同一个非终结符的不同产生式的右部由不同的终结符开头。

因此，在推导过程中可以根据当前的输入符号**唯一确定选哪个产生式**往下推导，分析过程是确定的。

S =>pA =>pcAd =>pccAdd =>pccadd

2. 自顶向下语法分析[Top-Down parsing]

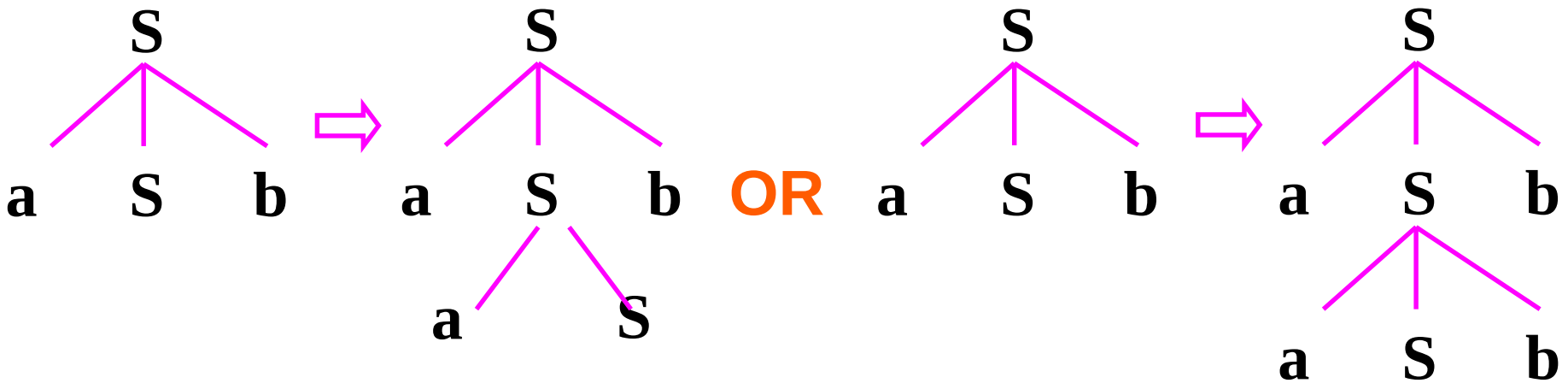
- 例：设有文法G[S]:

  S→aSb

  S→aS

  S→ε

若输入串W=aab。自顶向下的推导过程为:



该推导过程是不确定的！

原因：

同一个非终结符的不同产生式的右部由相同的终结符开头——**存在左公因子**。

因此，要设法**消除左公因子**。

2. 自顶向下语法分析[Top-Down parsing]

- 例：设有文法G[E]:

  E→E+T|T

  T→T*F|F

  F→(E)|id

若输入串W=id+id。自顶向下的推导过程为:

E=>E+T=>E+T+T=>E+T+T+T

该推导过程进入无限循环！

原因:

文法中存在**左递归**的产生式，

导致对该产生式的无限调用。

因此，要设法**消除左递归**。★

为什么不选择产生式E→T，而是一直递归调用E→E+T?

只有当输入串得到匹配时，向前看符号才会发生改变，而调用第一次E→E+T后，输入串id+id并没有得到任何匹配，因此，向前看符号未发生改变，第2次调用E时与第1次调用时采用相同的动作，故而陷入无限循环。

2. 自顶向下语法分析[Top-Down parsing]

- 例：设有文法G[S]:

  S→Sa

  S→b

若输入串W=baaaa。

则自顶向下分析时：

若直接采用S→b：S=>b，无法匹配输入串；

若采用S→Sa：S=>Sa=>Saa...，无法确定何时停止调用S→Sa，使用S→b。

该推导过程是不确定的！

原因：

文法中存在**左递归**的产生式，

导致对该产生式的无限调用。

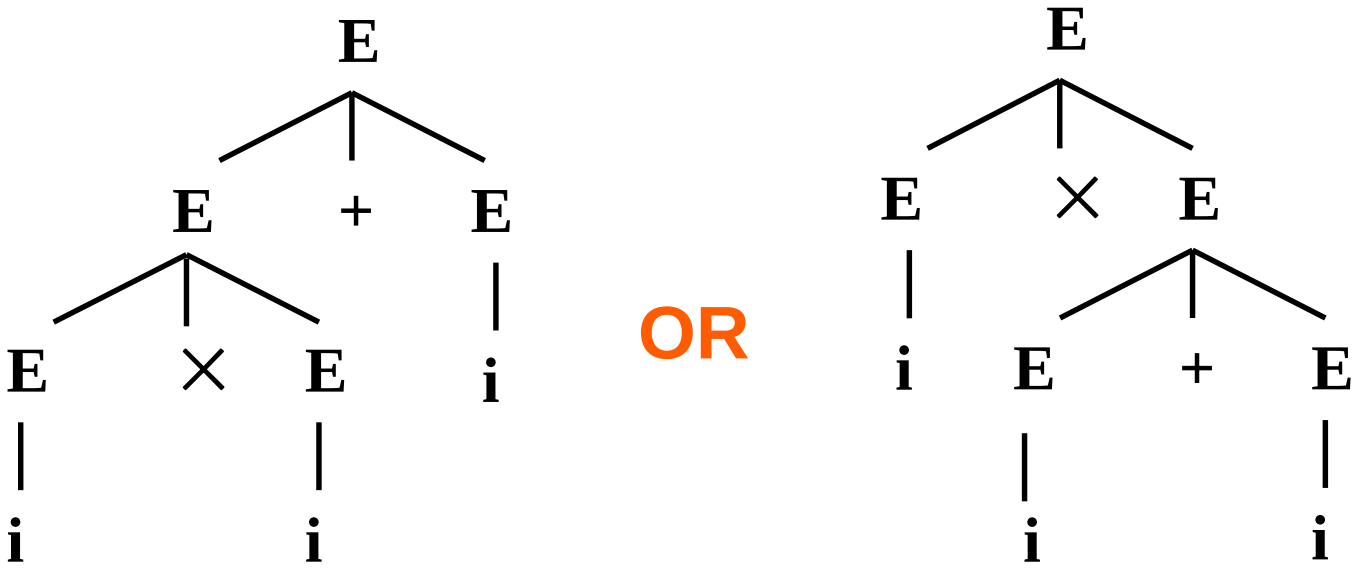
因此，要设法**消除左递归**。★

2. 自顶向下语法分析[Top-Down parsing]

- 例：设有文法G[E]:

E→E+E|E×E|（E）|i

若输入串W=i×i+i。自顶向下的推导过程为:



存在两个不同的最左推导:

推导1：$\underline{E} \Rightarrow \underline{E}+E \Rightarrow \underline{E}\times E+E \Rightarrow i\times\underline{E}+E \Rightarrow i\times i+\underline{E} \Rightarrow i\times i+i$

推导2：$\underline{E} \Rightarrow \underline{E}\times E \Rightarrow i\times\underline{E} \Rightarrow i\times\underline{E}+E \Rightarrow i\times i+\underline{E} \Rightarrow i\times i+i$

该推导过程是不确定的!

原因:

文法**存在二义性**，推导过程本身不唯一。

因此，要设法**消除二义性**。★

3. 改写文法——消除不确定性/无限循环

(1) 提取左公因子[Left Factoring]

– 规则：

✓ **提取左公因子**，将产生式A→**α**β|**α**r等价变换为：A→**α**(β|r)，

✓ 将括号内用一**新引入的非终结符**A’表示，得：A→αA’，A’→β|r

将做出决定的时间往后延！

– 一般形式：

✓ 若A→**α**β_1|**α**β_2|…|**α**β_n，提取左公共因子后变为：A→**α**(β_1|β_2|…|β_n)，

✓ **引进新的非终结符A’**，得：A→αA’，A’→ β_1|β_2|…|β_n

✓ 若在β_i中仍含有左公共因子，可再次提取。

例：文法G[S]：S→**aS**b|**aS**|ε

✓ **提取左公因子**得：S→aS(b|ε)|ε

✓ **引进新的非终结符S’**得：S→aSS’|ε，S’ →b|ε

随堂练习（1）

- 提取左公因子
  - 对文法G[S]：S→if E then S | if E then S else S | other，E → bool提取左公因子

✓ **提取左公因子**得：S→if E then S(ε|else S) | other

✓ **引进新的非终结符S'**得：S→if E then SS' | other

S'→ε|else S

E → bool

3. 改写文法——消除不确定性/无限循环

(2) 消除左递归[Eliminating Left Recursions]

– 直接左递归

✓ 文法G[E]：**E**→**E**+T|T，**T**→**T***F|F，F→(E)|n

– 间接左递归

✓ 文法G[S]：**S**→**Q**c|c，**Q**→**R**b|b，**R**→**S**a|a

3. 改写文法——消除不确定性/无限循环

(2) 消除左递归

① 消除直接左递归

- 规则:
  - ✓ **引入新的非终结符A'**，将产生式**A**→**A**α|β改写成：A→βA'，A'→αA'|ε
- 为什么能确保等价性?
  - ✓ 原产生式**A**→**A**α|β的语言：**β,βα,βαα,...**，即β后跟零个或多个α，隐式终止递归(不再调用A→Aα时)
  - ✓ **现产生式A→βA'，A'→αA'|ε能产生同样的语言**，变换成右递归，且利用ε产生式显式终止递归
- 一般形式:
  - ✓ 若**A**→**A**α_1|**A**α_2|...|**A**α_m|β_1|β_2|...|β_n，则消除左递归后改写成:

    $A\rightarrow\beta_1A'|\beta_2A'|...|\beta_nA'$　　$A'\rightarrow\alpha_1A'|\alpha_2A'|...|\alpha_mA'|\varepsilon$

3. 改写文法——消除不确定性/无限循环

(2) 消除左递归

① 消除直接左递归

例：文法G[E]：**E**→**E**+T|T，**T**→**T***F|F，F→(E)|n

- **引入E'**，E→E+T|T去除左递归得：E→TE'，E'→+TE'|ε
- **引入T'**，T→T*F|F去除左递归得：T→FT'，T'→*FT'|ε
- 最终：E→TE'，E'→+TE'|ε，T→FT'，T'→*FT'|ε，F→(E)|n

3. 改写文法——消除不确定性/无限循环

(2) 消除左递归

② 消除间接左递归

– 规则：把间接左递归变成直接左递归，再加以消除。

– 具体步骤：

✓ 把文法的所有非终结符**按任一顺序排列**，如：A_1，A_2，…，A_n

✓ 从A_1开始，按以下顺序处理A_i：

- 若左部为A_i的产生式的右部为非终结符A_j(**j<i**)开头，即$A_i \rightarrow A_j \ldots$，则**用左部为$A_j$的所有产生式的右部分别代替$A_i \rightarrow A_j \ldots$ 中的A_j**；
- 得到的左部为A_i的产生式若有直接左递归，则消除之。

✓ **去掉无用产生式**。

3. 改写文法——消除不确定性/无限循环

(2) 消除左递归

② 消除间接左递归

例：文法G[S]：(1)**S→Q**c|c　(2)**Q→R**b|b　(3)**R→S**a|a

- 将非终结符**排序 ： R，Q，S**
- **对R**：产生式(3)不含直接左递归，所以保持不变
- **对Q**：把(3)代入(2)得**(2')Q→Sab|ab|b**，无直接左递归
- **对S**：把(2')代入(1)得**(1')S→Sabc|abc|bc|c**，

  有直接左递归，消除直接左递归得： **S→abcS'|bcS'|cS'　S'→abcS'|ε**
- 处理结果为：R→Sa|a，Q→Sab|ab|b，S→abcS'|bcS'|cS'，S' →abcS'|ε
- 由于Q，R是不可到达的非终结符，其产生式应删除
- 最终得文法G'[S]：S→abcS'|bcS'|cS'，S'→abcS'|ε

若非终结符顺序为S, Q, R呢?

消除左递归的最终结果为：

S→Qc|c，Q→Rb|b，

R→bcaR'|caR'|aR'，

R'→bcaR'|ε。

当非终结符的排序不同时，结果的产生式形式不同，但它们是等价的。

随堂练习（2）

- 消除左递归
  - 对文法G[S]: S→Aa|b，A→Ac|Sd|ε消除左递归
    - ✓ 将非终结符**排序 ： S，A**
    - ✓ **对S**：无直接左递归，S→Aa|b保持不变
    - ✓ **对A**：用S的产生式替换A→Sd中的S：A→Ac|Aad|bd|ε
    - ✓ **引入A’，消除直接左递归**：A→bdA’|A’，A’→cA’|adA’|ε，S→Aa|b

3. 改写文法——消除不确定性/无限循环

(3) 消除二义性[Resolving Ambiguities]

例：文法G[stmt]:(**Dangling-else**)

stmt → **if** *expr* **then** *stmt*

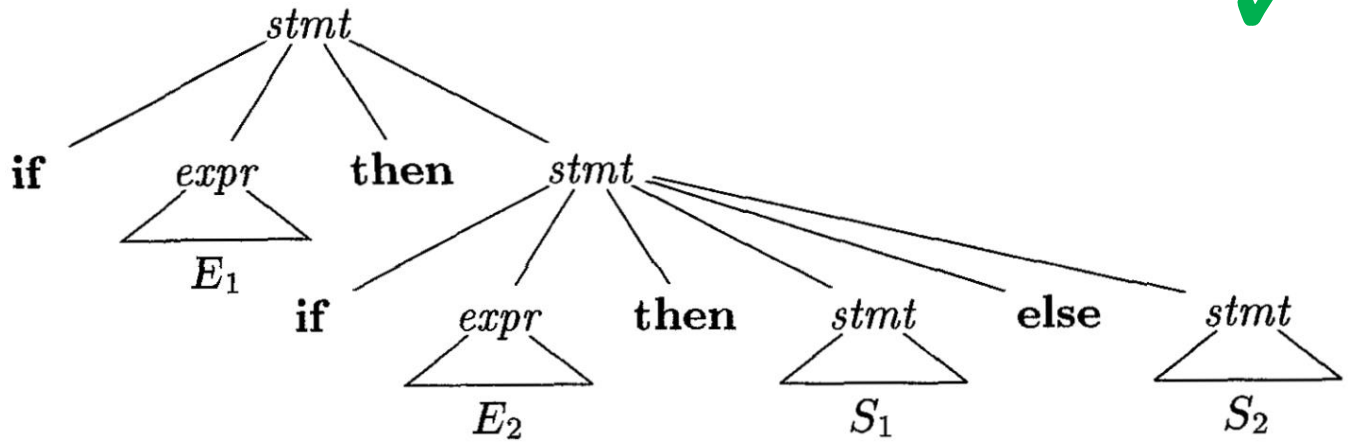
| **if** *expr* **then** *stmt* **else** *stmt*

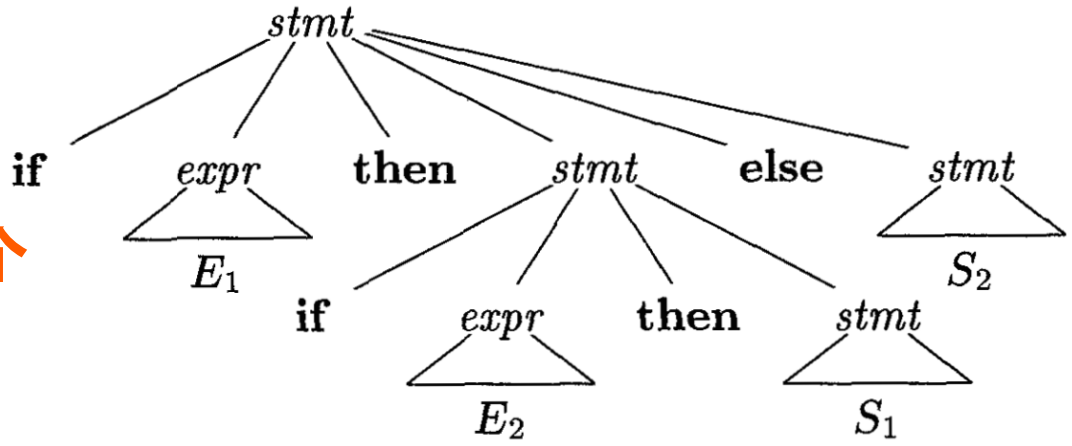
| **other**

– 存在二义性：

- ✓ if E1 then **if E2 then S1 else S2**
- ✓ if E1 then **if E2 then S1** else S2
- ✓ 可通过施加额外的规则来消除二义性，如“**每个else和最近的尚未匹配的then匹配**”
- ✓ 更好的做法是**改写成等价的无二义性的文法**

if E1 then S1 else **if E2 then S2 else S3** ✓





3. 改写文法——消除不确定性/无限循环

(3) 消除二义性

例：文法G[stmt]:(**Dangling-else**)

stmt → **if** *expr* **then** *stmt*

| **if** *expr* **then** *stmt* **else** *stmt*

| **other**



stmt → *matched_stmt* | *open_stmt*

matched_stmt → **if** *expr* **then** ***matched_stmt*** **else** *matched_stmt* **(完整)**

| other

open_stmt → **if** *expr* **then** *stmt* **(开放)**

| **if** *expr* **then** ***matched_stmt*** **else** *open_stmt*

改写原则：

- ✓ 在一个then和一个else**之间出现的语句必须是“已匹配的”**，不能以一个尚未匹配的(开放的)then结尾(没有对应的else)
- ✓ 已匹配的语句：要么是一个不包含开放语句的**完整的if-then-else语句**，要么是一个非条件语句
- ✓ 开放语句：以if-then结尾，没有对应的else

if E1 then if E2 then S1 else S2

3. 改写文法——消除不确定性/无限循环

(4) 消除ε产生式[Eliminating ε-Productions]

– 规则：

- ✓ **将每个产生式右部的非终结符均替换成ε，如果该非终结符能推出ε的话**
- ✓ **把所有可能性都替换出来**
- ✓ 对每个产生式$A \rightarrow X_1X_2...X_n$，其中$X_i \in \Sigma \cup N$，$1 \le i \le n$
- ✓ 增加新产生式$A \rightarrow a_1a_2...a_n$，其中：
  - **若$X_i \Rightarrow^* \varepsilon$，则$a_i=X_i|a_i=\varepsilon$；**
  - **若$X_i \not\Rightarrow^* \varepsilon$，则$a_i=X_i$**
  - $1 \le i \le n$，$a_i \neq \varepsilon$

例：文法G[S]：S→Aa|b，A→Ac|Sd|**ε**

改写为：S→Aa|**a**|b，A→Ac|**c**|Sd

随堂练习（4）

- 消除ε产生式
  - 对文法G[S]：S→aSbS|bSaS|ε消除ε产生式

改写为：

S'→S|**ε**

S→aSbS|abS|aSb|ab|bSaS|baS|bSa|ba，

注意：若由开始符号S→ε，则应补充S'→S|ε

4. 确定的自顶向下语法分析[predictive]

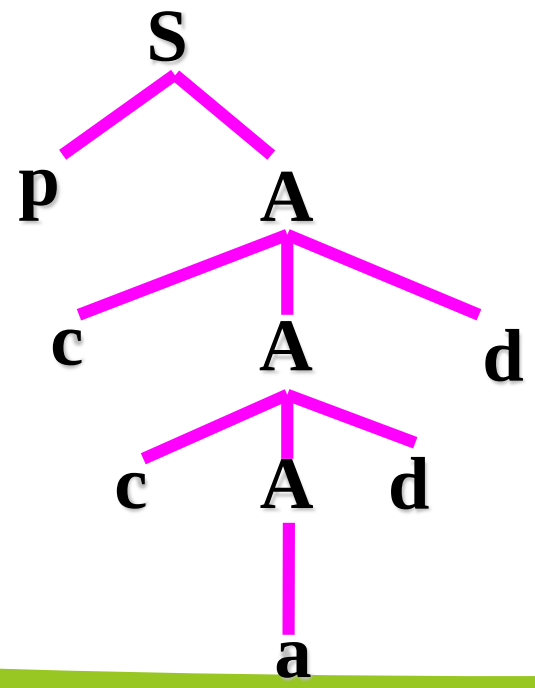
- 例：设有文法G[S]:

  S→pA|qB

  A→cAd|a

  B→dB|b

若输入串W=pccadd，自顶向下的推导过程为:



该推导过程是确定的！

原因:

(1) 每个产生式的右部由终结符开头

(2) 同一个非终结符的不同产生式的右部由不同的终结符开头。

因此，在推导过程中可以根据当前的输入符号**唯一确定选哪个产生式**往下推导，分析过程是确定的。

S =>pA =>pcAd =>pccAdd =>pccadd

4. 确定的自顶向下语法分析[predictive]

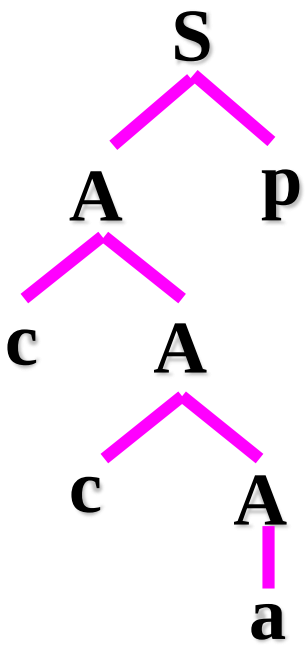
- 例：设有文法G[S]:

  S→Ap|Bq

  A→a|cA

  B→b|dB

  若输入串W=ccap，自顶向下的推导过程为：



S =>Ap =>cAp =>ccAp =>ccap

该推导过程可能是确定的

当：

(1) 产生式右部以终结符或非终结符开头（**无空产生式**）；

(2) **同一非终结符的不同产生式的右部由不同的符号开头**。

对于这种文法，在推导过程选用哪个产生式不直观，关键是判断产生式右部推出的**开始符号集——FIRST集**，分析过程可能是确定的。

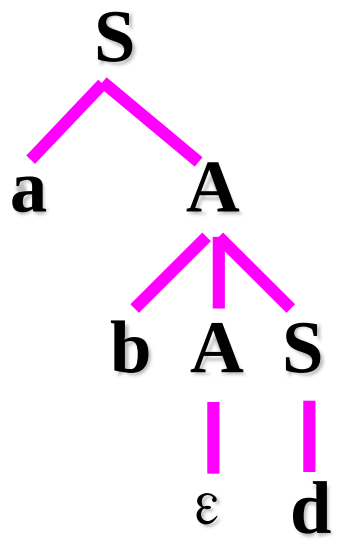
4. 确定的自顶向下语法分析[predictive]

- 例：设有文法G[S]

S→aA|d

A→bAS| ε

若输入串W=abd，自顶向下的推导过程为：



该推导过程可能是确定的

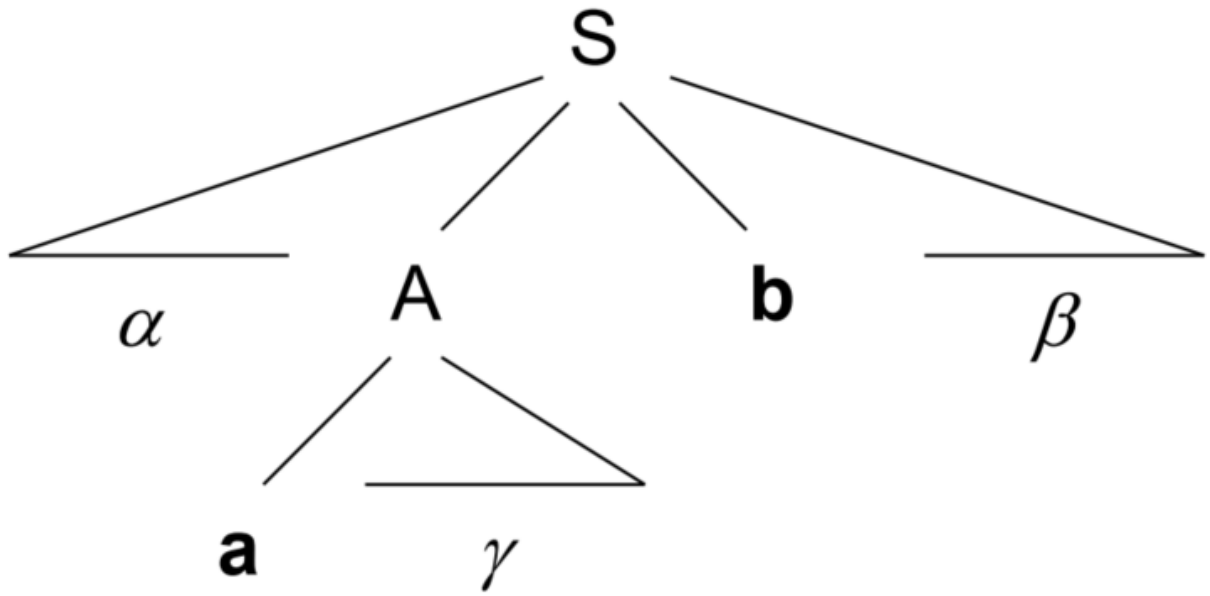
文法的特点是：**包含空产生式**。对于空产生式左部的非终结符，关键是判断该非终结符的**后跟符号集——FOLLOW集**，分析过程可能是确定的。

S =>aA =>abAS =>abS =>abd

4. 确定的自顶向下语法分析[predictive]

- **FIRST集**和**FOLLOW集**的直观理解

  a ∈ FIRST(A) and **b** ∈ FOLLOW(A)



- 开始符号集——**FIRST集**
  - 设G=(V_N, V_T, P, S)是上下文无关文法，**β∈(V_N∪V_T)***，
    FIRST(β) = { **a** ∈V_T | β⇒* **a**……}
  - 若β⇒* ε 则规定ε ∈FIRST(β)
  - 直观上说，文法符号串β的**开始符号集是由β推导出的开头的终结符（包括ε）组成的。**

4. 确定的自顶向下语法分析[predictive]

- 开始符号集——**FIRST集**

例：文法G[S]：

S→Ap	**FIRST(Ap)= {a,c}**
S→Bq	**FIRST(Bq)= {b,d}**
A→a	**FIRST(a)= {a}**
A→cA	**FIRST(cA)= {c}**
B→b	**FIRST(b)= {b}**
B→dB	**FIRST(dB)= {d}**

可进行确定的自顶向下分析

由于**同一非终结符的两个产生式**的右部推导出来的**开始符号集不相交**，因此可根据当前输入符属于哪个产生式右部的开始符号集而决定**唯一选哪个产生式进行推导**，可以进行确定的自顶向下分析。

4. 确定的自顶向下语法分析[predictive]

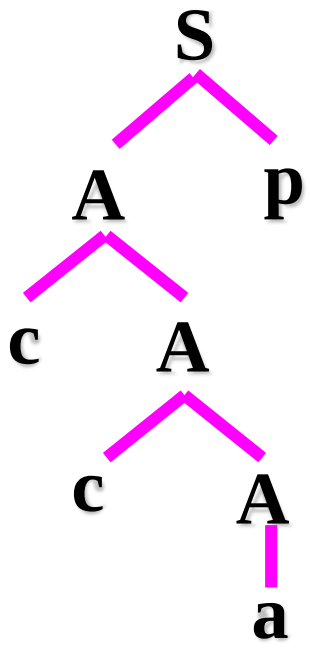
- 开始符号集——**FIRST集**

例：文法G[S]：

S→Ap|Bq

A→a|cA

B→b|dB



若输入串W=ccap,自顶向下的推导过程为：

S→Ap	FIRST(Ap)={a,c}
S→Bq	FIRST(Bq)={b,d}
A→a	FIRST(a)={a}
A→cA	FIRST(cA)={c}
B→b	FIRST(b)={b}
B→dB	FIRST(dB)={d}

S =>Ap =>cAp =>ccAp =>ccap

4. 确定的自顶向下语法分析[predictive]

- 开始符号集——**FIRST集**
  - 在求各个非终结符的FIRST集之前，先确定它们是否能⇒*ε
    - ✓ 第1次扫描——扫描文法中的产生式
      - 对能直接推出ε的产生式左部的终结符标“是”。
      - 删除所有右部含有终结符的产生式。若以某一非终结符为左部的所有产生式都被删除，则该非终结符不能推出ε，将其标为“否”。

例：G[S]

S→AB
~~S→bC~~
~~A→b~~
A→ε
~~B→aD~~

B→ε
C→AD
~~C→b~~
~~D→aS~~
~~D→c~~

非终结符	S	A	B	C	D
第1次扫描		是	是		否
第2次扫描					

4. 确定的自顶向下语法分析[predictive]

- 开始符号集——**FIRST集**
  - 在求各个非终结符的FIRST集之前，先确定它们是否能⇒*ε
    - ✓ 第2次扫描——扫描产生式右部的符号
      - 对每个产生式p: $A \rightarrow X_1 \ldots X_n$，如果$X_1,\ldots,X_n$都被标为“是”(即$X_1,\ldots,X_n$都能推出ε)，则A也能推出ε，将其标为“是”。
      - 如果$A \rightarrow Y_1 \ldots Y_n$中，$Y_1 \ldots Y_n$中任一个已被标为“否”，则删掉该产生式，若这么做使得A的所有产生式都被删去，则A不能推出ε，将其标为“否”。

S→AB　　B→ε
~~S→bC~~　　~~C→AD~~
~~A→b~~　　~~C→b~~
A→ε　　~~D→aS~~
~~B→aD~~　　~~D→c~~

非终结符	S	A	B	C	D
第1次扫描		是	是		否
第2次扫描	是			否	

4. 确定的自顶向下语法分析[predictive]

- 开始符号集——**FIRST集**
  - 对**每一文法符号X**($X \in V_T \cup V_N$)，求**FIRST(X)**的算法:
    - ✓ 对每个$a \in V_T$：FIRST(a)={a}
    - ✓ 对每个$A \in V_N$：若$A \Rightarrow^* \varepsilon$，则$\varepsilon \in$ FIRST(A)
    - ✓ 对每个$A \in V_N$：若$A \rightarrow a\cdots$，$a \in V_T$，则$a \in$FIRST(A)
    - ✓ 若$X, Y_1, Y_2, \ldots, Y_n$都$\in V_N$，有产生式$X \rightarrow Y_1 Y_2 \ldots Y_n$，当$Y_1, Y_2, \ldots, Y_{n-1}$都$\Rightarrow^* \varepsilon$时，$FIRST(Y_1)-\{\varepsilon\}, FIRST(Y_2)-\{\varepsilon\}, \ldots, FIRST(Y_{n-1})-\{\varepsilon\}, FIRST(Y_n)$都包含在FIRST(X)中
    - ✓ 当所有$Y_i \Rightarrow^* \varepsilon$，则$FIRST(X) = FIRST(Y_1) \cup FIRST(Y_2) \cup \ldots \cup FIRST(Y_n) \cup \{\varepsilon\}$

4. 确定的自顶向下语法分析[predictive]

- 开始符号集——**FIRST集**

例G[S]:

S→AB　S→ bC

A→b　　A→ ε

B→aD　B→ ε

C→AD　C→ b

D→aS　D→ c

已求出能推出ε的非终结符集为**{A,B,S}**

	First集(0)	First集(1)	First集(2)	First集(3)
S	ε	ε b	ε b a	ε b a
A	ε	ε b	ε b	ε b
B	ε	ε a	ε a	ε a
C		b	b a c	b a c
D		a c	a c	a c
a	a	a	a	a
b	b	b	b	b

4. 确定的自顶向下语法分析[predictive]

- 开始符号集——**FIRST集**
  - 利用求出每个文法符号的FIRST集求符号串的FIRST集，设$\alpha=X_1X_2...X_n$:
    - ✓ 当X_1不能=>* ε，则$FIRST(\alpha)=FIRST(X_1)$
    - ✓ 若对任何j($1\le j < n$）都有$\varepsilon\in FIRST(X_j)$，则$FIRST(\alpha)=(FIRST(X_1) - \{\varepsilon\}) \cup ... \cup (FIRST(X_j) - \{\varepsilon\}) \cup FIRST(X_{j+1})$
    - ✓ 若对所有i ($1\le i\le n$)，都有$\varepsilon\in FIRST(X_i)$，则 $FIRST(\alpha)=FIRST(X_1) \cup ... \cup FIRST(X_n) \cup \{\varepsilon\}$

4. 确定的自顶向下语法分析[predictive]

- 开始符号集——**FIRST集**

例：G[S]

S→AB|bC
A→b|ε
B→aD|ε
C→AD|b
D→aS|c

已求出非终结符的First集合如下:

First(S)={a,b, ε}　First(A)={b, ε}　First(B)={a, ε}
First(C)={a,b,c}　First(D)={a,c}

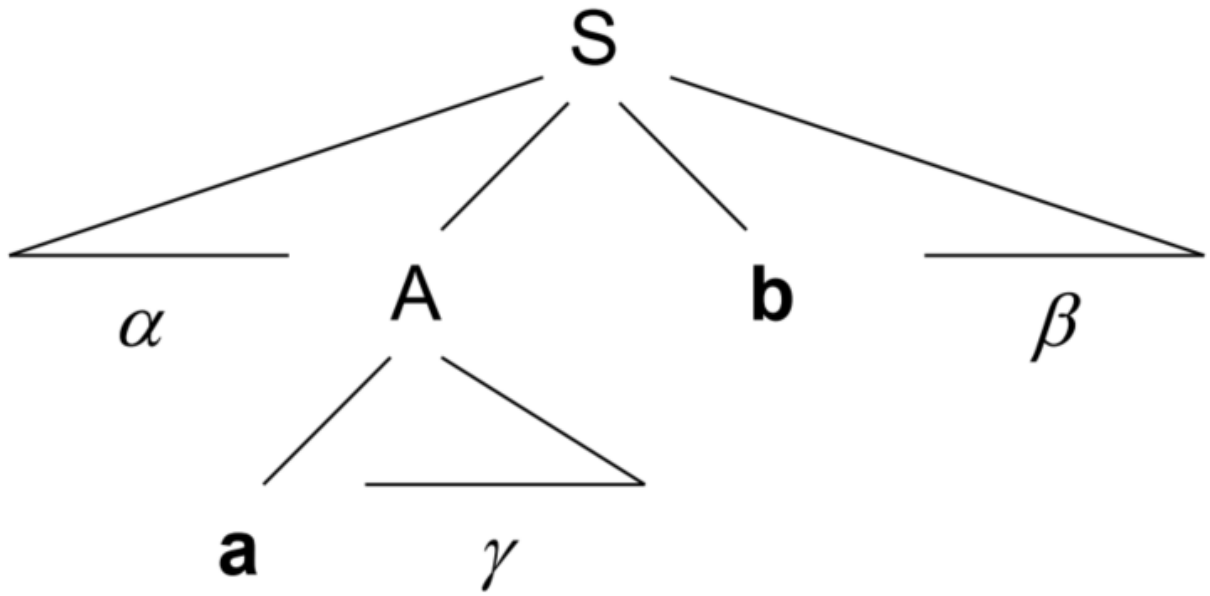
产生式右部符号串的开始符集合为:

S→AB　FIRST(AB)=FIRST(A)∪FIRST(B)∪{ε}={a,b,ε}
S→bC　FIRST(bC)={b}
A→ε　FIRST(ε)={ε}
A→b　FIRST(b)={b}
C→AD　FIRST(AD)=(FIRST(A)-{ε})∪FIRST(D) ={b,a,c}
C→b　FIRST(b)={b}
D→aS　FIRST(aS)={a}
D→c　FIRST(c)={c}

4. 确定的自顶向下语法分析[predictive]

- **FIRST集**和**FOLLOW集**的直观理解

  a ∈ FIRST(A) and **b** ∈ FOLLOW(A)



- 后跟符号集——**FOLLOW集**
  - 设G=(V_T, V_N, P, **S**)是上下文无关文法，

    A∈V_N，FOLLOW(**A**)={**b**|**S**=>*…**A**b…，**b** ∈V_T}，
  - 若有**S**=>* …A，则规定 $ ∈FOLLOW(A)

    (注：‘**$**’**做为输入串的结束符**)
  - 直观上说，非终结符A的**后跟符号集是由句型中紧跟A后的那些终结符（包括$）组成。**

4. 确定的自顶向下语法分析[predictive]

- 后跟符号集——**FOLLOW集**

例：文法G[S]:

S→aA|d

A→bAS|ε

➢ 由**S=>* aA** 得 **$ ∈FOLLOW(A)**

由**S=>* abAS=>* abAaA** 得 **a ∈FOLLOW(A)**

… =>* abAd 得 **d ∈FOLLOW(A)**

FOLLOW(A)={$,a,d}

➢ 由 **S=>* S** 得 **$ ∈FOLLOW(S)**

由**S=>aA=>abAS=>abbASS=>abbASaA**

…=>abbASd

FOLLOW(S)={$,a,d}

4. 确定的自顶向下语法分析[predictive]

- 后跟符号集——**FOLLOW集**
  - 计算**每个非终结符A的FOLLOW(A)集**

(1) 对所有$A \in V_N$，令FOLLOW(A)={ }; 对开始符号S，令FOLLOW(S)={$}

因为S=>*S，显然$∈FOLLOW(S)

(2) 对每条产生式A→xBy，考察产生式右部的每一非终结符B，x,y∈V*：

- 如果y不能推出ε：FOLLOW(B)=FOLLOW(B)∪First(y)
- 否则，若y ⇒*ε：FOLLOW(B)=FOLLOW(B)∪(First(y)-{ε})∪ FOLLOW(A)

若a∈FOLLOW(A)，则表明S=>*...Aa...，
由于A→xBy，且y=>*ε，则有：S=>*...Aa...=>...xBya=>...xBa...，
即S=>*...xBa...，所以a∈FOLLOW(B)

注意：ε不存在于任何FOLLOW集中

(3) 重复(2)，直至对所有$A \in V_N$，FOLLOW(A)**收敛为止**。

4. 确定的自顶向下语法分析[predictive]

- 后跟符号集——**FOLLOW集**

例G[S]:

[1]S→AB

[2]S→bC

[3]A→b

[4]A→ε

[5]B→aD

[6]B→ε

[7]C→AD

[8]C→b

[9]D→aS

[10]D→c

已求出非终结符的First集合如下:

First(S)={a,b, ε}　First(A)={b, ε}

First(B)={a, ε}　First(C)={a,b,c}

First(D)={a,c}

已求出能推出ε的非终结符集为**{A,B,S}**

	Follow集(0)	Follow集(1)	Follow集(2)
S	$	$	$
A		a $ c	a $ c
B		$	$
C		$	$
D		$	$

随堂练习（5）

- 对文法G[E]：E→TE’，E’→+TE’|ε，T→FT’，T’→*FT’|ε，F→(E)|n，求**每个非终结符的FIRST集和FOLLOW集**

(1) 能产生ε的非终结符：E’，T’

(2)
FIRST(E)={(, n}
FIRST(E’)={+, ε}
FIRST(T)={(, n}
FIRST(T’)={*, ε}
FIRST(F)={(, n}

(3)
FOLLOW(E)={), $}
FOLLOW(E’)={), $}
FOLLOW(T)={+,), $}
FOLLOW(T’)={+,), $}
FOLLOW(F)={*, +,), $}

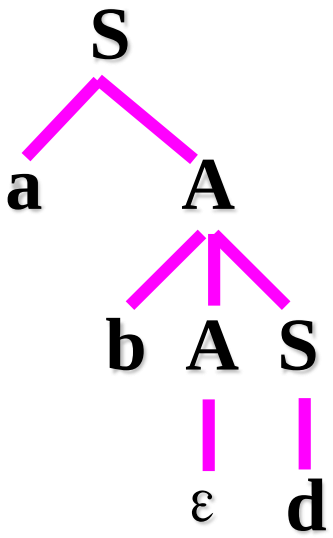
4. 确定的自顶向下语法分析[predictive]

- 例：文法G[S]:

  S→aA|d

  A→bAS|ε

  若输入串W=abd，自顶向下的推导过程为：



FIRST(bAS)={b}

FOLLOW(A)={#,a,d }

可进行确定的自顶向下分析

对于非终结符A的两个产生式A→bAS 和 A→ε:

当输入符号∈FIRST(bAS)={b}时，选A→bAS推导;

当输入符号∈FOLLOW(A)={#,a,d}时，选A→ε推导。

由于**FIRST(bAS)∩FOLLOW(A)=∅**，所以可进行**确定的**自顶向下分析。

S =>aA =>abAS =>abS =>abd

4. 确定的自顶向下语法分析[predictive]

- **LL(1)**文法
  - 含义
    - ✓ 第一个**L**表示**从左到右**扫描输入串
    - ✓ 第二个**L**表示分析过程用**最左推导**
    - ✓ **1**表明只需**向前看一个符号**便可以决定选哪个产生式进行推导，类似地LL(k)文法需要向前看K个符号才可以确定选用哪个产生式。
  - 定义
    - ✓ 一个上下文无关文法是LL(1)文法的充分必要条件是，若存在产生式A→α|β，则：
      - $FIRST(\alpha) \cap FIRST(\beta) = \Phi$
      - $\varepsilon \in FIRST(\beta) \Rightarrow FIRST(\alpha) \cap FOLLOW(A) = \Phi$

两个条件同时满足！

LL(1)文法就能在仅向前查看一个符号的情况下进行确定的自顶向下分析——LL(1)分析

4. 确定的自顶向下语法分析[predictive]

- 递归下降预测分析[Recursive Descent Predictive Parsing]
- LL(1)分析

5. 递归下降预测分析[Recursive Descent Predictive Parsing]

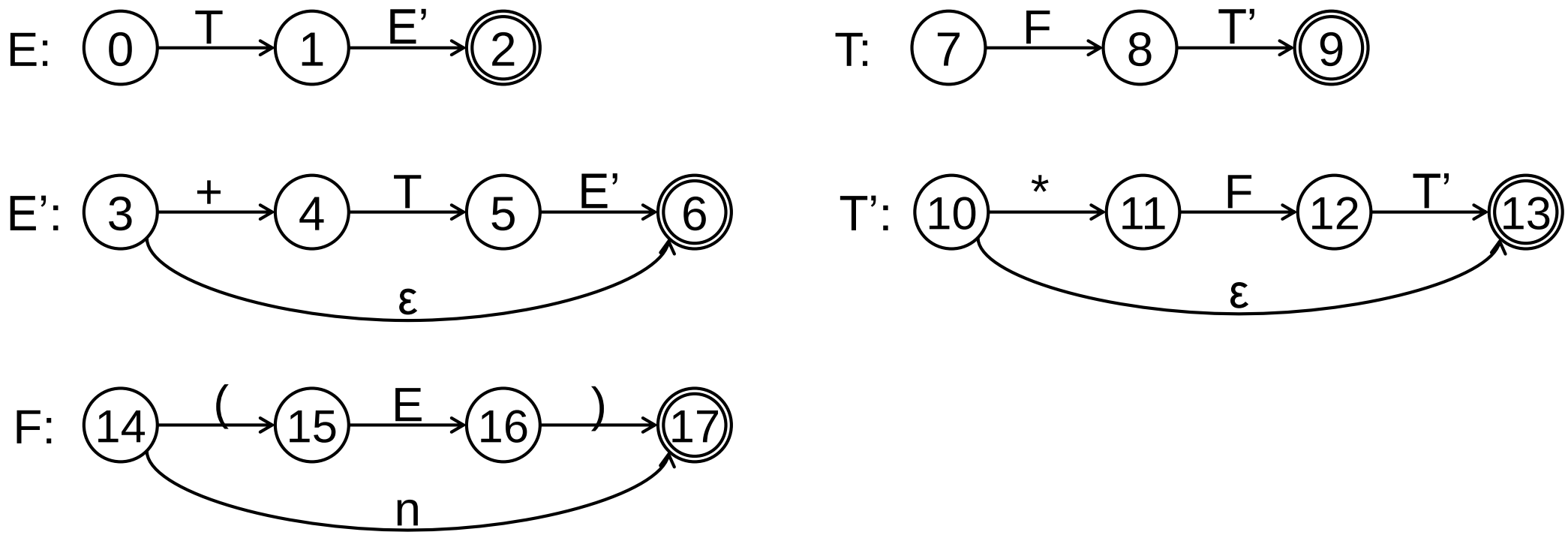
- 递归下降预测分析[Recursive Descent Predictive Parsing]
  - 基于**手写代码**实现，每个非终结符对应一个递归函数
  - 通过函数的递归调用模拟推导过程，**显式地编码产生式的选择**（通常通过if-else或switch分支实现）
  - 可以是LL(1)文法，也可以是更强的**LL(k)（通过向前看更多符号解决冲突）**
- 递归下降预测解析器的开发流程
  - 1. 消除语法中的二义性　　2. 消除语法中的左递归
  - 3. 提取左公因子　　4. 从语法构造转换图
  - 5. 简化转换图　　6. 使用转换图编写解析器作为蓝本

5. 递归下降预测分析[Recursive Descent Predictive Parsing]

- 转换图——可视化预测分析器
  - 对每个非终结符A
    - ✓ 创建初始状态和结束状态
    - ✓ 对于每个$A \rightarrow X_1X_2...X_n$，创建一条从初始状态到最终状态的路径，边标记为$X_1$，$X_2$，…，$X_n$。
    - ✓ 如果$A \rightarrow \varepsilon$：表示路径为ε。

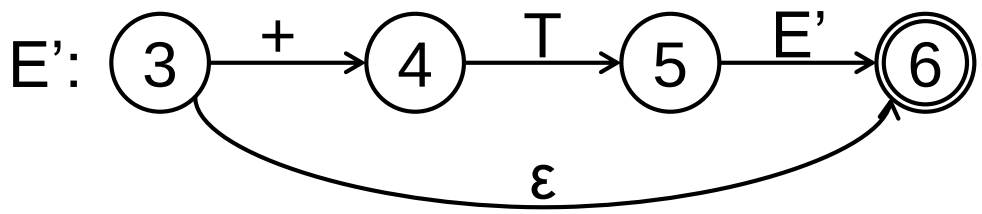
5. 递归下降预测分析[Recursive Descent Predictive Parsing]

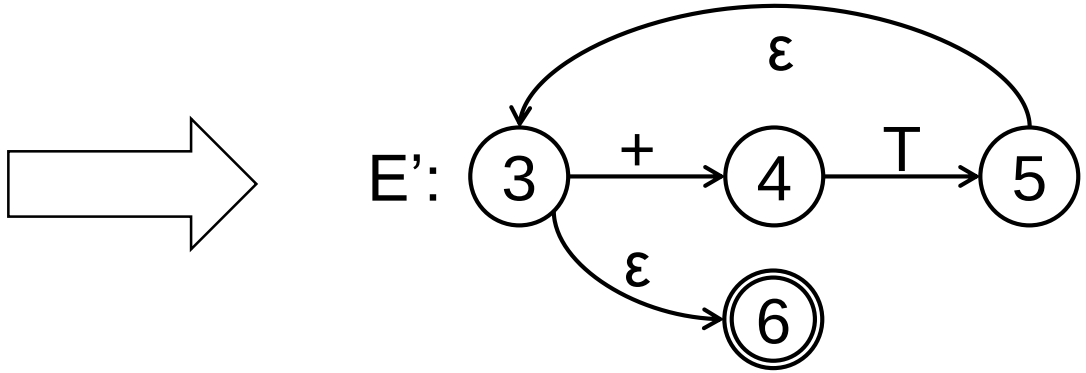
- 转换图——可视化预测分析器
  - 例：对文法G[E]：E→TE’，E’→+TE’|ε，T→FT’，T’→*FT’|ε，F→(E)|n构建转换图

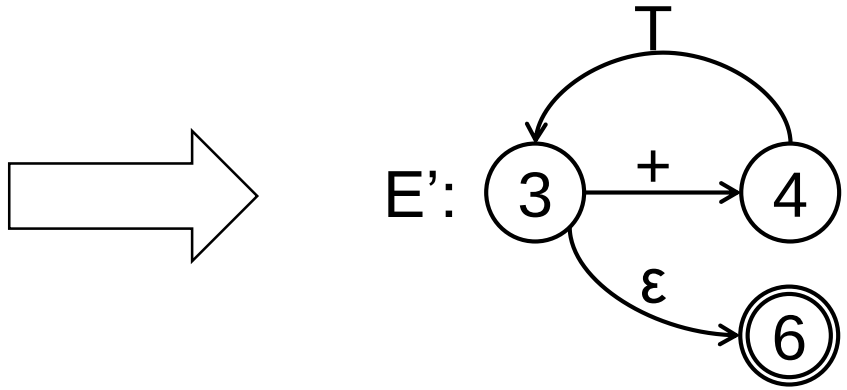


5. 递归下降预测分析[Recursive Descent Predictive Parsing]

- 化简转换图

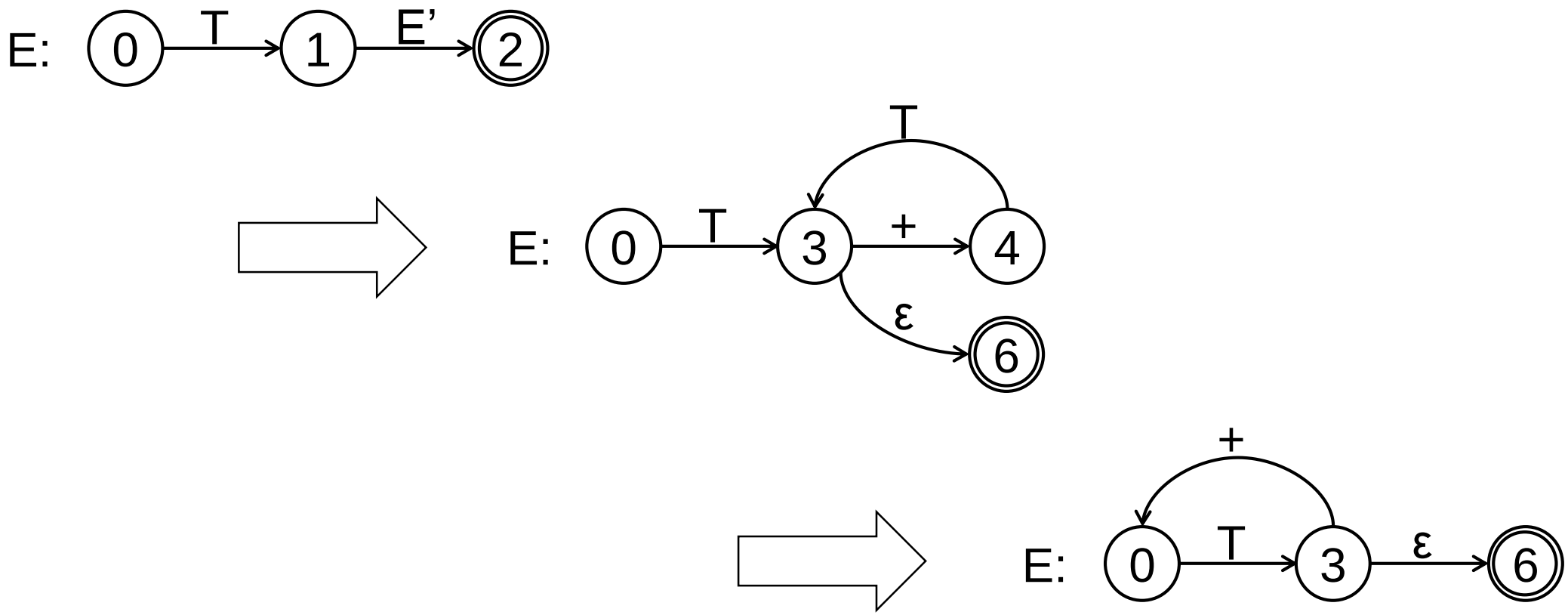






5. 递归下降预测分析[Recursive Descent Predictive Parsing]

- 化简转换图



5. 递归下降预测分析[Recursive Descent Predictive Parsing]

- 化简转换图

文法G[E]:

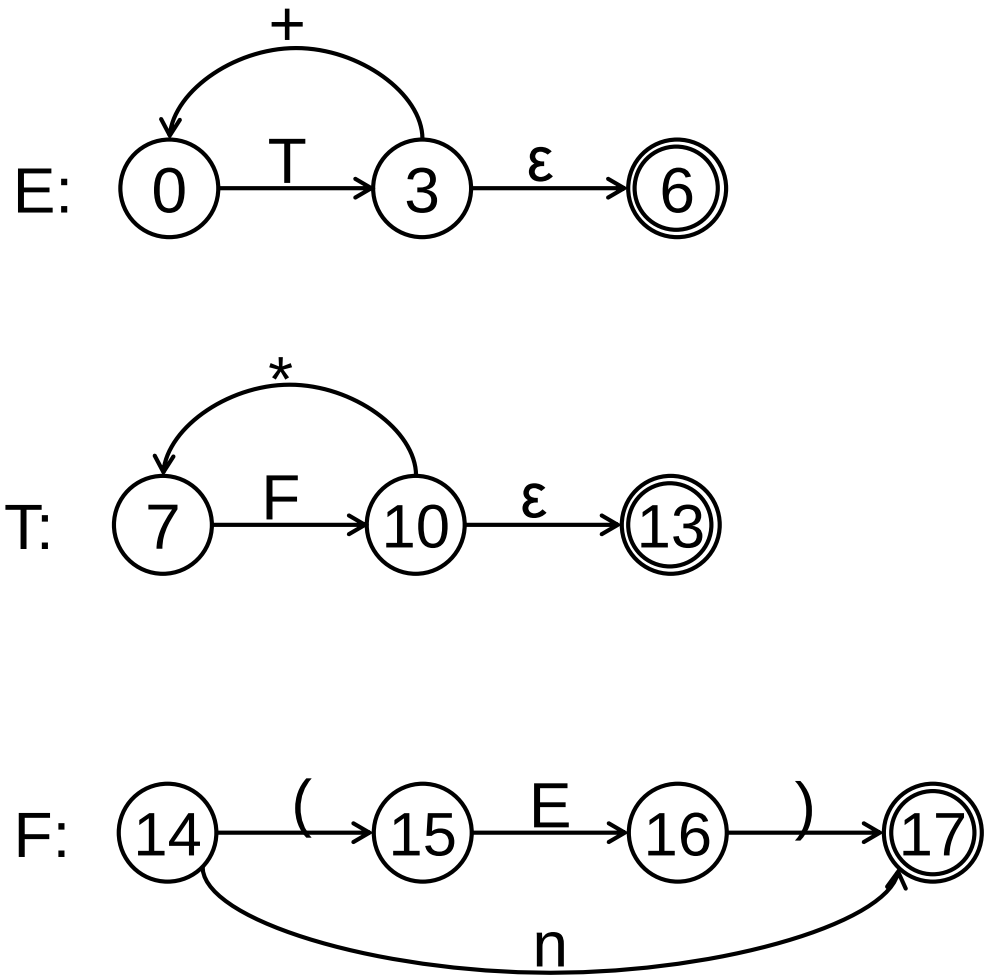
E→TE’

E’→+TE’|ε

T→FT’

T’→*FT’|ε

F→(E)|n

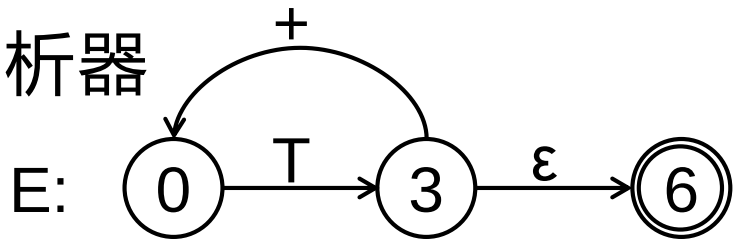


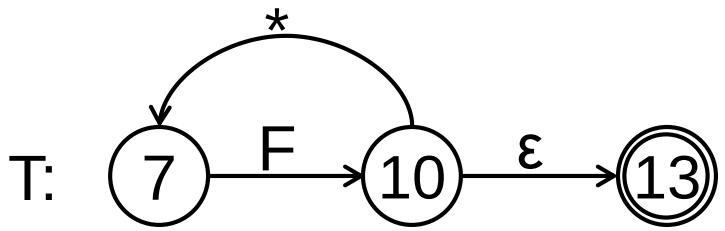
5. 递归下降预测分析[Recursive Descent Predictive Parsing]

- 编写解析器
  - 为每个图编写递归过程，**开始符号S**对应的转换图是程序的主要**main入口**
  - 设计程序的控制流，模仿图中的路径，考虑到前瞻
    - ✓ 如果路径中边的符号等于终结符，则执行**匹配[match]**动作
    - ✓ 如果路径中边的符号是非终结符，则执行**派生[derive]**操作，即调用非终结符的过程（递归）

5. 递归下降预测分析[Recursive Descent Predictive Parsing]

- 编写解析器



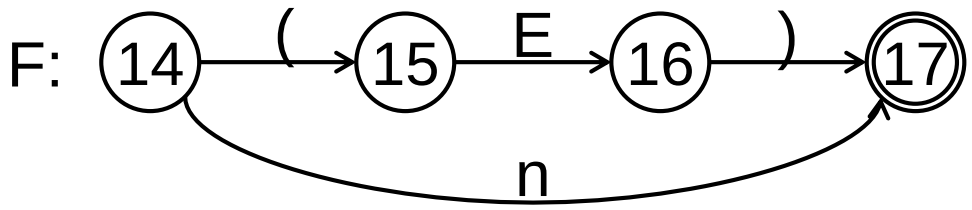



```
void E(){
   if(lookahead in FIRST(T)){
      T();
   }else error();
   while(lookahead=='+'){
      match('+');
      if(lookahead in FIRST(T)){
         T();
      }else error();
   }
   if(lookahead in FOLLOW(E)){
     // do nothing
   }else error();
}
```

```
void T(){
   if(lookahead in FIRST(F)){
      F();
   }else error();
   while(lookahead=='*'){
      match('*');
      if(lookahead in FIRST(F)){
         F();
      }else error();
   }
   if(lookahead in FOLLOW(T)){
     // do nothing
   }else error();
}
```

5. 递归下降预测分析[Recursive Descent Predictive Parsing]

- 编写解析器




```
void F(){
   if(lookahead=='('){
      match('(');
      if(lookahead in FIRST(E)){
         E();
      }else error();
      if(lookahead==')'){
         match(')');
      }else error();
   }else if(lookahead=='n'){
      match('n');
   }else error();
}
```

```
void match(Token tok){
   if(lookahead==tok){
      lookahead=scanner.getNextToken();
   }else error();
}
```

CONTENTS

目 录

01 自顶向下分析 Top-Down Parsing

02 LL(1)分析 LL(1) Parsing

03 自底向上分析 Bottom-Up Parsing

04 LR分析 LR Parsing

1. 前提条件

- LL(1)分析
  - **表驱动**，依赖预先计算的预测分析表
  - 使用统一的算法（栈+表驱动），通过查表决定下一步动作
  - **必须严格满足LL(1)文法规则**

- 一个上下文无关文法是LL(1)文法的充分必要条件是，若存在产生式$A \rightarrow \alpha|\beta$，则：
  - $FIRST(\alpha) \cap FIRST(\beta) = \Phi$
  - $\varepsilon \in FIRST(\beta) \Rightarrow FIRST(\alpha) \cap FOLLOW(A) = \Phi$

两个条件同时满足！

2. 构造预测分析表

- 预测分析表
  - 二维表
  - 元素**M[A,a]**的内容是当**非终结符A**面临**输入符号a**(终结符或结束符$)时应选取的产生式；当无产生式时，元素内容为转向出错处理。

例：文法G[E]:

E→TE’

E’→+TE’|ε

T→FT’

T’→*FT’|ε

F→(E)|n

Non-terminal	lookahead					
	n	+	*	(	)	$
E						
E’						
T						
T’						
F						

2. 构造预测分析表

- 预测分析表构造算法：
  - 对于文法G的每个产生式A→α:
    - ✓ 对于FIRST(α)中的每个终结符a，将A→α填入表M[A,a]中；
    - ✓ 如果ε∈FIRST(α)，则对于FOLLOW(A)中的每个终结符a，将A→α填入表M[A,a]中；如果$∈FOLLOW(A)，也将A→α填入表M[A,$]中。

2. 构造预测分析表

例：文法G[E]：

E→TE’

E’→+TE’|ε

T→FT’

T’→*FT’|ε

F→(E)|n

FIRST(E)={(, n}　FOLLOW(E)={), $}

FIRST(E’)={+, ε}　FOLLOW(E’)={), $}

FIRST(T)={(, n}　FOLLOW(T)={+,), $}

FIRST(T’)={*, ε}　FOLLOW(T’)={+,), $}

FIRST(F)={(, n}　FOLLOW(F)={*, +,), $}

- **对于FIRST(α)中的每个终结符a，将A→α填入表M[A,a]中；**
- **如果ε∈FIRST(α)，则对于FOLLOW(A)中的每个终结符a，将A→α填入表M[A,a]中；如果$∈FOLLOW(A)，也将A→α填入表M[A,$]中。**

Non-terminal	lookahead					
	n	+	*	(	)	$
E	E→TE’			E→TE’		
E’		E'→+TE’			E'→ ε	E'→ε
T	T→ FT’			T→ FT’		
T’		T'→ ε	T'→*FT'		T'→ ε	T'→ε
F	F→ n			F→ (E)		

随堂练习（6）

- 填写以下文法的预测分析表
  - G[S]:S→if E then S S’|other，S’→ε|else S，E→bool

Non-terminal	lookahead					
	if	then	else	other	bool	$
S						
S’						
E						

随堂练习（6）

- 填写以下文法的预测分析表
  - G[S]:S→if E then S S’|other，S’→ε|else S，E→bool

Non-terminal	lookahead					
	if	then	else	other	bool	$
S	S→if E then S S’			S→other		
S’			**S’→ε** **S’→else S**			S’→ε
E					E→bool	

FIRST(if E then S S’)={if}　　FIRST(other)={other}　　FIRST(ε)={ε}
FIRST(else S)={else}　　FIRST(bool)={bool}　　FOLLOW(S’)={else $}

2. 构造预测分析表

- **冲突[conflicts]**
  - **同一个单元格中存在多条产生式**
  - 例：G[S]:S→if E then S S’|other，S’→ε|else S，E→bool

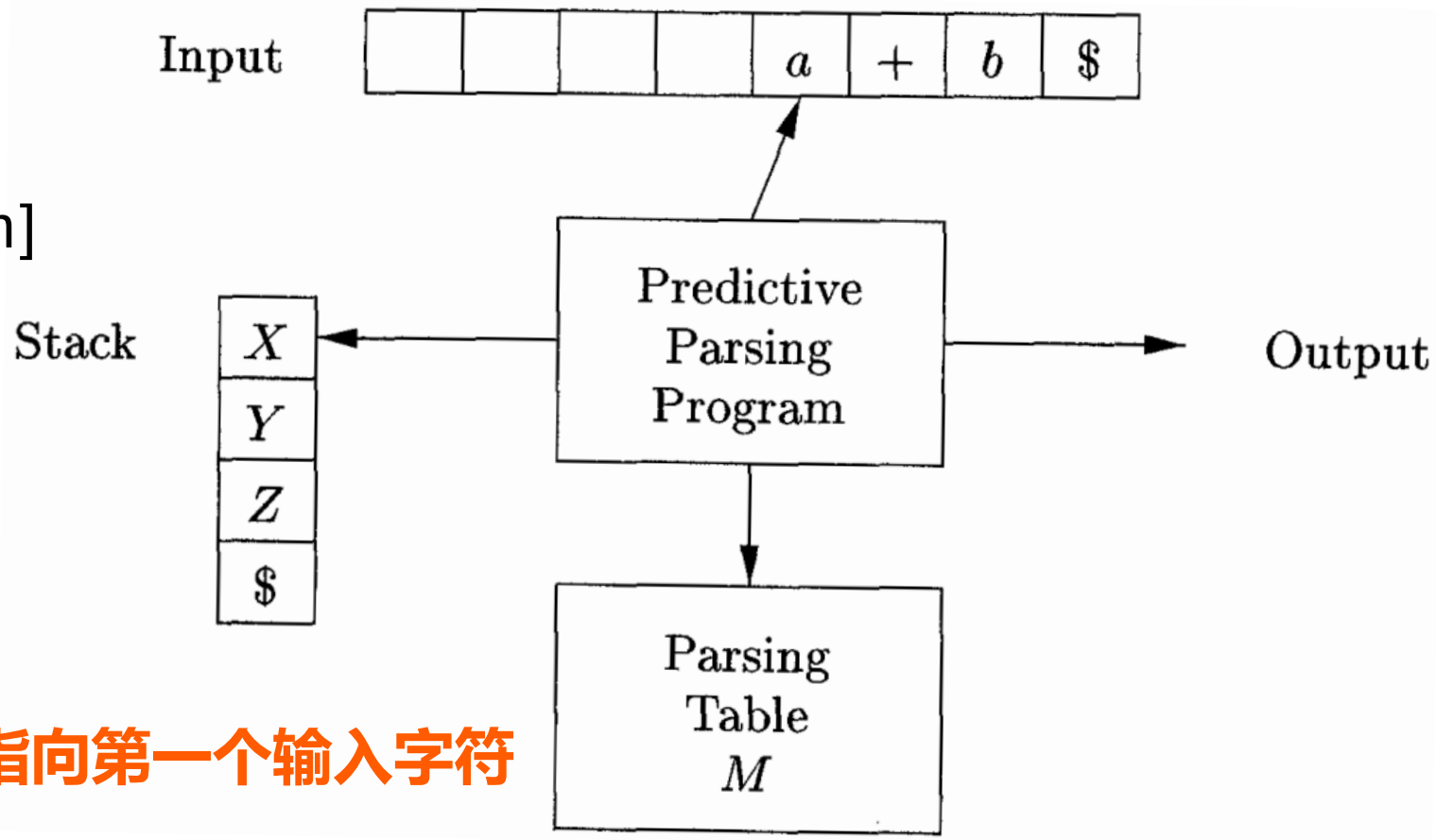
Non-terminal	lookahead					
	if	then	else	other	bool	$
S	S→if E then S S’			S→other		
S’			**S’→ε S’→else S**			S’→ε
E					E→bool	

因存在二义性导致冲突

  - 可能引起冲突的情况：左递归、左公因子、二义性　**应消除这些情况后再建表**

3. Table-Driven Parser

- Parser由预测分析表/解析表[Parsing Table]驱动
- Parser的构成：
  - 预测分析表
  - 解析程序[Parsing Program]
  - 栈[Stack]
  - 输入缓冲区
  - 输出流



初始：

Stack：$S；lookahead：指向第一个输入字符

接受：

Stack：$；lookahead：指向$

3. Table-Driven Parser

- LL(1)预测分析算法

令a=输入符号串w的第一个符号；令X=栈顶符号；

while(X!=$){ //栈非空

if(X==a){ 弹出栈顶符号X；令a指向下一个输入符号； } //a字符匹配

else if(X是一个终结符) error();

else if(M[X,a]是一个报错条目) error();

else if(M[X,a]==$X \rightarrow Y_1 Y_2 \ldots Y_k$){

输出产生式$X \rightarrow Y_1 Y_2 \ldots Y_k$；弹出栈顶符号X；

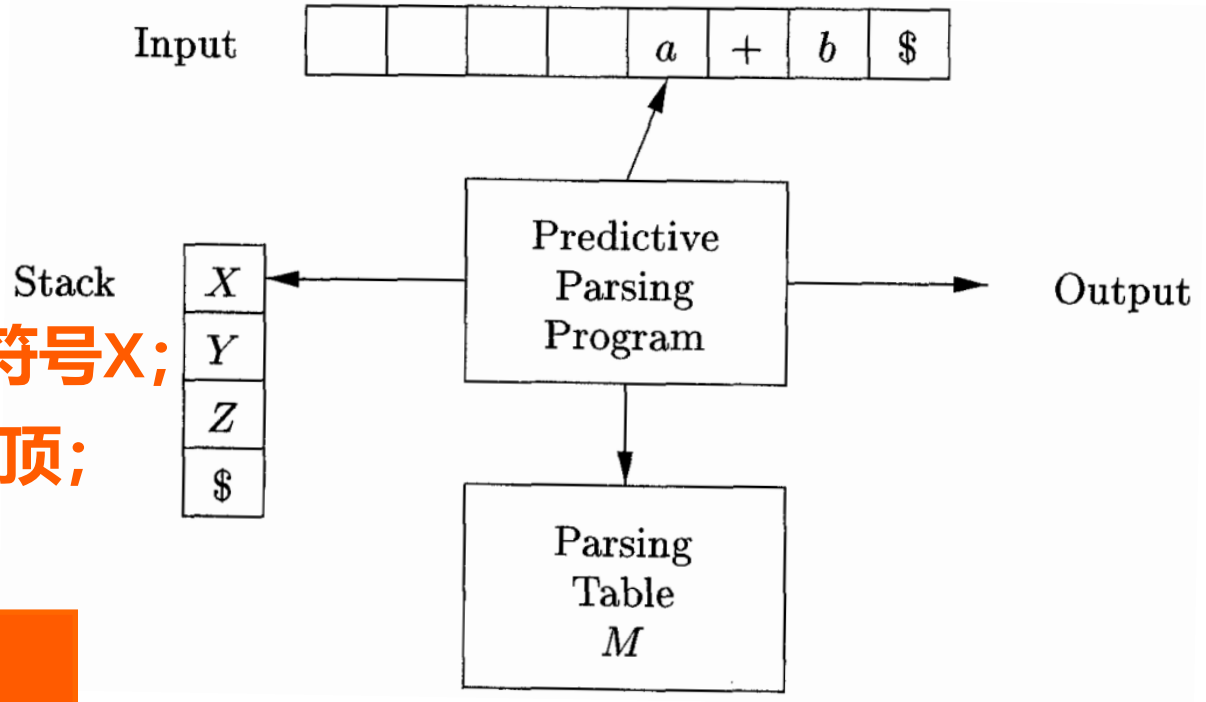
将$Y_1 Y_2 \ldots Y_k$压入栈中，其中Y_1位于栈顶；

}

令X=栈顶符号；

}

反序压栈



3. Table-Driven Parser

- 例:

	n	+	*	(	)	$
E	E→TE'			E→TE'		
E'		E'→+TE'			E'→ ε	E'→ ε
T	T→ FT'			T→ FT'		
T'		T'→ ε	T'→* FT'		T'→ ε	T'→ ε
F	F→ n			F→ (E)		

反序压栈

步骤	分析栈	剩余输入串	动作
1	**$E**	**n+n*n$**	输出 E→TE’
2	$E’T	n+n*n$	输出 T→FT’
3	$E’T’F	n+n*n$	输出 F→n
4	$E’T’n	n+n*n$	n 匹配
5	$E’T’	+n*n$	输出 T’→ ε
6	$E’	+n*n$	输出 E’→+TE’
7	$E’T+	+n*n$	+ 匹配

3. Table-Driven Parser

- 例:

	n	+	*	(	)	$
E	E→TE'			E→TE'		
E'		E'→+TE'			E'→ ε	E'→ ε
T	T→ FT'			T→ FT'		
T'		T'→ ε	T'→* FT'		T'→ ε	T'→ ε
F	F→ n			F→ (E)		

8	$E’T	n*n$	输出 T→FT’
9	$E’T’F	n*n$	输出 F→n
10	$E’T’n	n*n$	n 匹配
11	$E’T’	*n$	输出T’→*FT’
12	$E’T’F*	*n$	* 匹配
13	$E’T’F	n$	输出 F→n
14	$E’T’n	n$	n 匹配
15	$E’T’	$	输出 T’→ ε
16	$E’	$	输出 E’→ ε
17	$	$	ACCEPT

3. Table-Driven Parser

- 例：
  - **缺少操作数**

  如：n+*n

	n	+	*	(	)	$
E	E→TE'			E→TE'		
E'		E'→+TE'			E'→ ε	E'→ ε
T	T→ FT'			T→ FT'		
T'		T'→ ε	T'→* FT'		T'→ ε	T'→ ε
F	F→ n			F→ (E)		

步骤	分析栈	剩余输入串	动作
1	$E	**n+*n**$	输出 E→TE’
2	$E’T	n+*n$	输出 T→FT’
3	$E’T’F	n+*n$	输出 F→n
4	$E’T’n	n+*n$	n 匹配
5	$E’T’	+*n$	输出 T’→ ε
6	$E’	+*n$	输出 E’→+TE’
7	$E’T+	+*n$	+ 匹配
8	$E’T	*n$	**ERROR**

[T, *]=empty

3. Table-Driven Parser

- 例:
  - **缺少操作符**

    如：nn*n

	n	+	*	(	)	$
E	E→TE'			E→TE'		
E'		E'→+TE'			E'→ ε	E'→ ε
T	T→ FT'			T→ FT'		
T'		T'→ ε	T'→* FT'		T'→ ε	T'→ ε
F	F→ n			F→ (E)		

步骤	分析栈	剩余输入串	动作
1	$E	**nn*n**$	输出 E→TE’
2	$E’T	nn*n$	输出 T→FT’
3	$E’T’F	nn*n$	输出 F→n
4	$E’T’n	nn*n$	n 匹配
5	$E’T’	n*n$	**ERROR**

[T’, n]=empty

3. Table-Driven Parser

- 例：
  - **缺少操作符**

    如：n+n)

	n	+	*	(	)	$
E	E→TE'			E→TE'		
E'		E'→+TE'			E'→ ε	E'→ ε
T	T→ FT'			T→ FT'		
T'		T'→ ε	T'→* FT'		T'→ ε	T'→ ε
F	F→ n			F→ (E)		

步骤	分析栈	剩余输入串	动作
1	$E	**n+n)**$	输出 E→TE’
2	$E’T	n+n)$	输出 T→ FT’
3	$E’T’F	n+n)$	输出 F→n
4	$E’T’n	n+n)$	n 匹配
5	$E’T’	+n)$	输出 T’→ ε
6	$E’	+n)$	输出 E’→+TE’
7	$E’T+	+n)$	+ 匹配

步骤	分析栈	剩余输入串	动作
8	$E’T	n)$	输出 T→ FT’
9	$E’T’F	n)$	输出 F→n
10	$E’T’n	n)$	n 匹配
11	$E’T’	)$	输出 T’→ ε
12	$E’	)$	输出 E’→ ε
13	$	)$	**ERROR**

随堂练习（7）

- 对文法G[S]：S→aBa，B→bB|ε，构造预测分析表，并对输入符号串abba及aabb分别进行LL(1)预测分析，写出分析的全过程（表格形式）。

Non-terminal	lookahead		
	a	b	$
S	S→aBa		
B	B→ε	B→bB	

步骤	分析栈	剩余输入串	动作
1	$S	abba$	输出 S→aBa
2	$aBa	abba$	a 匹配
3	$aB	bba$	输出 B→bB
4	$aBb	bba$	b 匹配
5	$aB	ba$	输出 B→bB
6	$aBb	ba$	b 匹配
7	$aB	a$	输出 B→ε
8	$a	a$	a 匹配
9	$	$	ACCEPT

随堂练习（7）

- 对文法G[S]：S→aBa，B→bB|ε，构造预测分析表，并对输入符号串abba及aabb分别进行LL(1)预测分析，写出分析的全过程（表格形式）。

Non-terminal	lookahead		
	a	b	$
S	S→aBa		
B	B→ε	B→bB	

步骤	分析栈	剩余输入串	动作
1	$S	aabb$	输出 S→aBa
2	$aBa	aabb$	a 匹配
3	$aB	abb$	输出 B→ε
4	$a	abb$	a 匹配
5	$	bb$	**ERROR**

4. LL(1)预测分析中的错误恢复[Error Recovery]

- Parser在Error情况下应该做什么?
  - **给出Error信息**
    - ✓ Error信息越详尽越好
  - **从Error中恢复**
    - ✓ 能够继续对其余输入进行解析
- Error类型
  - **终结符不匹配**：栈顶符号和下一个输入符号不匹配
  - **M[A,a]为空**：无候选产生式
  - **栈为空**，输入符号仍有剩余

4. LL(1)预测分析中的错误恢复[Error Recovery]

- **恐慌模式[Panic Mode]**
  - 是一种错误恢复策略
  - 恐慌：遇到危险先跑再说（**三十六计走为上计**），不尝试精确修复错误，直接跳过
  - 目标：让parser尽快回到可以继续分析的状态，而不是纠结于错误的细节
  - 基本思想：当parser遇到一个无法处理的错误时，采取“紧急避险”的方式，简单粗暴地**忽略/跳过**输入中的一些符号，直到输入中出现一些特定的符号（**同步符号synch**）

4. LL(1)预测分析中的错误恢复[Error Recovery]

- 恐慌模式[Panic Mode]
  - 规则：
    - ✓ 将FOLLOW(A)的所有符号都放到A的同步集合中，即，对于a∈FOLLOW(A)，预测分析表中M[A, a]处均填写**synch**
    - ✓ 如果**M[top, lookahead]==empty**，则跳过lookhead字符，栈保持不变
    - ✓ 如果**M[top, lookahead]==synch**，则跳过栈顶top，将栈顶弹出，输入字符不变
    - ✓ 如果**top!=lookahead**，则同样跳过栈顶top，将栈顶弹出，输入字符不变

4. LL(1)预测分析中的错误恢复[Error Recovery]

- 恐慌模式[Panic Mode]
  - 例:

	i	+	*	(	)	$
E	E→TE'			E→TE'	**synch**	**synch**
E'		E'→+TE'			E'→ ε	E'→ ε
T	T→ FT'	**synch**		T→ FT'	**synch**	**synch**
T'		T'→ ε	T'→* FT'		T'→ ε	T'→ ε
F	F→ i	**synch**	**synch**	F→ (E)	**synch**	**synch**

FOLLOW(E)={), $}　　FOLLOW(E')={), $}

FOLLOW(T)={+,), $}　　FOLLOW(T')={+,), $}

FOLLOW(F)={*, +,), $}

4. LL(1)预测分析中的错误恢复[Error Recovery]

- 恐慌模式[Panic Mode]
  - 例:

	i	+	*	(	)	$
E	E→TE'			E→TE'	**synch**	**synch**
E'		E'→+TE'			E'→ ε	E'→ ε
T	T→ FT'	**synch**		T→ FT'	**synch**	**synch**
T'		T'→ ε	T'→*FT'		T'→ ε	T'→ ε
F	F→ i	**synch**	**synch**	F→ (E)	**synch**	**synch**

步骤	分析栈	剩余输入串	动作
1	$E	*i*+i$	**empty**, skip input, **输出error**
2	$E	i*+i$	输出 E→TE’
3	$E’T	i*+i$	输出 T→ FT’
4	$E’T’F	i*+i$	输出 F→ i
5	$E’T’i	i*+i$	i 匹配
6	$E’T’	*+i$	输出 T'→*FT'
7	$E’T’F*	*+i$	* 匹配
8	$E’T’F	+i$	**synch**, skip top, **输出error**
9	$E’T’	+i$	输出 T'→ ε

4. LL(1)预测分析中的错误恢复[Error Recovery]

- 恐慌模式[Panic Mode]
  - 例:

	i	+	*	(	)	$
E	E→TE'			E→TE'	synch	synch
E'		E'→+TE'			E'→ ε	E'→ ε
T	T→ FT'	synch		T→ FT'	synch	synch
T'		T'→ ε	T'→*FT'		T'→ ε	T'→ ε
F	F→ i	synch	synch	F→ (E)	synch	synch

步骤	分析栈	剩余输入串	动作
10	$E’	+i$	输出 E'→+TE’
11	$E’T+	+i$	+ 匹配
12	$E’T	i$	输出 T→ FT’
13	$E’T’F	i$	输出 F→ i
14	$E’T’i	i$	i 匹配
15	$E’T’	$	输出 T'→ ε
16	$E’	$	输出 E'→ ε
17	$	$	end

CONTENTS

目 录

01	02	03	04
自顶向下分析 Top-Down Parsing	LL(1)分析 LL(1) Parsing	自底向上分析 Bottom-Up Parsing	LR分析 LR Parsing

1. 自底向上语法分析[Bottom-Up Parsing]

- 定义：
  - **从输入符号串开始**，逐步进行**归约**，直至归约到文法的开始符号。
- 语法树的构造：
  - 从输入符号串开始，以它**作为语法树的末端结点符号串**，**自底向上**的构造语法树。

归约过程：　　**cabd |-cAd |-S**

用“|-”表示归约，下划线部分为被归约符号

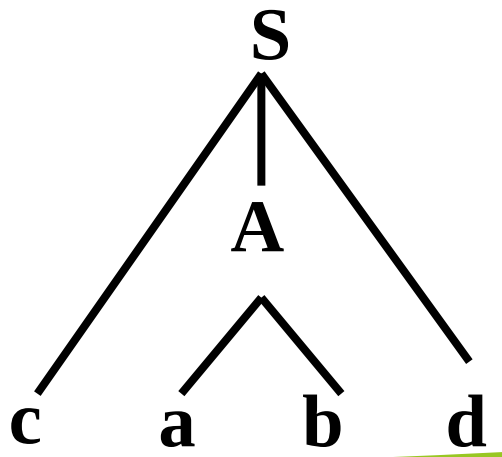
例：文法G：

S → cAd

A → ab

A → a

识别输入串w=cabd是否为该文法的句子



1. 自底向上语法分析[Bottom-Up Parsing]

- 规范归约
  - 自底向上分析的**归约**过程**是**自顶向下**推导的逆过程**。
  - 最右推导为规范推导，所以**自左向右的归约称为规范归约**。
  - 例 文法：　(1)　S→aAcBe　　(2)　A→b

    (3)　A→Ab　　(4)　B→d

    输入串 abbcde的最右推导(规范推导)的过程为：

    S=>**aAcBe**=>a**A**c**d**e=>a**Ab**cde=>a**b**bcde

    自底向上分析的规范归约过程为：

    abbcde|-a**A**bcde|-a**A**cde|-aAc**B**e|-**S**

1. 自底向上语法分析[Bottom-Up Parsing]

- 自底向上分析的主要问题

例：对输入串cabd的两种归约过程

（1）**cabd|-cAd|-S**　　归约到开始符

（2）**cabd|-cAbd**　　不能归约到开始符

S → cAd
A → ab
A → a

– 在自底向上的分析方法中，每一步都是从当前串中选择一个子串加以归约，该子串暂称“**可归约串**”。

– 自底向上分析的**主要问题：如何确定选用哪个产生式，即，如何确定“可归约串”**

✓ **句柄(最左直接短语)**

✓ **最左素短语**

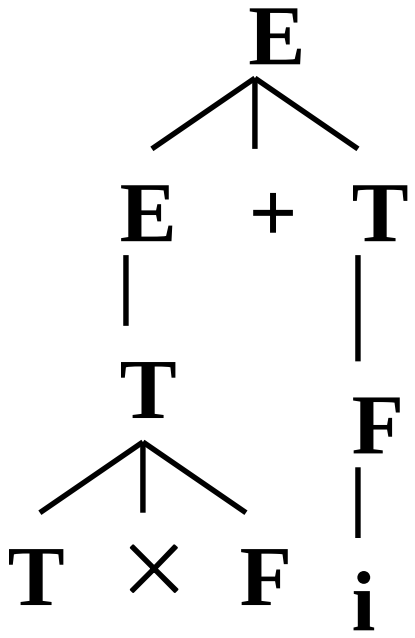
1. 自底向上语法分析[Bottom-Up Parsing]

- **短语[phrase]，直接短语和句柄[handel]**
  - 设αβδ是文法G[S]中的一个句型，如果有S=*>αAδ且A=+>β，则称β是句型αβδ相对于非终结符A的**短语[phrase]**。
  - 特别的如有A=>β，则称β是句型αβδ相对于规则A→β的**直接短语（简单短语）**。
  - 一个句型的最左直接短语称为该句型的**句柄[Handle]**。
  - **句柄就是“可归约串”。**

1. 自底向上语法分析[Bottom-Up Parsing]

- **短语[phrase]，直接短语和句柄[handel]**

例：文法G[E]: E→E+T|T，T→T×F|F，F→(E)| i的一个句型是**T×F+i**，相应的语法树见右图：



1. 因为E=*> T+i 且 T=>T×F，所以**T×F**是句型相对于T的短语，且是相对于T→T×F的直接短语；
2. 因为E=*> T×F+F 且 F=>i，所以**i**是句型相对于F的短语，且是相对于F→i的直接短语；
3. 因为E=*> E 且E=⁺>T×F+i，所以**T×F+i**是句型相对于E的短语；
4. **T×F**是最左直接短语，即句柄。

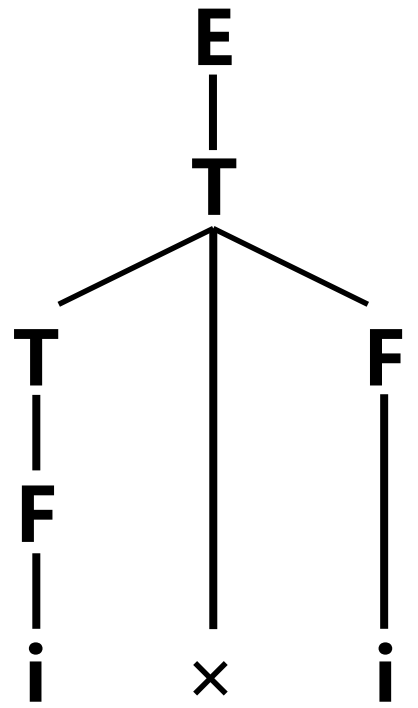
注意：虽然**F+i**是句型T×F+i的一部分，但**不是短语**，因为尽管有E=⁺>F+i，但是**不存在从文法开始符E=* >T×E的推导**

若有S=*>αAδ且A=⁺>β，则称β是句型αβδ相对于非终结符A的短语。

1. 自底向上语法分析[Bottom-Up Parsing]

- 句柄作为可归约串
  - 例：文法G[E]：E→E+T|T，T→T×F|F，F→(E)|i，识别输入串w=i×i是否为该文法的句子
    - 对于句型i×i，第1个i为句柄(E=>*F×F, F=>i)
    - 对于句型F×i，F为句柄(E=>*T×F, T=>F)
    - 对于句型T×i，i为句柄(E=>*T×F, F=>i)
    - 对于句型T×F，T×F为句柄(E=>*T, T=>T×F)
    - 对于句型T，T为句柄(E=>*T, T=>T)

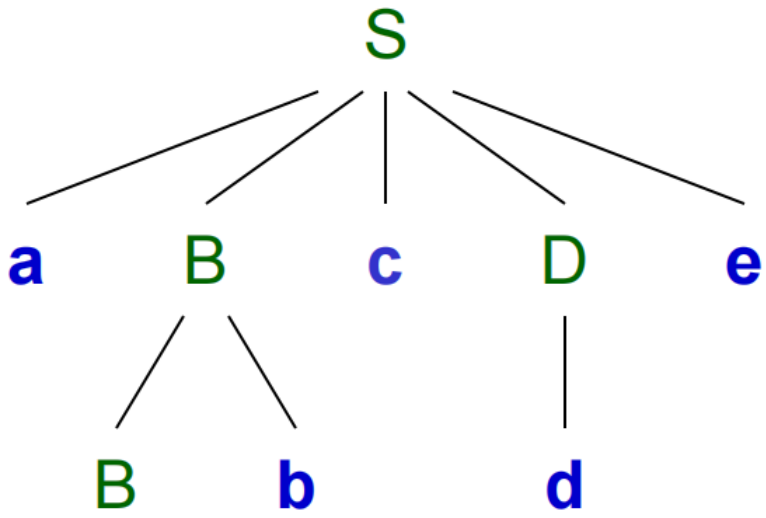
i×i |-F×i |-T×i |-T×F |-T |-E

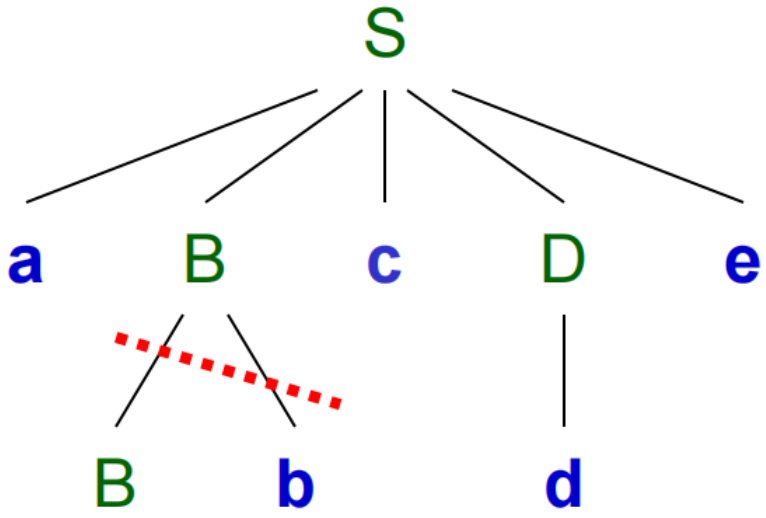


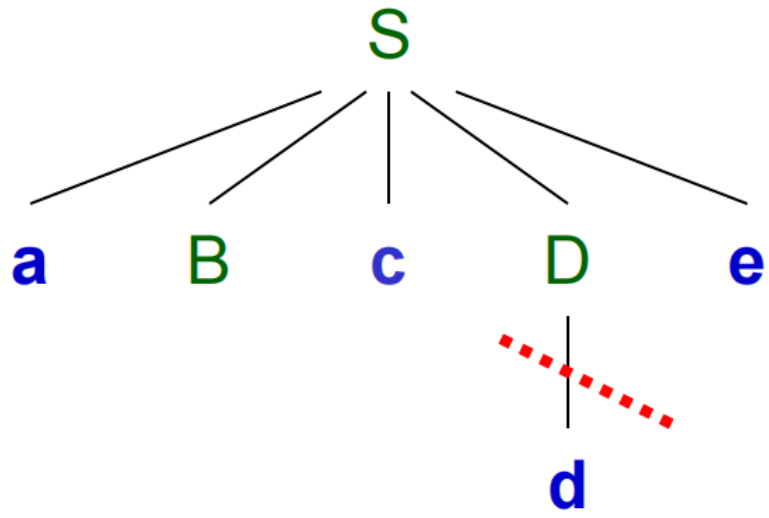
若有S=*>αAδ且A=>β，则称β是句型αβδ相对于非终结符A的直接短语。

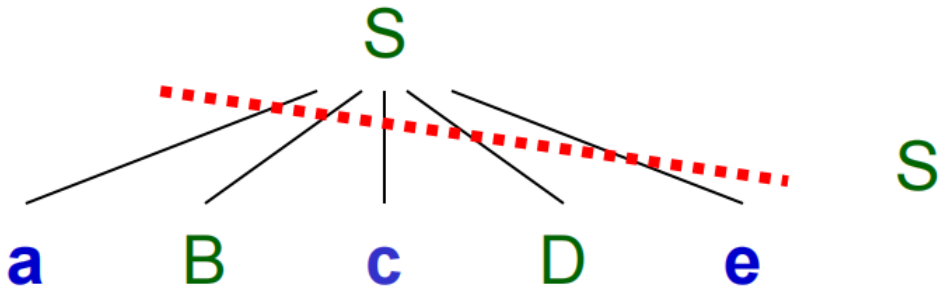
1. 自底向上语法分析[Bottom-Up Parsing]

- 句柄剪枝
  - 例：文法G[S]：S→aBcDe，B→Bb|b，D→d，识别输入串w=abbcde是否为该文法的句子









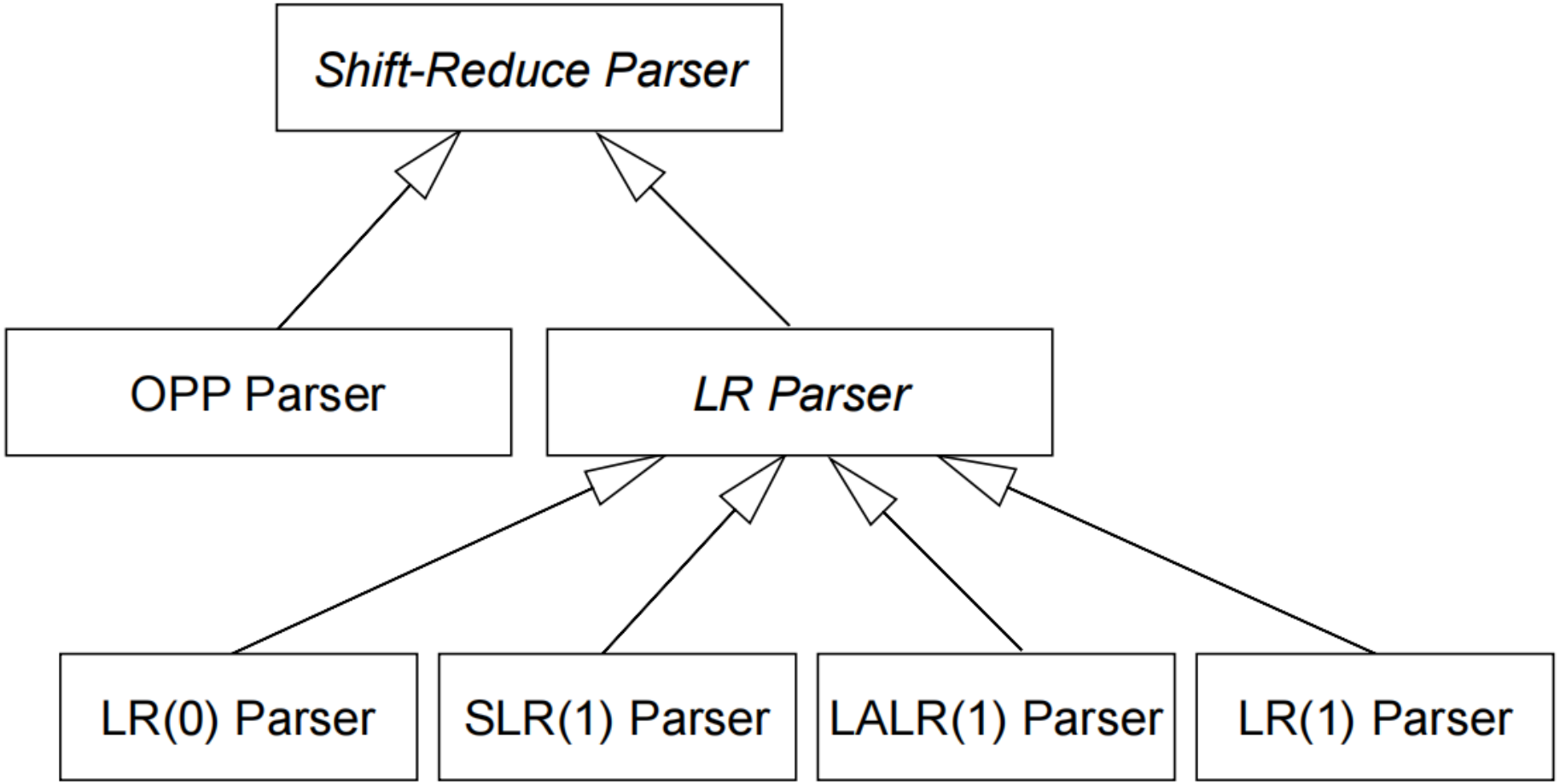
S

abbcde|-aBbcde|-aBcde|-aBcDe |-S

2. 移进-归约分析[Shift-Reduce Parsing]

- Shift-Reduce分析与LL(1)分析比较
  - 方向对称：
    - LL(1)分析：Top-down
    - Shift-Reduce分析：Bottom-up
  - 同为Table-driven方法
    - LL(1)分析： LL(1)预测分析表
    - Shift-Reduce分析：算符优先关系表/LR(0)/SLR(1)/LALR(1)/LR(1)分析表
  - 栈与输入串
    - LL(1)分析：**[$S, *w*$] ⇒ [$, $]**
    - Shift-Reduce分析：**[$, *w*$] ⇒ [$S, $]**
  - 动作
    - LL(1)分析：**match-derive**
    - Shift-Reduce分析：**shift-reduce**

2. 移进-归约分析[Shift-Reduce Parsing]



2. 移进-归约分析[Shift-Reduce Parsing]

- Shift-Reduce分析
  - 引进一个先进后出的**符号栈**来存放符号;
  - 对输入符号串自左向右进行扫描，并把当前输入符号下推入栈中（**移进**），边移进边分析，一旦栈顶符号串形成某个句型的**句柄**（为某产生式的右部）时，就用相应的非终结符（产生式的左部）替换它（**归约**）。
  - 重复这一过程，直到输入符号串的末端，**此时如果栈中只剩文法开始符号，则输入符号串是文法的句子**，否则不是。**[$, *w*$] ⇒ [$S, $]**

2. 移进-归约分析[Shift-Reduce Parsing]

- 例：文法G[S]：S→aBcDe，B→Bb|b，D→d，对输入串abbcde进行移进-归约分析

步骤	符号栈	输入符号串	动作
1	$	abbcde$	移进a
2	$a	bbcde$	移进b
3	$a**b**	bcde$	归约（B→b）
4	$aB	bcde$	移进b
5	$a**Bb**	cde$	归约（B→Bb）
6	$aB	cde$	移进c
7	$aBc	de$	移进d
8	$aBc**d**	e$	归约（D→d）
9	$aBcD	e$	移进e
10	$**aBcDe**	$	归约（S→aBcDe）
11	**$S**	**$**	**接受**

2. 移进-归约分析[Shift-Reduce Parsing]

- 例：文法G[E]:

E→E+E|E×E|(E)|n

对输入串n+n×n

进行移进-归约分析

步骤	符号栈	输入字符串	动作
1	$	n+n×n$	移进
2	$n	+n×n$	归约E→n
3	$E	+n×n$	移进
4	$E+	n×n$	移进
5	$E+n	×n$	归约E→n
6	$E+E	×n$	**归约E→E+E**
7	$E	×n$	**ERROR**
6’	$E+E	×n$	**移进**
7’	$E+E×	n$	移进
8	$E+E×n	$	归约E→n
9	$E+E×E	$	**归约E→E×E**
10	$E+E	$	**归约E→E+E**
11	$E	$	接受

3. 算符优先分析[Operator-Precedence Parsing]

- 基本思想：只定义文法中**终结符**之间的优先关系（不考虑非终结符），并由这种关系指导分析过程。
- **不是规范归约**。
- 优点：算符优先分析法分析速度快，特别适用于表达式的分析。
- 缺点：不规范，常常要采用适当措施克服其缺点。
- 前提：**文法必须是算符文法**

3. 算符优先分析[Operator-Precedence Parsing]

- 算符文法[Operater Grammar]
  - 设有上下文无关文法G，如果G中产生式的右部**没有两个非终结符相连**，则称G为算符文法
  - 算符文法中不含ε产生式
  - 例：G[E]：E→E+E|E×E|(E)|i 是算符文法
  - 例：G[E]：E→EAE|(E)|−E|id 不是算符文法

3. 算符优先分析[Operator-Precedence Parsing]

- 算符优先
  - 优先关系只存在于句型中**相邻**出现的符号
  - 算符优先分析法**只考虑终结符之间的优先关系**，不考虑非终结符，所以**两个终结符相邻指它们之间没有其他的终结符**（但可以有非终结符）
  - 如：E+T×i，则＋和×相邻，×和i相邻，但＋和i不相邻
  - 算符有三种优先关系：
    - ✓ $a \lessdot b$　　表示a的优先级低于b
    - ✓ $a \equiv b$　　表示a的优先级等于b
    - ✓ $a \gtrdot b$　　表示a的优先级高于b

3. 算符优先分析[Operator-Precedence Parsing]

- 算符优先
  - 注意：
    - ✓ $\lessdot, \equiv, \gtrdot$ 是**偏序关系**
    - ✓ **不具备自反性**：a不一定≡a
    - ✓ **不具备对称性**：a$\lessdot$b不意味着b$\gtrdot$a，a≡b不意味着b≡a
    - ✓ **不具备传递性**：a$\lessdot$b，b$\lessdot$c不意味着a$\lessdot$c
    - ✓ 算法优先关系**仅考虑相邻两个终结符之间的关系**，若终结符a和b在任何句型中都不相邻，则无法比较它们的优先关系

3. 算符优先分析[Operator-Precedence Parsing]

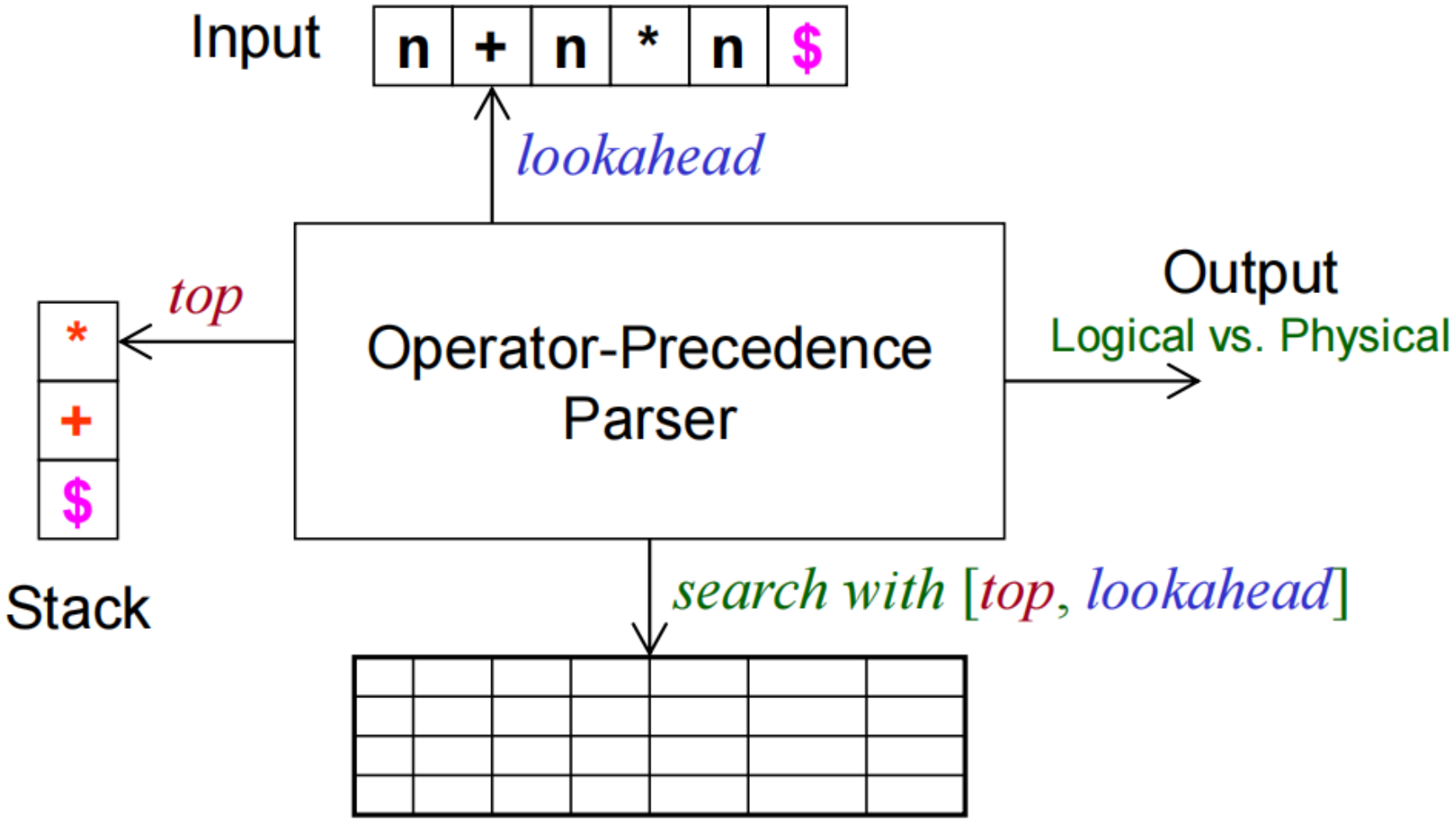
- 同样是Table-driven
- 依赖于算符优先关系表[Operator-Precedence Relation Table]

	id	+	*	$
id		⋗	⋗	⋗
+	⋖	⋗	⋖	⋗
*	⋖	⋗	⋗	⋗
$	⋖	⋖	⋖	

算符优先关系表

3. 算符优先分析[Operator-Precedence Parsing]

- 算符优先分析



3. 算符优先分析[Operator-Precedence Parsing]

- 算法

```
initialize();
for (;;) {
    if (top == $ && lookahead == $)  accept();
    topOp = stack.topMostTerminal();
    if (topOp ≺ lookahead || topOp ≡ lookahead) {  // shift
        stack.push(lookahead);
        lookahead = scanner.getNextToken();
    } else if (topOp ≻ lookahead) {  // reduce
        do {
            topOp = stack.pop();
        } while (stack.topMostTerminal() ≻ || ≡ topOp);
    } else  error();
}
```

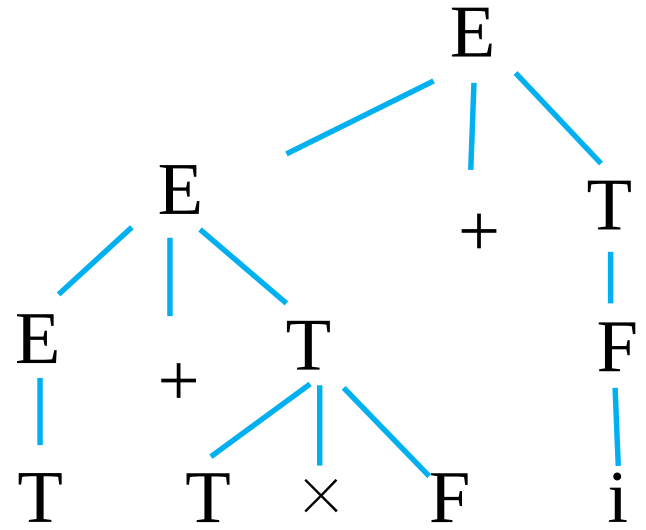
寻找最左素短语，用其进行归约

3. 算符优先分析[Operator-Precedence Parsing]

- **最左素短语**
  - 设有文法G[S]，其句型的**素短语是一个短语**，它**至少包含一个终结符**，并**除自身外不包含其它素短语**。
  - 最左边的素短语称为最左素短语。
  - 因此，找素短语，就要先找短语。

3. 算符优先分析[Operator-Precedence Parsing]

- **最左素短语**
  - 例：文法G[E]：E→E+T|T，T→T×F|F，F→(E)|i 的一个句型为T+T×F+i
    - ✓ 短语为：T，T×F，i，T+T×F，T+T×F+i
    - ✓ 素短语为：T×F，i
    - ✓ 最左素短语为：T×F



  - **规范归约**归约当前句型中的**句柄(T)**
  - **算符优先分析**归约当前句型中的**最左素短语(T×F)**
  - **算符优先分析**去掉了单个非终结符的归约，因为若只有一个非终结符，无法与句型中该非终结符左部和右部的串比较优先关系，也就无法确定如何归约。

素短语：是一个短语，至少包含一个终结符，并除自身外不包含其它素短语。

3. 算符优先分析[Operator-Precedence Parsing]

- 算法
  - 将输入符号串$a_1a_2...a_t$依次逐个存入符号栈S中，直至符号栈顶终结符a_j与当前输入符a_{j+1}的关系为$a_j \succ a_{j+1}$为止；
  - **最左素短语尾已在符号栈S的栈顶，由栈顶向下找最左素短语的头符号**，即找到第一个$\prec$关系$a_{i-1} \prec a_i$；
  - 已找到**最左素短语**$N_ia_iN_{i+1}a_{i+1}.....N_ja_jN_{j+1}$(其中$N_k(i \leqslant K \leqslant j+1)$为**非终结符**或**空**)，用相应的产生式进行归约；
  - 重复上述过程，当处理到**输入符号串尾$时**，**若栈中只剩：$N（N为任何一个非终结符），则分析成功**，否则分析失败。

3. 算符优先分析[Operator-Precedence Parsing]

- 文法G[E]：E→E+T|T，T→T×F|F，F→(E)|i，采用算符优先分析法识别输入串w=i×i是否为该文法的句子

	+	×	(	)	i	$
+	≻	≺	≺	≻	≺	≻
×	≻	≻	≺	≻	≺	≻
(	≺	≺	≺	≡	≺	
)	≻	≻		≻		≻
i	≻	≻		≻		≻
$	≺	≺	≺		≺	≡

表格中空白的位置表示error

步骤	栈	优先关系	当前符号	剩余输入串	动作
1	$	≺	i	+i$	移进
2	$i	≻	+	i$	归约 F→i
3	$F	≺	+	i$	移进
4	$F+	≺	i	$	移进
5	$F+i	≻	$		归约 F→i
6	$F+F	≻	$		归约 E→E+T
7	$E	≡	$		ACCEPT

3. 算符优先分析[Operator-Precedence Parsing]

- 例：文法G[E]: E→E+E|E×E|(E)|n，对输入串n+n×n进行移进-归约分析

步骤	符号栈	输入字符串	动作
1	$	n+n×n$	移进
2	$n	+n×n$	归约E→n
3	$E	+n×n$	移进
4	$E+	n×n$	移进
5	$E+n	×n$	归约E→n
6	$E+E	×n$	**移进？归约？**
7	$E+E×	n$	移进
8	$E+E×n	$	归约E→n
9	$E+E×E	$	**归约？**
10	$E+E	$	归约E→E+E
11	$E	$	**ACCEPT**

	+	×	(	)	n	$
+	≻	≺	≺	≻	≺	≻
×	≻	≻	≺	≻	≺	≻
(	≺	≺	≺	≡	≺	
)	≻	≻		≻		≻
n	≻	≻		≻		≻
$	≺	≺	≺		≺	≡

移进

归约E→E×E

3. 算符优先分析[Operator-Precedence Parsing]

- 算法说明：
  - 算法中每次都**取终结符进行比较**，当栈顶符号不是终结符时，便取其下一个符号（这时一定是终结符）
  - 归约时检查是否有与最左素短语对应的产生式，查看产生式的右部：
    - ✓ 符号个数与该素短语的符号个数相等
    - ✓ **非终结符位置对应，不管其符号名是什么**
    - ✓ 终结符名字和位置都对应相等
  - 若有满足以上条件的产生式才归约，否则出错

随堂练习（8）

- 有文法G[S]：S→(L)|a，L→L,S|S，对输入符号串(a, (a, a))进行算法优先分析（假设有如下算法优先关系表）

步骤	符号栈	优先关系	输入字符串	动作
1	$	≺	(a, (a, a))$	移进
2	$(	≺	a, (a, a))$	移进
3	$(a	≻	, (a, a))$	归约S→a
4	$(S	≺	, (a, a))$	移进
5	$(S,	≺	(a, a))$	移进
6	$(S,(	≺	a, a))$	移进
7	$(S,(a	≻	, a))$	归约S→a
8	$(S,(S	≺	, a))$	移进
9	$(S,(S,	≺	a))$	移进

	a	(	)	,	$
a			≻	≻	≻
(	≺	≺	≡	≺	
)			≻	≻	≻
,	≺	≺	≻	≻	
$	≺	≺			

随堂练习（8）

- 有文法G[S]：S→(L)|a，L→L,S|S，对输入符号串(a, (a, a))进行算符优先分析（假设有如下算符优先关系表）

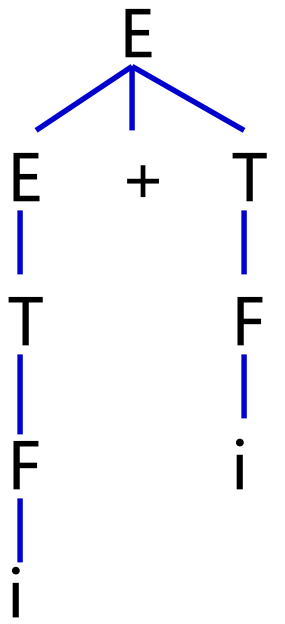
步骤	符号栈	优先关系	输入字符串	动作
10	$(S,(S, **a**	≻	**)**)$	归约S→a
11	$(S,(S**,** S	≻	**)**)$	归约L→L,S
12	$(S,**(**L	≡	**)**)$	移进
13	$(S,(L**)**	≻	**)**$	归约S→(L)
14	$(S**,**S	≻	**)**$	归约L→L,S
15	$**(**L	≡	**)**$	移进
16	$(L**)**	≻	$	归约S→(L)
17	$S		$	**ACCEPT**

	a	(	)	,	$
a			≻	≻	≻
(	≺	≺	≡	≺	
)			≻	≻	≻
,	≺	≺	≻	≻	
$	≺	≺			

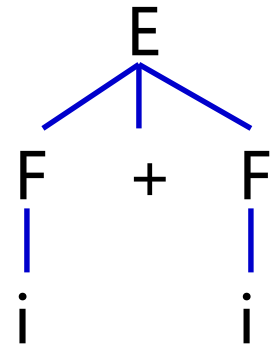
3. 算符优先分析[Operator-Precedence Parsing]

- 规范归约 vs.算符优先分析

1. 规范归约将输入符号串**归约为文法的开始符号**；算符优先分析将输入符号串**归约为任意一个非终结符**
2. 规范归约每次**归约句柄**；算符优先分析每次**归约最左素短语**，去掉了单个非终结符的归约
3. 规范归约选取右部与句柄一致的产生式进行归约；算符优先分析归约时选择产生式时不管非终结符的名字



采用规范归约识别i+i的过程



采用算符优先分析识别i+i的过程

3. 算符优先分析[Operator-Precedence Parsing]

- 优先函数
  - 优先矩阵占用大量空间: $(n+1)^2$
  - 在实际应用中用优先函数来代替优先矩阵表示优先关系
  - 优先函数：对给定的优先关系表，定义两个函数 f，g，它们应满足：
    - ✓ 当$a \equiv b$　则令 f(a)=g(b)
    - ✓ 当$a \lessdot b$　则令 f(a)<g(b)
    - ✓ 当$a \gtrdot b$　则令 f(a)>g(b)

    我们称 f，g 为优先函数，其取值可用整数表示。

3. 算符优先分析[Operator-Precedence Parsing]

- 优先函数

优先关系表 $(n+1)^2$ 个单元

	i	×	+	#
i		≻	≻	≻
×	≺	≻	≻	≻
+	≺	≺	≻	≻
#	≺	≺	≺	≡

优先函数表 2(n+1) 个单元

	i	×	+	#
f	6	6	4	2
g	7	5	3	2

– 优点：节省大量内存

– 缺点：原先无优先关系的对，如：(i,i) 变成有关系：f(i)=6<g(i)=7，不能准确指出错误位置

3. 算符优先分析[Operator-Precedence Parsing]

- 优先函数
  - 用关系图法构造优先函数
    - ✓ 对每个终结符a(包括‘$’)画出$f_a$, g_a结点，
      - 若$a \succ b$或$a \equiv b$则从f_a到g_b画一条弧，
      - 若$a \prec b$或$a \equiv b$则从g_b到f_a画一条弧。
  - 函数值：**从该结点出发所能到达的结点(含自身结点)的个数**

3. 算符优先分析[Operator-Precedence Parsing]

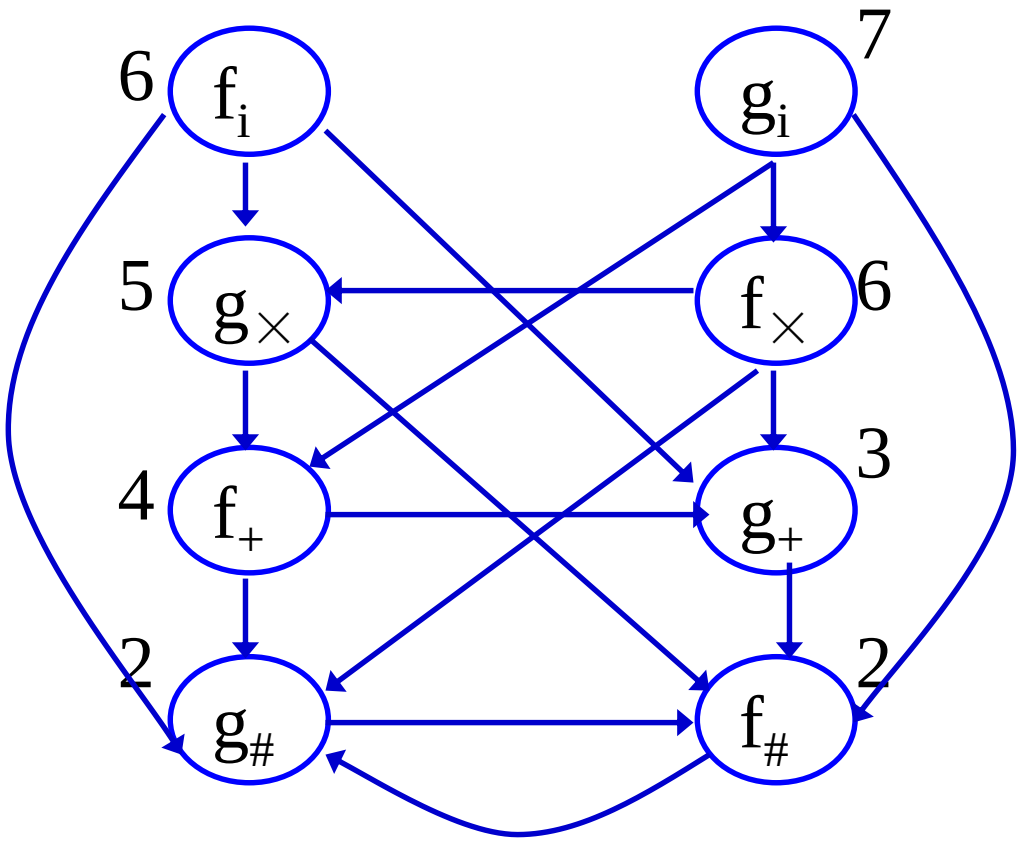
- 例：有优先关系表如下：

	i	×	+	#
i		≻	≻	≻
×	≺	≻	≻	≻
+	≺	≺	≻	≻
#	≺	≺	≺	≡

优先函数结果为：

	i	×	+	#
f	6	6	4	2
g	7	5	3	2

若a≻b或a≡b则从f_a到g_b画一条弧，
若a≺b或a≡b则从g_b到f_a画一条弧。



从f_i出发所能到达的结点有：

f_i，$g_\#$，$f_\#$，$g_\times$，f_+，g_+

3. 算符优先分析[Operator-Precedence Parsing]

- 局限性
  - 适用范围窄，很多实用语言的文法不是算符优先文法
  - 只能处理非常简单的语言
  - 有时错误的句子得到了正确的归约，原因：
    - ✓ 去掉了单个非终结符的归约
    - ✓ 归约时，选择产生式不管产生式右部非终结符的名字，只要非终结符位置对应相同就归约
- 商业应用中仍存在
  - Sun javac编译器
  - 早期的THINK C编译器
  - 贝尔实验室的原始C编译器

CONTENTS

目 录

01 自顶向下分析 Top-Down Parsing

02 LL(1)分析 LL(1) Parsing

03 自底向上分析 Bottom-Up Parsing

04 LR分析 LR Parsing

1. LR分析概述

- 由Donald Ervin Knuth（高德纳）提出
  - 《计算机程序设计的艺术》[The Art of Computer Programming]系列
  - 《计算机与排版》（Computers and Typesetting）
  - 提出“算法”和“数据结构”概念
  - 提出LR分析法
  - 提出双向链表
  - 提出KMP算法(Knuth-Morris-Pratt)
  - 提出Tex排版系统
  - 算法大师Knuth如何看待GPT?
  - 最年轻的图灵奖获得者记录的保持者（36岁）



1. LR分析概述

- 自底向上分析法的关键问题是在分析过程中**如何确定句柄**
- LR分析法：根据当前符号栈中的符号串和向前顺序查看输入串的k个符号（k>=0）就可唯一地确定句柄
- 目前最流行的自底向上语法分析器都是基于LR(k)语法分析，其中
  - **L**表示**从左到右**扫描输入串
  - **R**表示最左归约（即**最右推导**的逆过程）
  - **k**表示**向前查看输入串符号的个数**
    - ✓ 当k=1时，能满足当前绝大多数高级语言编译程序的需要，所以着重介绍LR(0), SLR(1), LR(1), LALR(1)方法
    - ✓ 省略(k)时，一般指k=1

1. LR分析概述

- LR分析的特点
  - 是**规范归约**（**最右推导/规范推导**的逆过程）
  - **适用范围广**，适用于大多数上下文无关文法描述的语言（比LL(k)分析方法和算符优先分析方法对文法的限制要少得多）
    - ✓ LR(k)：只要在一个最右句型中看到某个产生式右部，再向前看k个符号就可以决定是否使用这个产生式进行归约
    - ✓ LL(k)：要向前查看某个产生式右部推导出的串的前k个符号，才能决定是否使用这个产生式进行推导
    - ✓ 算符优先分析：仅适用于算符优先文法（如表达式）
  - **分析速度快**
  - 在对输入进行自左到右扫描时能尽早准确定位错误
  - 缺点：**LR分析器构造的工作量大**

2. LR(0)分析

- LR(0)分析
  - 根据当前符号栈中的符号串和向前顺序查看输入串的**0个符号**就可唯一地确定句柄以**进行归约**，即，**仅凭符号栈中的符号串即可确定句柄，做出归约决定，不需要向前查看输入符号**
  - 对文法的限制较大，对绝大多数高级语言的语法分析器不适用
  - 构造LR(0)分析表的思想和方法是构造其他LR分析表的基础

2. LR(0)分析

- 项目[item]
  - LR语法分析器通过维护一些状态，用这些状态来表明我们在语法分析过程中所处的位置，从而做出**shift或reduce决定**
  - **状态由项目[item]集表示**
  - 在文法G中每个产生式**右部的适当位置添加一个圆点**构成项目
  - 例：产生式A→XYZ 对应有4个项目

    [0] A→ • XYZ　　[1] A→X • YZ

    [2] A→XY • Z　　[3] A→XYZ •

    产生式A→ε只有一个项目A→•

2. LR(0)分析

- 项目的含义
  - 圆点在最左部（A→•XYZ）表示希望用产生式的右部归约
  - 圆点的左部（A→**X**•YZ）表示分析过程中已识别过的部分
  - 圆点的右部（A→X•**YZ**）表示待识别部分
  - 圆点达到最右边（A→XYZ•）表示句柄已形成，可以进行归约

2. LR(0)分析

- 项目分类
  - **移进项目[shift item]**
    - ✓ 形如**A→α•aβ**的项目，其中**a∈V_T**，α,β∈V*
    - ✓ 即圆点后面为**终结符**的项目
    - ✓ 分析时把a移进符号栈
  - **待约项目[reduce-expected item]**
    - ✓ 形如**A→α•Bβ**的项目，其中**B∈V_N**，α,β∈V*
    - ✓ 即圆点后面为**非终结符**的项目
    - ✓ 表明：用产生式A的右部归约时，首先要将B的产生式右部归约为B，对A的右部才能继续进行分析，即，期待着继续分析过程中**首先能归约得到B**

2. LR(0)分析

- 项目分类
  - **归约项目[reduce item]**
    - ✓ 形如**A→α•**的项目，α∈V*，α=ε对应的项目为A→•
    - ✓ 即**圆点在最右端的项目**
    - ✓ 表明该产生式的右部已分析完，**句柄已形成**，可以把α归约为A
  - **接受项目[accept item]**
    - ✓ 当归约项目为**S'→S•**，其中**S'是文法开始符**
    - ✓ 即对文法开始符的归约项目
    - ✓ **表明输入串可归约为文法开始符，分析结束**

2. LR(0)分析

- LR(0)项目集[canonical LR(0) collection]
  - 用一个LR(0)项目的集合来表示LR(0)自动机的一个状态
  - 为了构造文法的LR(0)项目集，需定义

    (1) 拓广文法[augmented grammar]

    (2) CLOSURE函数

    (3) GOTO函数

2. LR(0)分析

(1) 拓广文法

- 在原有文法上，**新增开始符号S’**和**产生式S’→S**
- 例：文法G[S]：S→aAcBe，A→b，A→Ab，B→d

  其拓广文法G’[S’]为：S’→S，S→aAcBe，A→b，A→Ab，B→d
- 拓广原因：文法开始符S可能出现在产生式的右部，在归约过程中，不能判断是否已经归约到文法的最初开始符，还是归约到在产生式右部出现的开始符，而S’只在产生式左部出现，确保不会混淆

2. LR(0)分析

(2) CLOSURE函数

– 如果I是文法G的一个项目集，定义和构造I的闭包CLOSURE(I)如下：

① I的项目均在CLOSURE(I)中

② 若A→α•Bβ属于CLOSURE(I)，B∈V_N，则每一形如B→•r的项目也属于CLOSURE(I)

③ 重复②直到CLOSURE(I)不再扩大为止

– 说明：圆点后为终结符或圆点在产生式的最后，求闭包时不会增加新的项目

– 例：S'→E，E→aA|bB，A→cA|d，B→cB|d，若I={ S'→•E }

则 CLOSURE(I)={ S'→•E，E→•aA，E→•bB }

- CLOSURE(I)作为DFA的一个状态

2. LR(0)分析

(3) GOTO函数

– 由DFA的一个状态求其他状态通过**状态转换函数GOTO**

– 设I为文法G的某一项目集(状态)，$X \in V_N \cup V_T$，则:

GOTO(I, X)=CLOSURE(J)

其中J = {任何形如A→αX•β的项目 | A→α•Xβ∈I}，称J为“**核**”

– 例： S’→E，E→aA|bB，A→cA|d，B→cB|d

若I = {S’→•E，E→•aA，E→•bB}，则:

✓ GOTO(I,E)=CLOSURE({S’→E•})={S’→E•}

✓ GOTO(I,a)=CLOSURE({E→a•A})={E→a•A, **A→•cA, A→•d**}

✓ GOTO(I,b)=CLOSURE({E→b•B})={E→b•B, **B→•cB, B→•d**}

2. LR(0)分析

- 构造以LR(0)项目集为状态的DFA

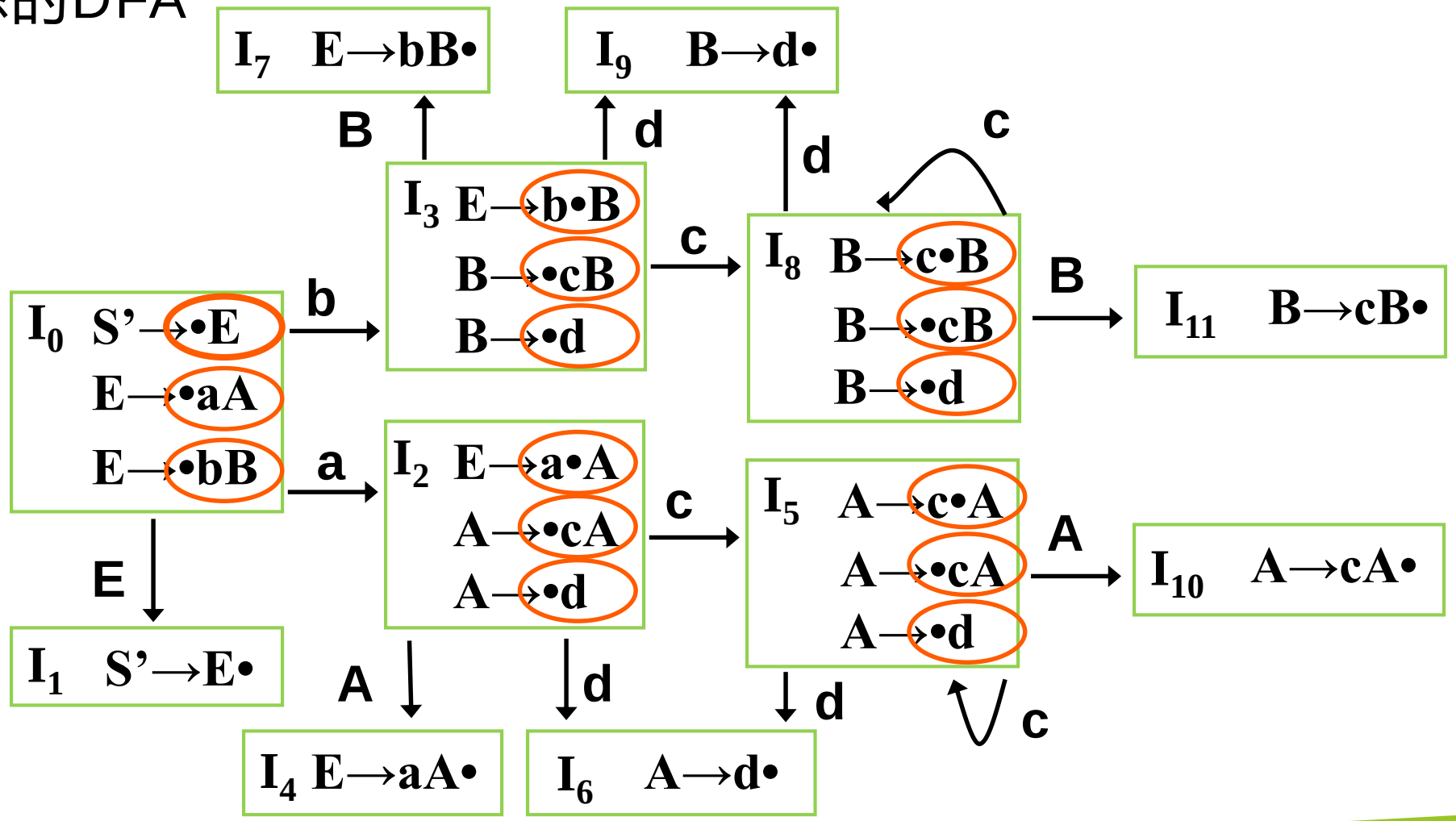
① 构造**初始状态IS_0=CLOSURE({S'→•S})**，并且它是未被标记的；

② 从已经构造的DFA部分图中选择一个未被标记的状态IS_i，标记IS_i，若项目集**IS_i中含有项目U->x•Ry**(R∈V,x,y为任一符号串)，且**GOTO(IS_i,R)=IS_j**，若IS_j不在DFA中，则将IS_j作为未被标记的加入DFA中，且从IS_i出发引一条标记为R的弧到IS_j；

③ 重复②直到没有未被标记的状态为止。

2. LR(0)分析

- 例：拓广文法G'：S'→E，E→aA|bB，A→cA|d，B→cB|d，构造以LR(0)项目集为状态的DFA



2. LR(0)分析

- LR(0)DFA识别的语言
  - 可行前缀[viable prefix]
    - ✓ 一个**可行前缀是**一个**最右句型(规范句型)的前缀**，并且它没有越过该最右句型的句柄的右端
    - ✓ 形成**句柄**之前(**包括句柄在内**)的所有规范句型的前缀称为**可行前缀**，即**规范句型的不含句柄右边符号的前缀称为可行前缀**

      例：文法G[S]：S→aAcBe，A→b，A→Ab，B→d，输入串abbcde的规范归约：

      abbcde ⊢ a**A**bcde ⊢ a**A**cde ⊢ aAc**B**e ⊢ S

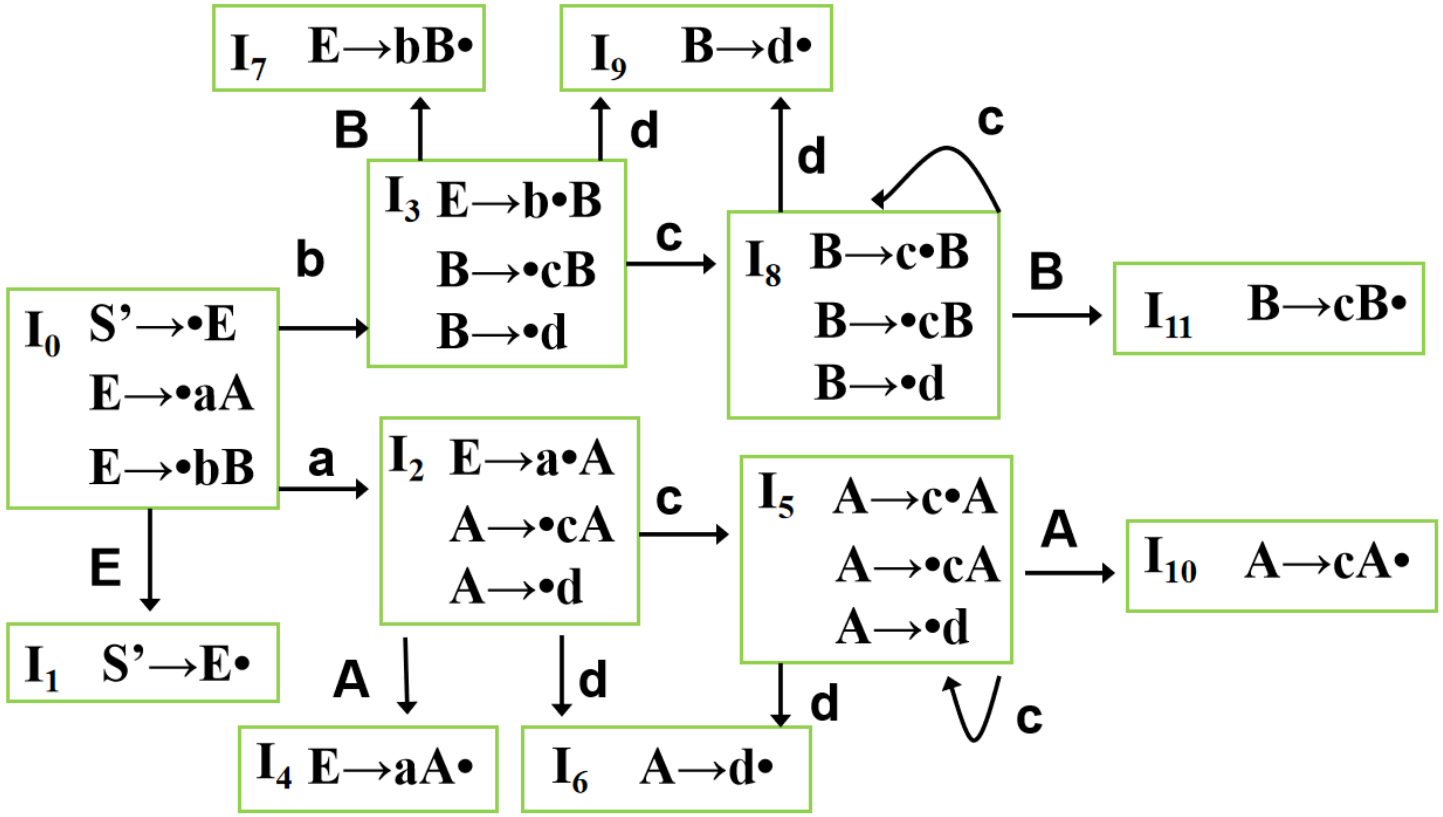
      以规范句型a**A**bcde为例，其句柄为Ab，其可行前缀为：ε，a，aA，aAb

      以规范句型aAc**B**e为例，其句柄为aAcBe，其可行前缀为：ε，a，aA，aAc，aAcB，aAcBe

2. LR(0)分析

- LR(0)DFA识别的语言
  - 句型acd的归约过程：

    acd|-acA|-aA|-E|-S’
  - 对句型acd：
    - ✓ 可行前缀为ε，a，ac，acd
  - 对句型acA：
    - ✓ 可行前缀为ε，a，ac，acA
  - 对句型aA：
    - ✓ 可行前缀为ε，a，aA



  - 可见，**可行前缀是已被DFA正确识别的规范句型的一部分**，其对应的LR分析的操作为**移进(符号栈)**

2. LR(0)分析

- 可行前缀[viable prefix]
  - 可行前缀一定是某个规范句型(右句型)的前缀，即，$S\Rightarrow^{*}_{rm}\alpha\omega$，其中$\alpha$为符号栈中的内容，$\omega$不含非终结符
  - 并不是所有规范句型的前缀是可行前缀

    例：对文法G[E]：E→TE’，E’→+TE’|ε，T→FT’，T’→*FT’|ε，F→(E)|n，

    $E\Rightarrow^{*}_{rm}F\times n\Rightarrow_{rm}(E)\times n$，**(**，**(E**，**(E)**是可行前缀，但**(E)×不是**可行前缀

    因为**(E)是句柄**，句柄一旦出现，就被归约，不可能与后面的字符构成可行前缀。

2. LR(0)分析

- 有效项目[valid item]
  - 如果存在一个推导过程$S' \Rightarrow_{rm}^{*} \alpha A\omega \Rightarrow_{rm} \alpha\beta_1\beta_2\omega$，则称项目$A \rightarrow \beta_1 \bullet \beta_2$对于可行前缀$\alpha\beta_1$有效（是$\alpha\beta_1$的有效项目），其中$\omega$仅含终结符
  - 可帮助决定分析时是进行移进或归约：
    - ✓ 若**$\beta_2 \neq \varepsilon$**，则表明句柄还没有被全部移进栈中，应**移进[shift]**
    - ✓ 若**$\beta_2 = \varepsilon$**，则β_1为句柄，应选用产生式$A \rightarrow \beta_1$进行**归约[reduce]**
  - 可用于构造DFA的状态：
    - ✓ CLOSURE()：若$A \rightarrow \beta_1 \bullet B\beta_2$对于可行前缀$\alpha\beta_1$有效且有产生式$B \rightarrow \gamma$，则$B \rightarrow \bullet\gamma$也对于可行前缀$\alpha\beta_1$有效（$S' \Rightarrow_{rm}^{*} \alpha A\omega \Rightarrow_{rm} \alpha\beta_1 B\beta_2\omega \Rightarrow_{rm} \alpha\beta_1\gamma\beta_2\omega$）
    - ✓ GOTO()：若$A \rightarrow \beta_1 \bullet X\beta_2$是$\alpha\beta_1$的有效项目，则$A \rightarrow \beta_1 X \bullet \beta_2$是$\alpha\beta_1 X$的有效项目（$S' \Rightarrow_{rm}^{*} \alpha A\omega \Rightarrow_{rm} \alpha\beta_1 X\beta_2\omega$）

2. LR(0)分析

- LR(0)分析同样是**table-driven**的
- LR(0)分析表
  - **动作表(ACTION)**：表示当前状态下面临输入符（终结符和‘$’）应做的动作是移进、归约、接受或出错
  - **转换表(GOTO)**：表示在当前状态下面临文法符号（可能是终结符或非终结符）时应转向的下一个状态
  - 把关于**终结符**部分的GOTO表和ACTION表重叠，也就是把当前状态下面临**终结符**应做的移进-归约动作和转向动作表示在一起
- 进行LR(0)分析前应构造LR(0)分析表

2. LR(0)分析

- LR(0)分析表的构造算法
  - **含S'→•S项目的项目集对应的状态为初始状态**
  - 分析表的ACTION表和GOTO表构造步骤为：
    - ✓ 若项目A→α•aβ∈k，**a∈V_T**，且**GOTO(k,a)=j**，则置**ACTION[k,a]='S_j'**，**移进**；
    - ✓ 若项目A→α•Bβ∈k，**B∈V_N**，且**GOTO(k,B)=j**，则置**GOTO[k,B]='j'**；
    - ✓ 若项目A→α•∈k，且产生式A→α的编号为**j**，则对任何a（终结符和'$'），置**ACTION[k,a]='$r_j$'**，**归约**；**对任何a都归约，不用考虑a具体是哪个字符——LR(0)的0之所在**
    - ✓ 若项目S'→**S**•∈k，**S是原文法开始符号**，则置**ACTION[k,$]='acc'**，**ACCEPT**；
    - ✓ 不能用上述方法填入的分析表的元素可空着，表示**ERROR**。

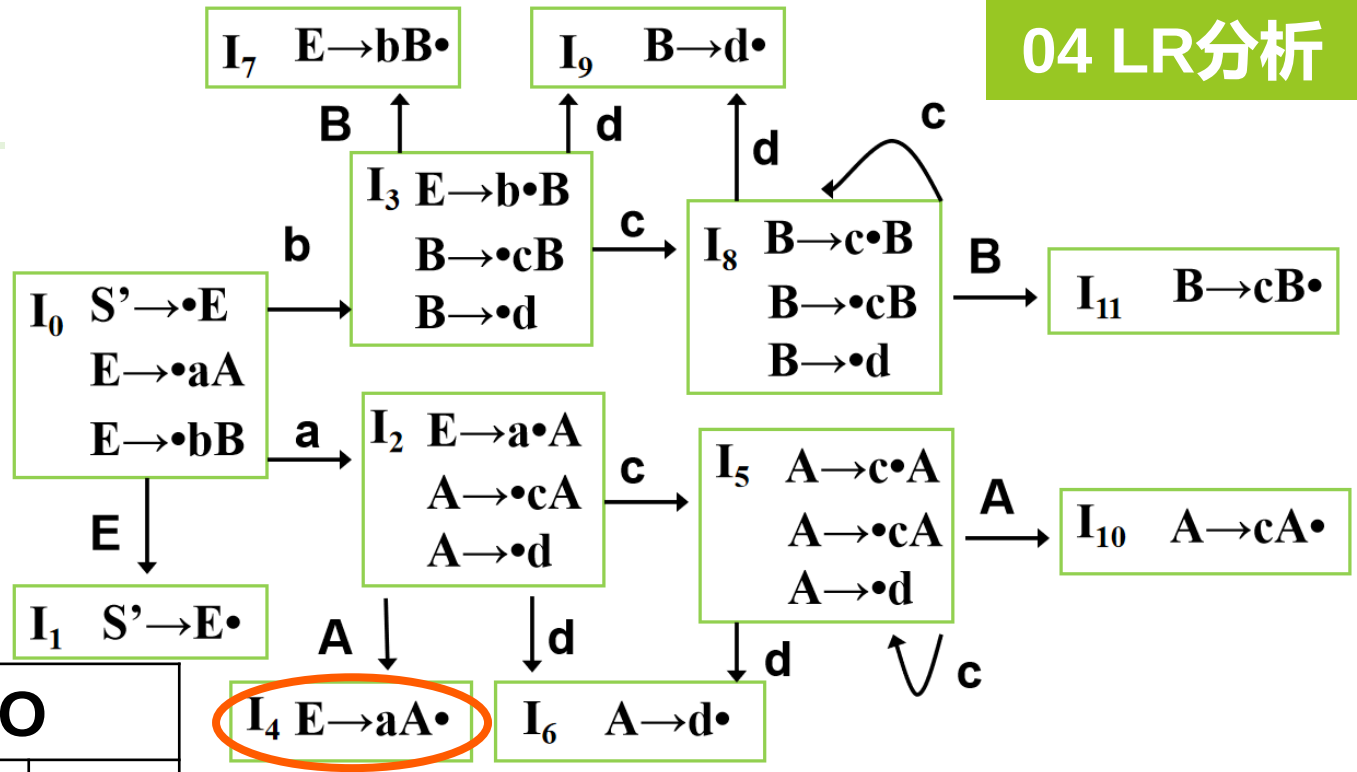
2. LR(0)分析

(0) S′→E　(1) E→aA

(2) E→bB　(3) A→cA

(4) A→d　(5) B→cB

(6) B→d



状态	ACTION					GOTO		
	a	b	c	d	$	E	A	B
0	S_2	S_3				1		
1					acc			
2			S_5	S_6			4	
3			S_8	S_9				7
4	r_1	r_1	r_1	r_1	r_1			
5			S_5	S_6			10	

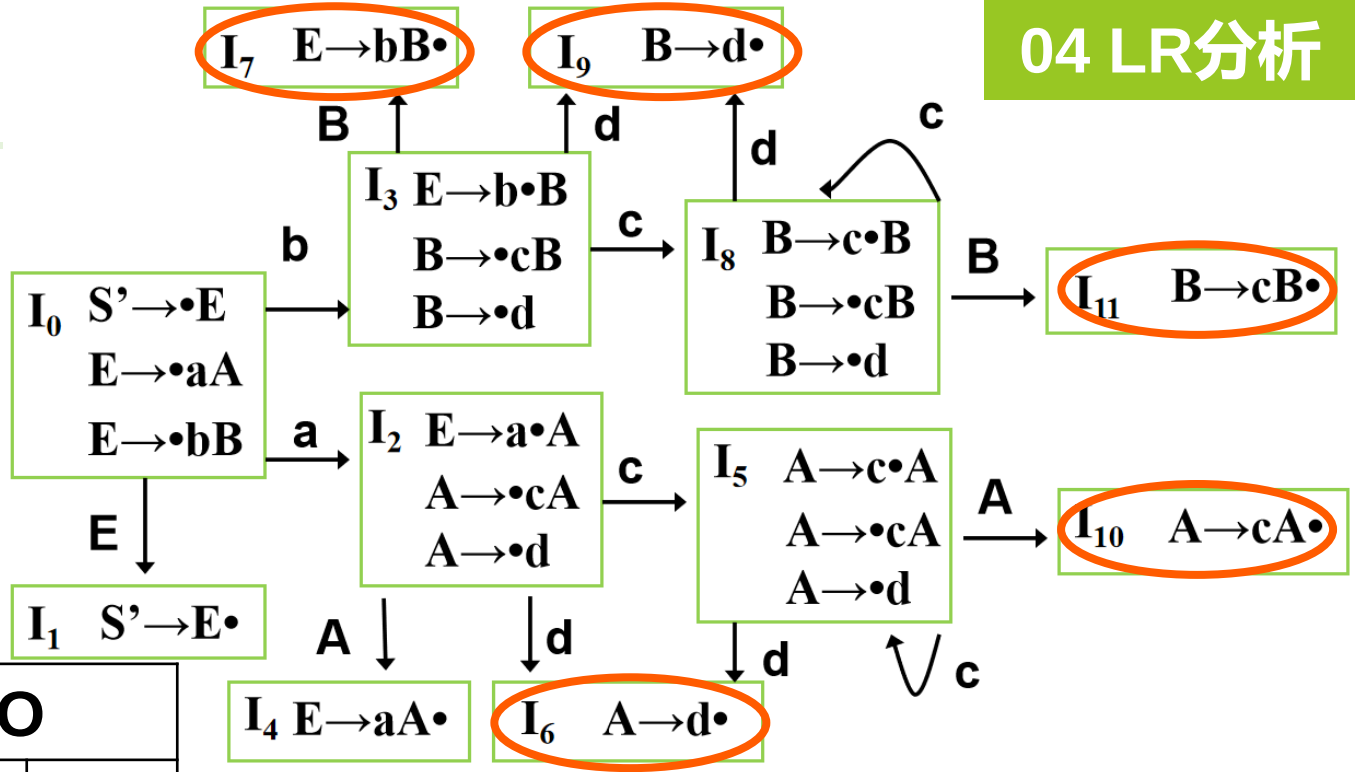
2. LR(0)分析

(0) S′→E　(1) E→aA

(2) E→bB　(3) A→cA

(4) A→d　(5) B→cB

(6) B→d



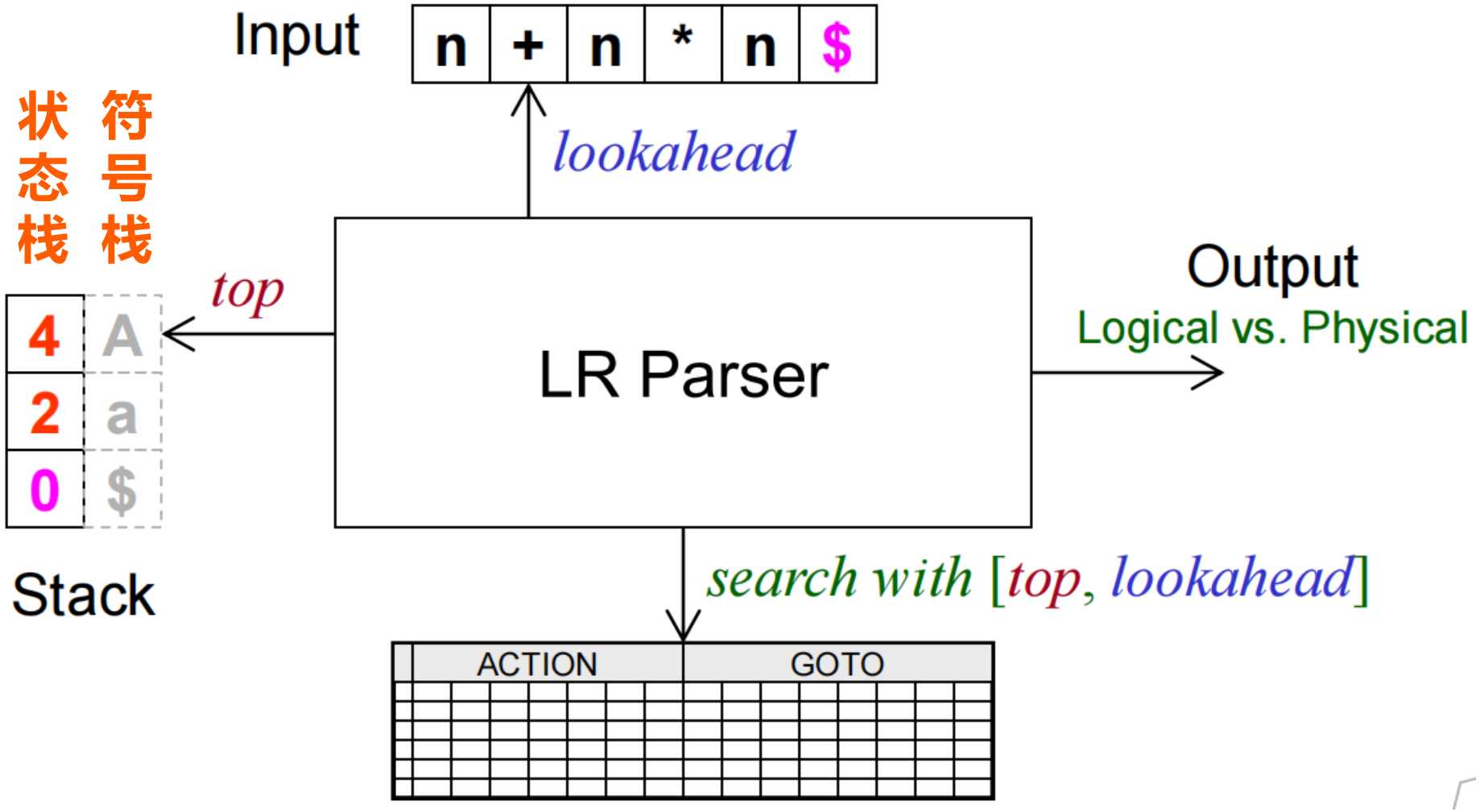
状态	ACTION					GOTO		
	a	b	c	d	$	E	A	B
6	r_4	r_4	r_4	r_4	r_4			
7	r_2	r_2	r_2	r_2	r_2			
8			S_8	S_9				11
9	r_6	r_6	r_6	r_6	r_6			
10	r_3	r_3	r_3	r_3	r_3			
11	r_5	r_5	r_5	r_5	r_5			

2. LR(0)分析

- LR(0)分析器
  - 使用LR(0)分析表的LR分析器称为LR(0)分析器
  - LR(**0**)分析器在分析的过程中**只根据符号栈的内容就能确定句柄，不需要向右查看输入符号**
  - 对文法的限制较大，对绝大多数高级语言的语法分析器不适用

2. LR(0)分析

- LR(0)分析器



2. LR(0)分析

- LR(0)分析器的工作过程
  - 根据**符号栈的栈顶状态**和**输入串的当前符号**查分析表确定应采取的动作（移进、归约、接受或报错），对**状态栈**和**符号栈**进行相应的操作。
    - ① 若ACTION[S,a]=S_j，a为终结符，则把**a移入**符号栈，**j移入**状态栈；
    - ② 若ACTION[S,a]=r_j，a为终结符或$，则：
      - 用第**j**个产生式(A->β)**归约**，将**两个栈弹出**k个元素，其中k为第j个产生式右部符号串β的长度，此时当前面临符号为第j个产生式左部的非终结符(A)；
      - 则非终结符**A移入符号栈**，且：若状态栈当前的栈顶状态为Q，且有GOTO[Q,A]=P，**P移入状态栈**；

      同进同出
    - ③ 若ACTION[S, $]=**acc**，则为**接受**，表明分析成功；
    - ④ 若ACTION[S,a]=**空白**，则转向**出错处理**。

2. LR(0)分析

- LR(0)分析器的工作过程

(0) S′→E　(1) E→aA

(2) E→bB　(3) A→cA

(4) A→d　(5) B→cB

(6) B→d

状态	ACTION					GOTO		
	a	b	c	d	$	E	A	B
0	S_2	S_3				1		
1					acc			
2			S_5	S_6			4	
3			S_8	S_9				7
4	r_1	r_1	r_1	r_1	r_1			
5			S_5	S_6			10	
6	r_4	r_4	r_4	r_4	r_4			
7	r_2	r_2	r_2	r_2	r_2			
8			S_8	S_9				11
9	r_6	r_6	r_6	r_6	r_6			
10	r_3	r_3	r_3	r_3	r_3			
11	r_5	r_5	r_5	r_5	r_5			

2. LR(0)分析 (0)S'→E (1)E→aA (2)E→bB (3)A→cA (4)A→d (5)B→cB (6)B→d

状态	ACTION					GOTO		
	a	b	c	d	$	E	A	B
0	S_2	S_3				1		
1					acc			
2			S_5	S_6			4	
3			S_8	S_9				7
4	r_1	r_1	r_1	r_1	r_1			
5			S_5	S_6			10	

状态	ACTION					GOTO		
	a	b	c	d	$	E	A	B
6	r_4	r_4	r_4	r_4	r_4			
7	r_2	r_2	r_2	r_2	r_2			
8			S_8	S_9				11
9	r_6	r_6	r_6	r_6	r_6			
10	r_3	r_3	r_3	r_3	r_3			
11	r_5	r_5	r_5	r_5	r_5			

输入串 bccd$

步骤	状态栈	符号栈	输入串	ACTION	GOTO
1	0	$	bccd$	S_3	
2	03	$b	ccd$	S_8	
3	038	$bc	cd$	S_8	
4	0388	$bcc	d$	S_9	
5	03889	$bccd	$	r_6	11

2. LR(0)分析 (0)S'→E (1)E→aA (2)E→bB (3)A→cA (4)A→d (5)B→cB (6)B→d

状态	ACTION					GOTO		
	a	b	c	d	$	E	A	B
0	S_2	S_3				1		
1					acc			
2			S_5	S_6			4	
3			S_8	S_9				7
4	r_1	r_1	r_1	r_1	r_1			
5			S_5	S_6			10	

状态	ACTION					GOTO		
	a	b	c	d	$	E	A	B
6	r_4	r_4	r_4	r_4	r_4			
7	r_2	r_2	r_2	r_2	r_2			
8			S_8	S_9				11
9	r_6	r_6	r_6	r_6	r_6			
10	r_3	r_3	r_3	r_3	r_3			
11	r_5	r_5	r_5	r_5	r_5			

输入串
bccd$

步骤	状态栈	符号栈	输入串	ACTION	GOTO
6	0388 (11)	$bccB	$	r_5	11
7	038 (11)	$bcB	$	r_5	7
8	037	$bB	$	r_2	1
9	01	$E	$	acc	

2. LR(0)分析

- 小结——LR(0)分析过程

① 对文法进行拓广：对文法G[S]，增加一条产生式S’→S，拓广为文法G’[S’]

② 根据产生式构造LR(0)项目集：CLOUSRE函数和GOTO函数

③ 根据项目集构造LR(0)DFA

④ 根据LR(0)DFA构造LR(0)分析表

⑤ 根据LR(0)分析表进行LR(0)分析

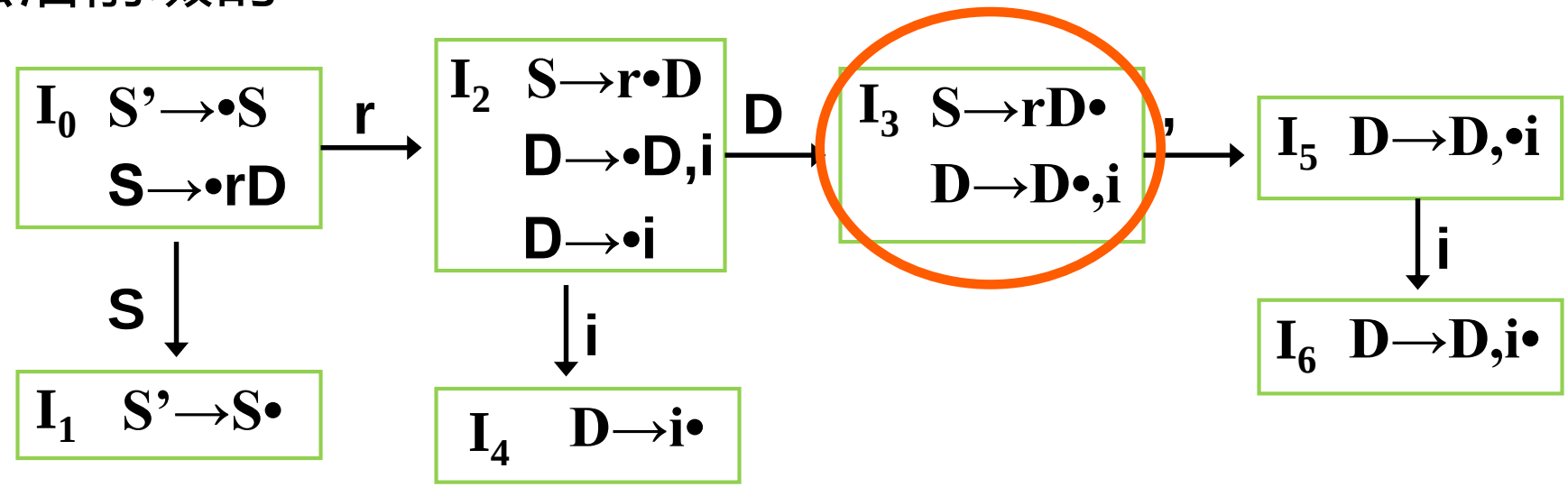
2. LR(0)分析

- 项目集中的冲突：一个项目集中存在下列情况称为项目冲突：
  - **移进-归约冲突**
    - ✓ 移进项目A→α•aβ和归约项目B→r•同在一个项目集中，当面临输入符a时，不能确定移进a还是把r归约为B；
  - **归约-归约冲突**
    - ✓ 归约项目A→β•和归约项目B→r•同在一个项目集中，不管面临什么输入符都不能确定把β归约为A还是把r归约为B。

一个文法的LR(0)项目集规范族中的项目集不存在移进-归约冲突，也不存在归约-归约冲突时，称该文法为LR(0)文法。

2. LR(0)分析

- 例：文法G'[S']：(0) S'→S　(1) S→rD　(2) D→D,i　(3) D→i
- 识别文法活前缀的DFA：



不管面临什么输入符号，都归约

当面临输入为","时，移进

在项目集I_3中S→rD•为**归约项目**，D→D•,i为**移进项目**，

存在移进-归约冲突，该文法不是LR(0)文法。

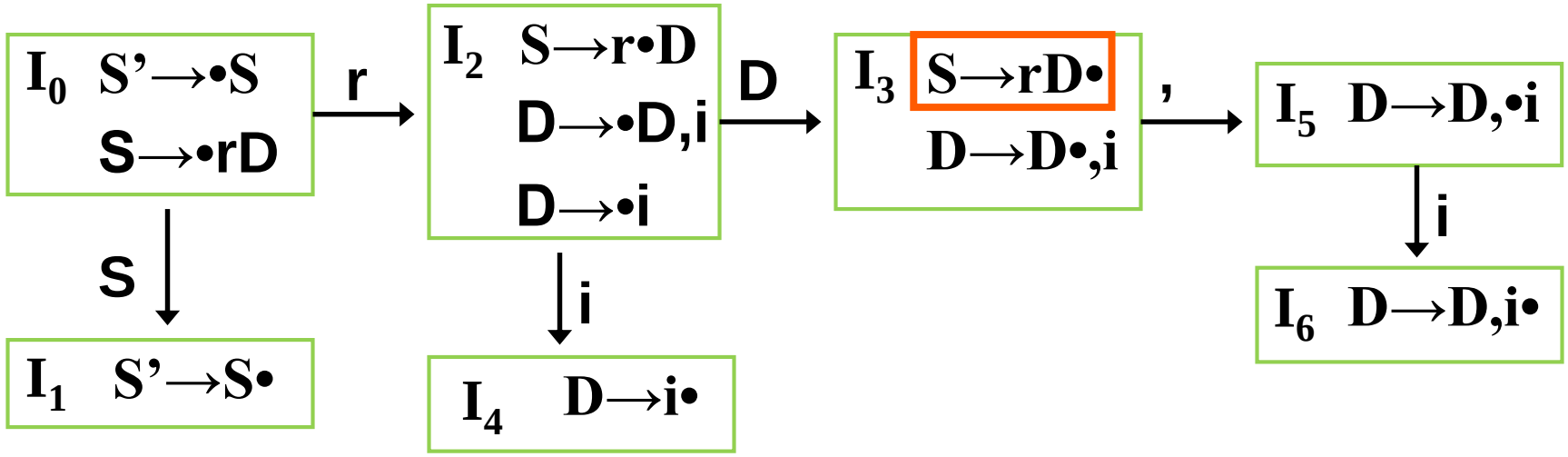
3. SLR(1)分析

- SLR(1)分析方法可解决这种冲突
  - 基本思想：
    - ✓ 对于LR(0)有**冲突**的**项目集**用**向前查看输入符号串的1个符号**的办法加以解决
  - 解决方法：
    - ✓ 对归约项目A→r•，**只有当输入符号a∈FOLLOW(A)才进行归约**，缩小归约范围，有可能解决冲突。

而不是不管面临什么输入符号都归约

3. SLR(1)分析

- 例：文法G'[S']：(0) S'→S　(1) S→rD　(2) D→D,i　(3) D→i
- 识别文法活前缀的DFA：



	Follow集
S'	$
S	$
D	$,

对项目集I_3：
当前输入符为$时，进行归约；
当前输入符为,时，进行移进操作；
冲突得以解决。

3. SLR(1)分析

- SLR(1)的处理方法：
  - 一个LR(0)项目集I含有冲突项目I={X→α•**b**β，**A**→r•，**B**→δ•}，其中b∈V_T，如果FOLLOW(A)和FOLLOW(B)互不相交，且不含b，则当状态I面临某输入符a时，可采用下列动作：
    - ✓ 若a=b，则移进；
    - ✓ 若a∈**FOLLOW(A)**，则用产生式A→r归约；
    - ✓ 若a∈**FOLLOW(B)**，则用产生式B→δ归约；
    - ✓ 此外，报错。

上述解决项目集(状态)中冲突的方法称为SLR(1)方法（Simple因为只对有冲突的状态才向前查看一个符号，以确定动作）

3. SLR(1)分析

- 如果一个文法的LR(0)项目集中某些项目集所含有的**动作冲突都能用SLR(1)方法解决**，则称这个文法为**SLR(1)文法**
- SLR(1)分析表的构造算法
  - 与LR(0)分析表的构造类似
  - **仅对表格中有冲突的行采用SLR(1)方法区分移进或归约对应的列**
- 改进的SLR(1)分析表的构造算法
  - 对所有的归约项目仅对**当前输入符号**包含在该**归约项目左部非终结符的FOLLOW集**中，才采取归约动作。

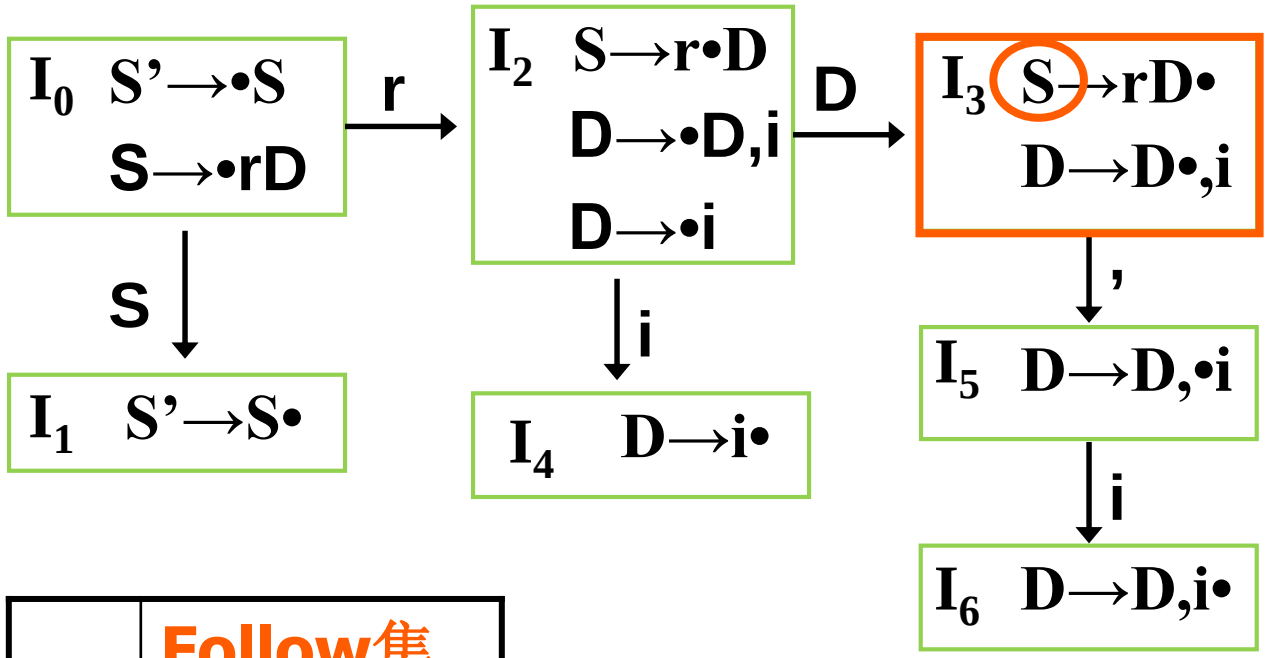
3. SLR(1)分析

- SLR(1)分析表的构造方法：
  - 含S'→•S项目的项目集为初态
  - 分析表的ACTION表和GOTO表构造步骤为：
    - ✓ 若项目A→α•aβ∈k，a∈V_T，且GOTO(k,a)=j，则置ACTION[k,a]='S_j'，移进；
    - ✓ 若项目A→α•Bβ∈k，B∈V_N，且GOTO(k,B)=j，则置GOTO[k,B]='j'；
    - ✓ 若项目A→α•∈k，且产生式A→α的编号为j，则对任何a(终结符和'$')，**且a∈FOLLOW(A)**，置ACTION[k,a]='r_j'，归约；

      与LR(0)的唯一区别，SLR(1)的1之所在
    - ✓ 若项目S'→S•∈k，则置ACTION[k,$]='acc'，ACCEPT；
    - ✓ 不能用上述方法填入的分析表的元素可空着，表示ERROR。

3. SLR(1)分析

- 例：文法G'[S']：(0) S'→S　(1) S→rD　(2) D→D,i　(3) D→i



	Follow集
S'	$
S	$
D	$,

状态	ACTION				GOTO	
	r	,	i	$	S	D
0	S_2				A	
1				acc		
2			S_4			3
3		S_5		r_1		
4		r_3		r_3		
5			S_6			
6		r_2		r_2		

3. SLR(1)分析

- 仍有许多文法构造的LR(0)项目集规范族存在的动作冲突不能用SLR(1)方法解决

文法G[S']:

(0) S' →S

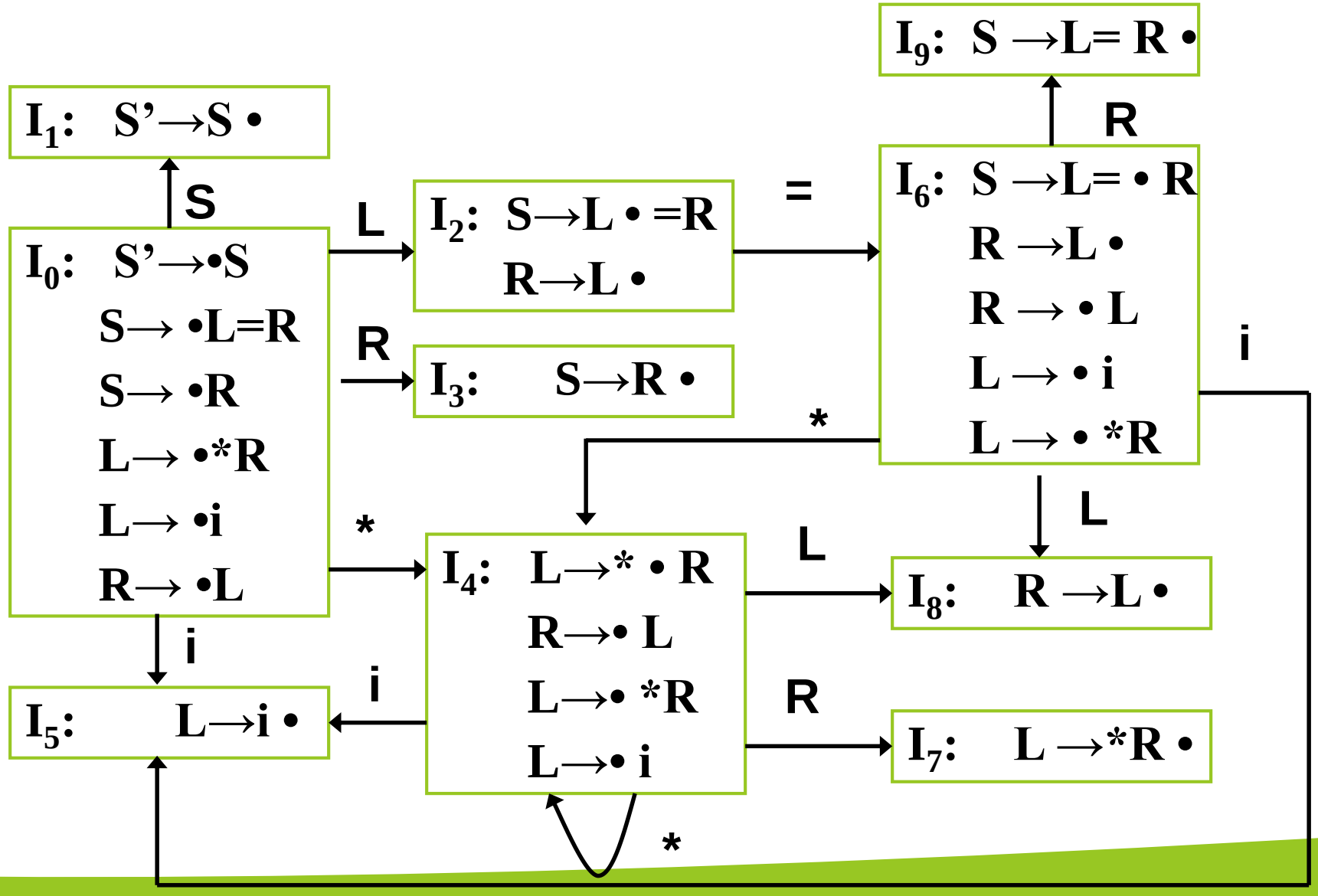
(1) S → L=R

(2) S →R

(3) L →*R

(4) L →i

(5) R →L



3．SLR(1)分析

S' →S

S →L=R |R

L →*R | i

R →L

	Follow集
S’	$
S	$
L	= $
R	$ =

I_6: S →L= • R

R →L •

R → • L

L → • i

L → • *R

Follow(R)={$,=}与移进符号集{ *, i}交集为空，可用SLR(1)方法解决冲突

I_2: S →L • =R

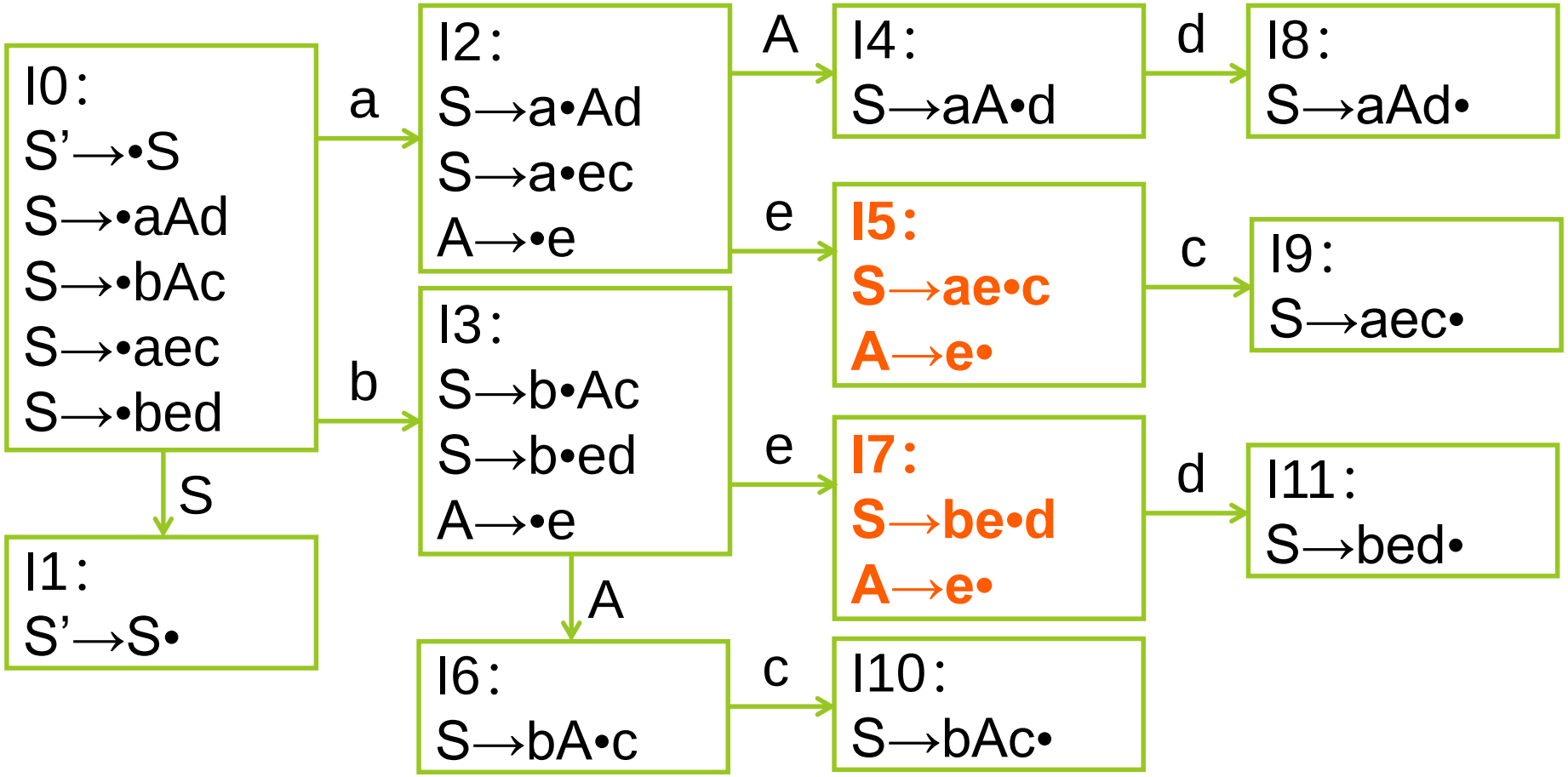
R →L •

Follow(R)={$,=}与移进符号集{=}交集不为空，SLR(1)方法不能解决冲突，该文法不是SLR(1)文法。

3. SLR(1)分析

- 例：文法G'[S']:

(0) S'→S (1) S→aAd (2) S→bAc (3) S→aec (4)S →bed (5) A→e



3. SLR(1)分析

- 例：文法G'[S']:

(0) S'→S (1) S→aAd (2) S→bAc (3) S→aec (4)S →bed (5) A→e

I5：
S→ae•c
A→e•

FOLLOW(A)={c,d}与移进符号集{c}交集不为空，
SLR(1)仍无法解决冲突。

I7：
S→be•d
A→e•

FOLLOW(A)={c,d}与移进符号集{d}交集不为空，
SLR(1)仍无法解决冲突。

3. SLR(1)分析

- SLR(1)用FOLLOW信息作为展望信息（只对属于FOLLOW集的输入符号归约），缩小了归约范围，消除了一些无效归约，解决了项目集中的一些简单的冲突。

- 尽管FOLLOW(A)中包含了所有含A的句型中A后的可能终结符，但**Follow(A)可能与移进符号有交集**，且**并不是每个含有A的句型中，A的后面都可以出现Follow(A)中的每一个符号**，所以SLR(1)未能从根本上消除所有无效归约。

4. LR(1)分析

- LR(1)分析的基本思想
  - LR(1)方法按每个具体的句型设置展望信息。
  - 例：如果存在如下句型…αAa…，…βAb…，…γAc…，则

    FOLLOW(A)={a,b,c}
    - ✓处理到句型…αA，只当输入符号为a时归约；
    - ✓处理到句型…βA，只当输入符号为b时归约；
    - ✓处理到句型…γA，只当输入符号为c时归约。

4. LR(1)分析

- LR(1)项目
  - 在 LR(0)项目基础上增加一个终结符，所增加的终结符称为向前搜索符（lookahead），表示产生式右端完整匹配后所允许在剩余符号串中的下一个终结符
  - LR(1)项目形如：[A→α•β, **a**]，其中，A→α•β同LR(0)项目，**a为向前搜索符，或为终结符，或为输入结束标志符$**
  - 对于形如：[A→α•，a]的LR(1)项目，对应LR(0)的归约项目，但只有当下一个输入符是a时才能进行归约
  - 对于其它形式的LR(1)项目，a只起到信息传承的作用

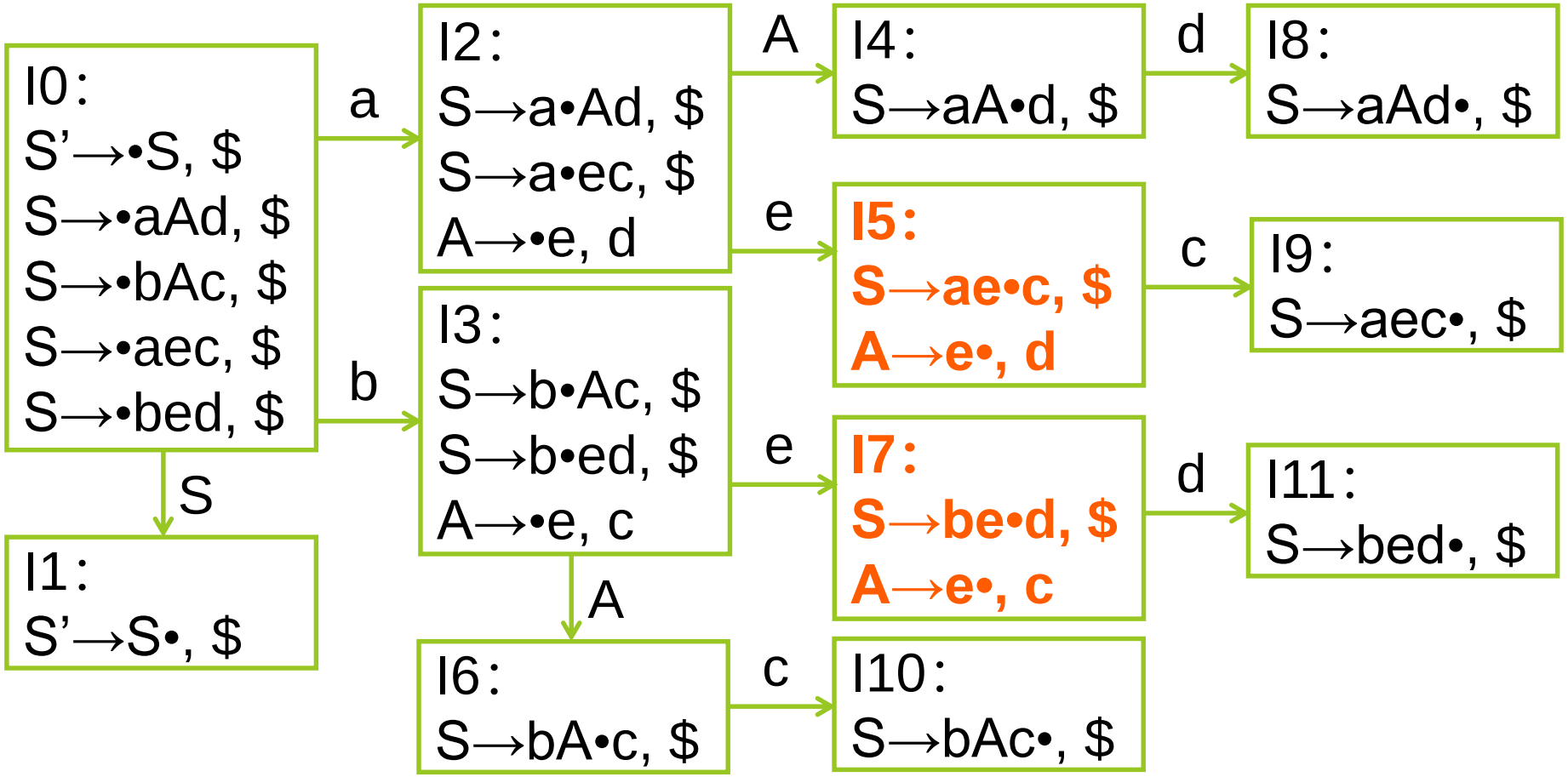
4. LR(1)分析

- 初始项目集
  - [S'→•S，$]属于初始项目集，此时，把$作为向前搜索符，表示要把S产生式的右部归约成S时，必须面临输入符为$才行
- CLOSURE函数
  - ① 如果I是一个项目集，I的任何项目都属于CLOSURE(I)
  - ② 若有项目[A→α•Bβ, a]属于CLOSURE(I)，B→γ是文法的产生式，β∈V*，**b∈FIRST(βa)**，则[B→•γ, **b**]也属于CLOSURE(I)
  - ③ 重复②直到CLOSURE(I)不再扩大为止。
- GOTO函数
  - 与LR(0)相似
  - GOTO(I,X)=CLOSURE(J)，J={任何形如[A→αX•β，a]的项目|[A→α•Xβ，a]∈I}

4. LR(1)分析

- 例：文法G'[S']:

(0) S'→S (1) S→aAd (2) S→bAc (3) S→aec (4)S →bed (5) A→e



4. LR(1)分析

- 例：文法G'[S']:

(0) S'→S (1) S→aAd (2) S→bAc (3) S→aec (4)S →bed (5) A→e

I5:
S→ae•c, $
A→e•, d

遇到输入字符c时，移进；
遇到输入字符d时，用产生式A→e归约；
冲突得以解决。

I7:
S→be•d, $
A→e•, c

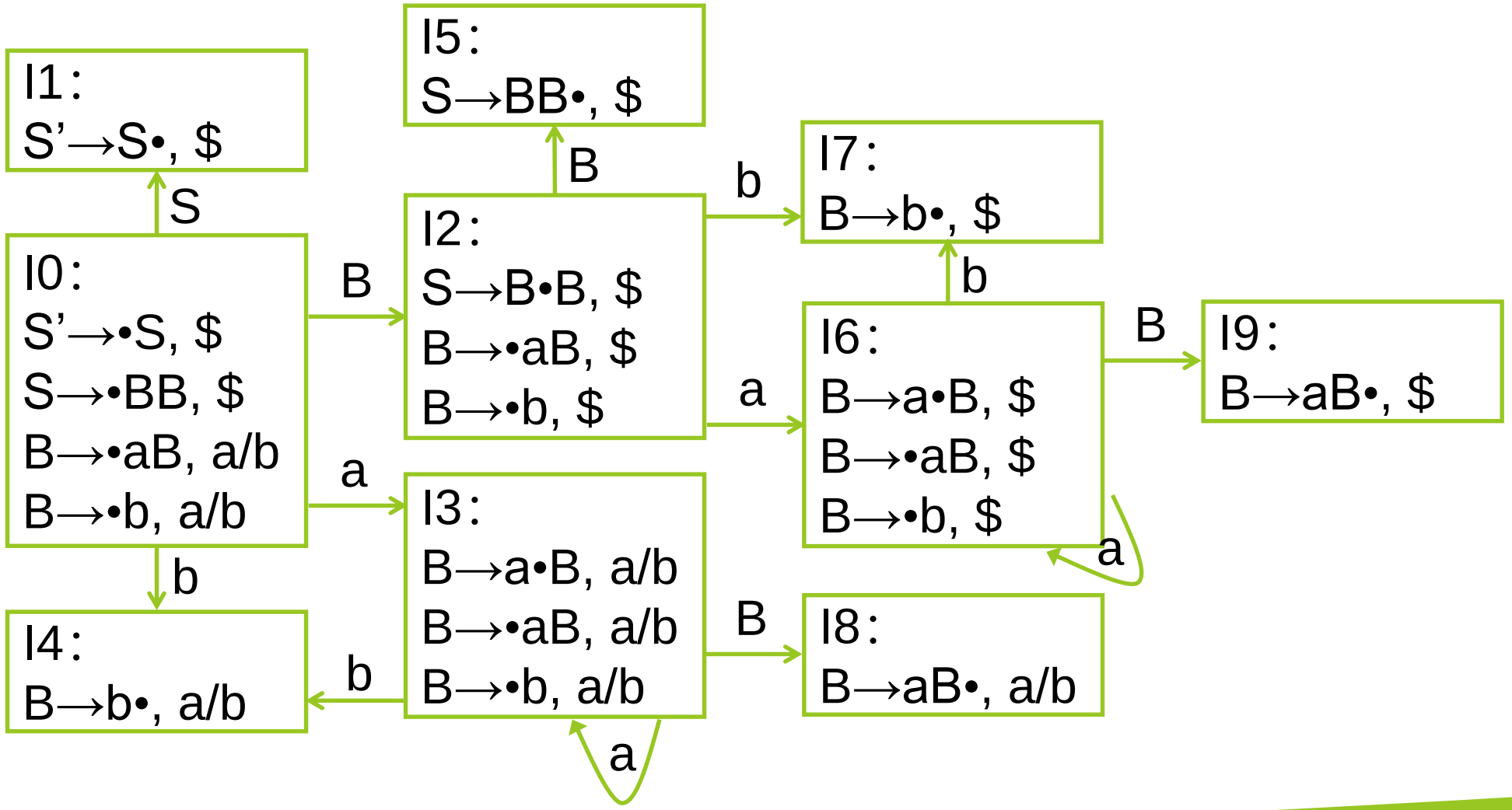
遇到输入字符d时，移进；
遇到输入字符c时，用产生式A→e归约；
冲突得以解决。

4. LR(1)分析

- 如果一个文法的LR(1)项目集中没有移进-归约或归约-归约冲突，则称该文法为LR(1)文法。

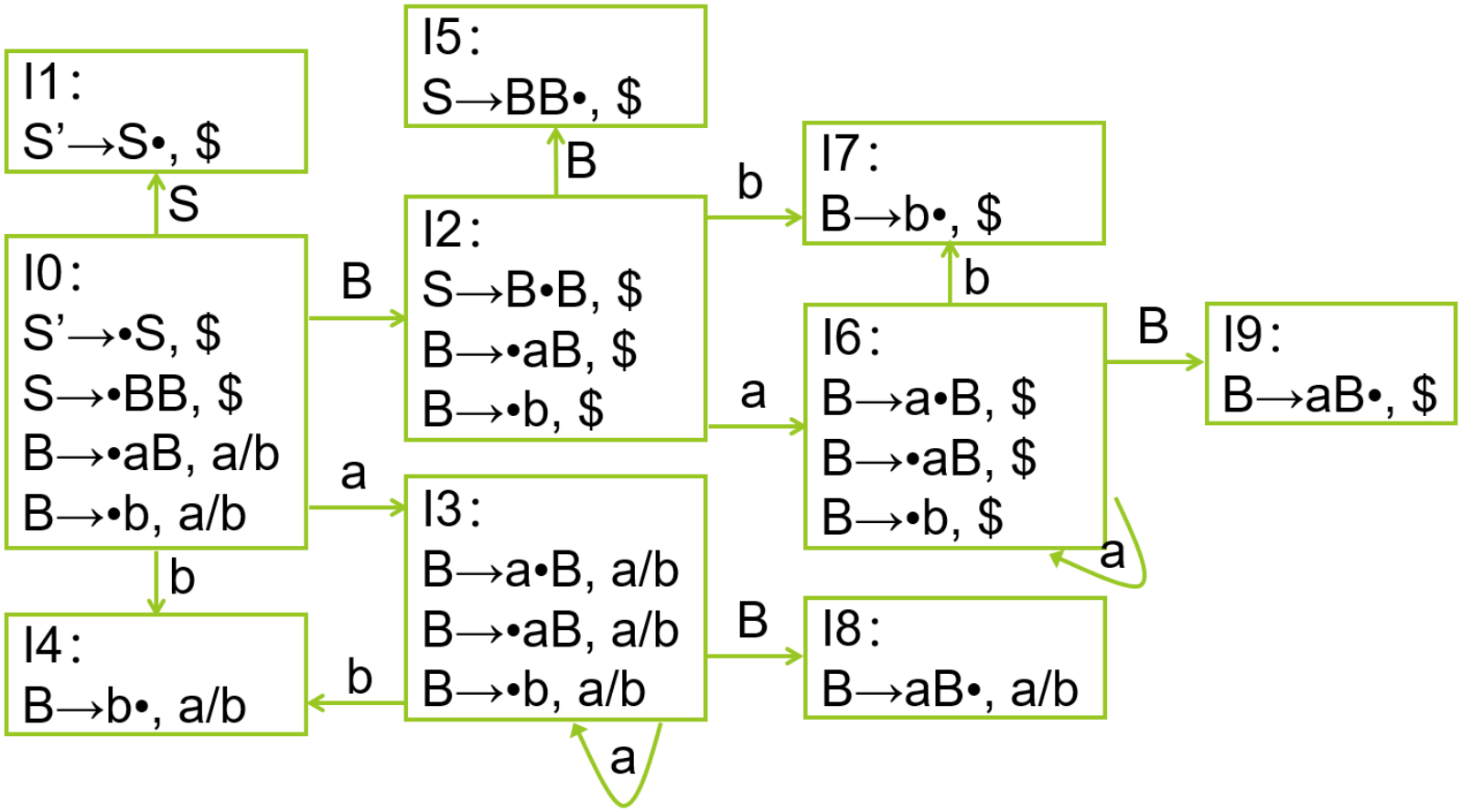
随堂练习（9）

- 若文法G'为：(0) S'→S (1) S→BB (2) B→aB (3) B→b，请画出其对应的LR(1)DFA。



4. LR(1)分析

- 构造LR(1)分析表
  - G': (0)S'→S (1)S→BB (2)B→aB (3)B→b

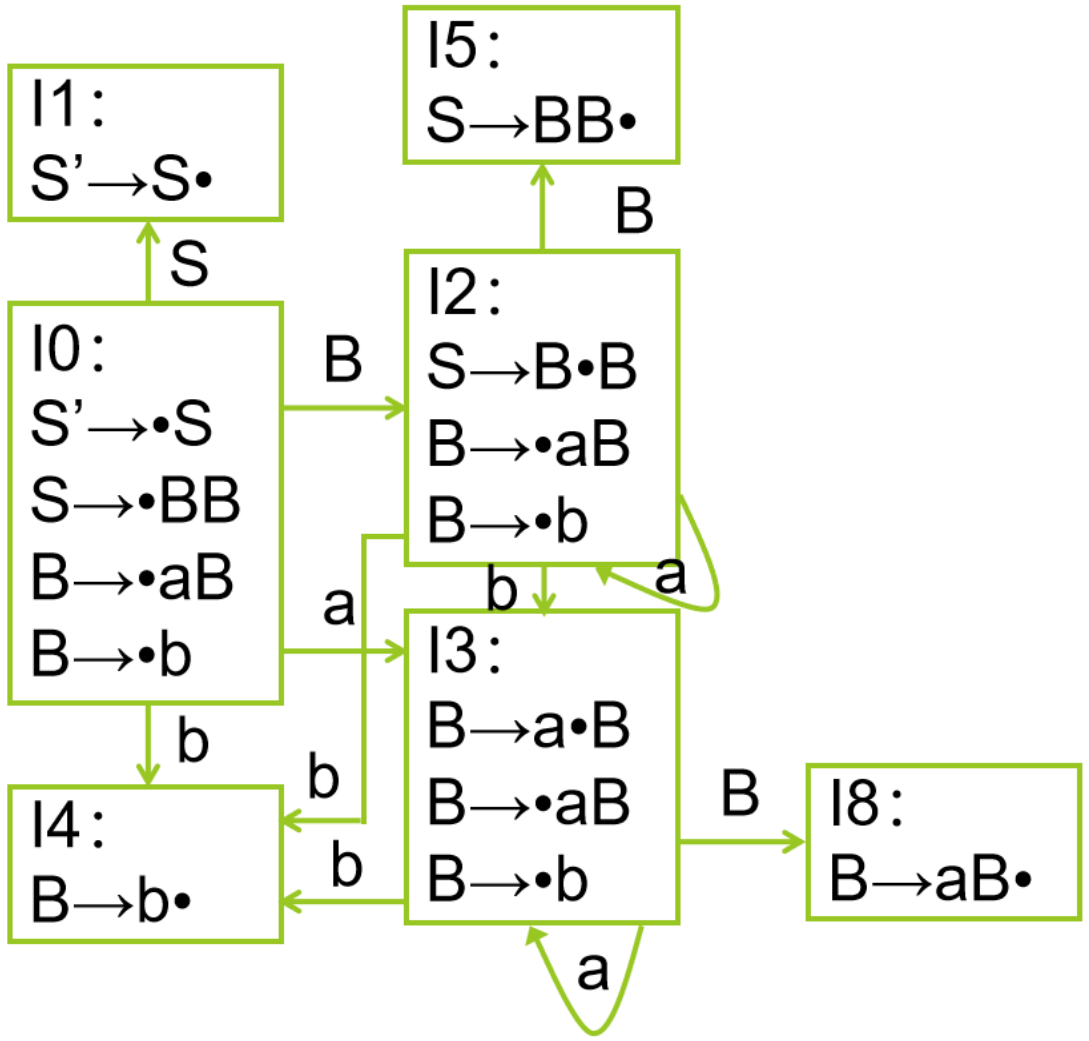


状态	ACTION			GOTO	
	a	b	$	S	B
0	S_3	S_4		1	2
1			acc		
2	S_6	S_7			5
3	S_3	S_4			8
4	r_3	r_3			
5			r_1		
6	S_6	S_7			9
7			r_3		
8	r_2	r_2			
9			r_2		

发现在该例子中，即使不考查搜索符，也不存在冲突
——实际上是一个LR(0)文法

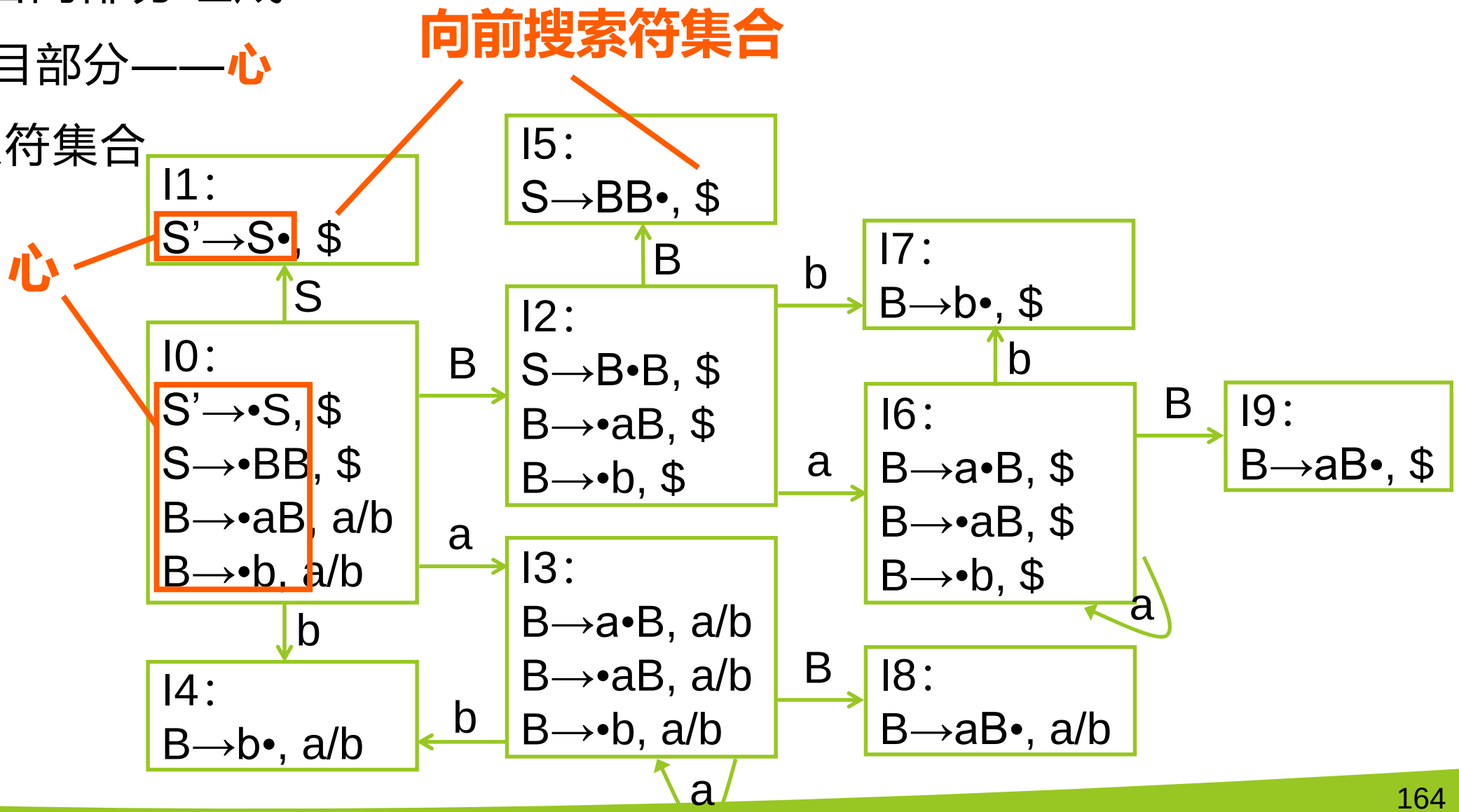
4. LR(1)分析

- 一个文法是LR(0)文法，就一定也是SLR(1)文法，也是LR(1)文法，反之则不一定成立
- 上述例子的LR(0)项目集，仅有7个状态
- LR(1)的优缺点：
  - 优点：LR(1)分析对搜索符的计算方法比较确切，能消除更多的无效归约，适应的文法更广
  - 缺点：LR(1)分析表的状态数目庞大，可能存在项目集的冗余——**合并，LALR(1)**；LR(1)分析方法也不能解决所有冲突——**LR(k), k>2**。



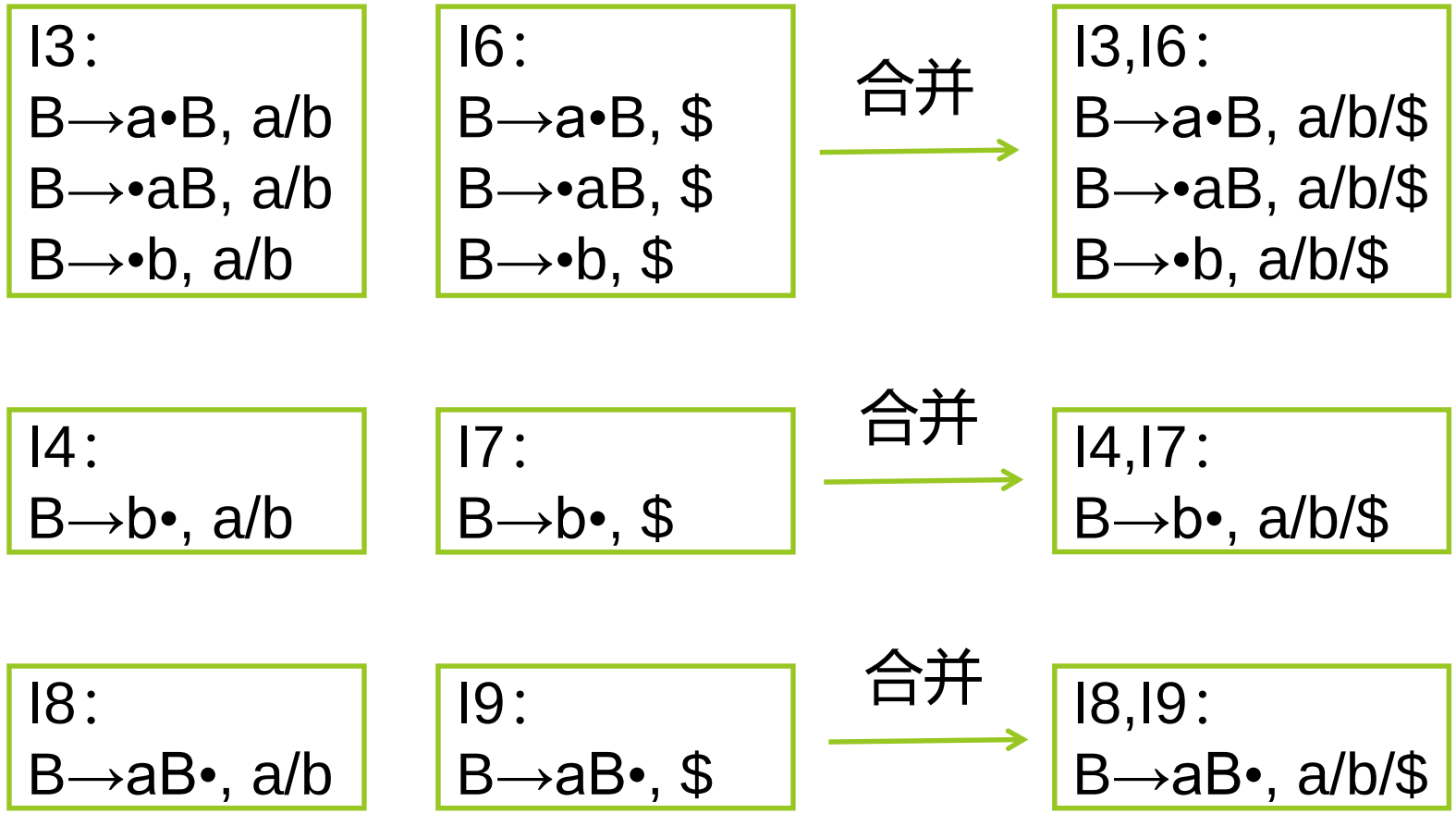
5. LALR(1)分析

- LR(1)项目由两部分组成：
  - LR(0)项目部分——心
  - 向前搜索符集合



5. LALR(1)分析

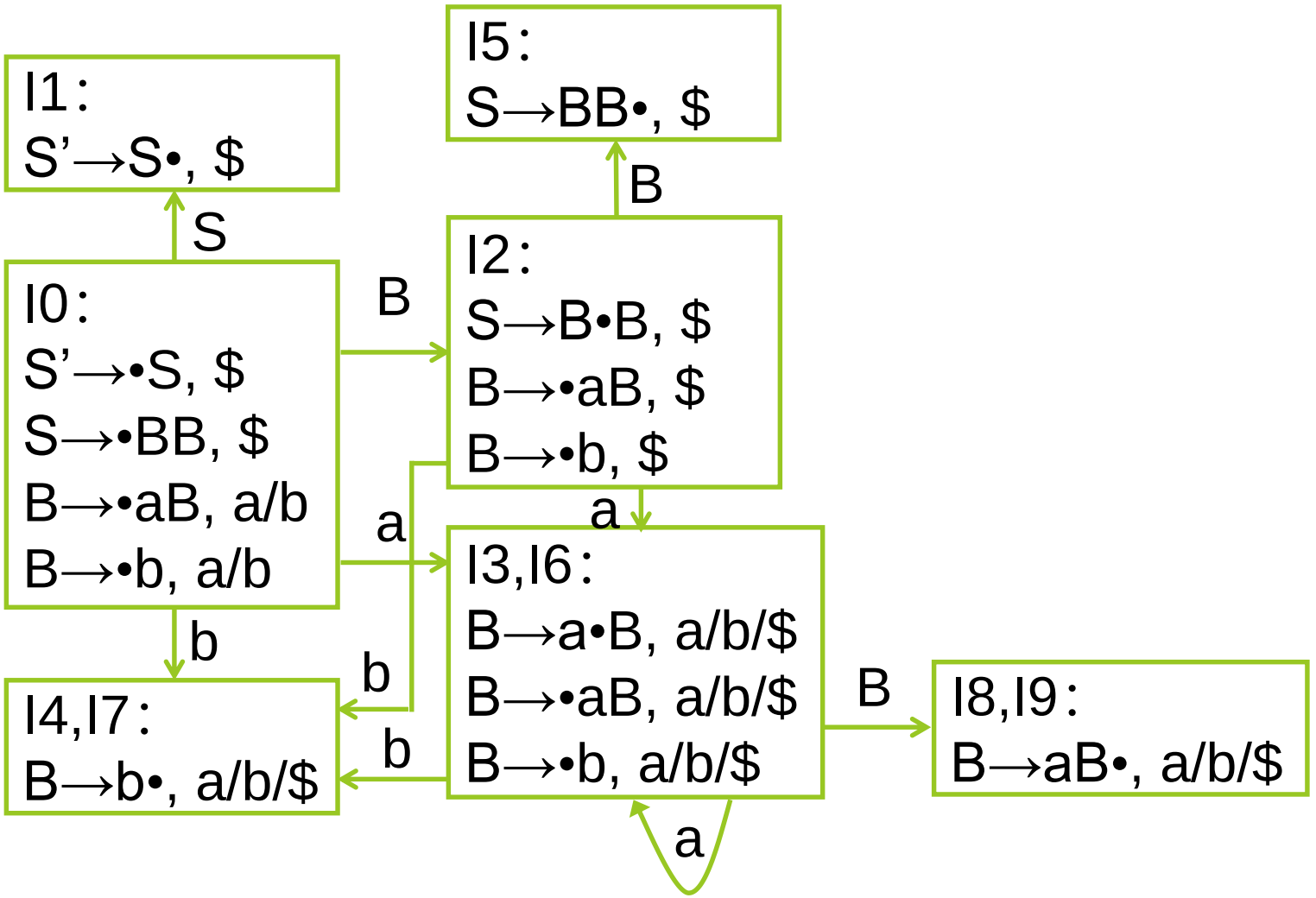
- 对同心的项目集，合并——**合并同心集**



同心集合并后不包含冲突，是LALR(1)项目集，文法是LALR(1)文法，可以用LALR(1)方法分析

5. LALR(1)分析

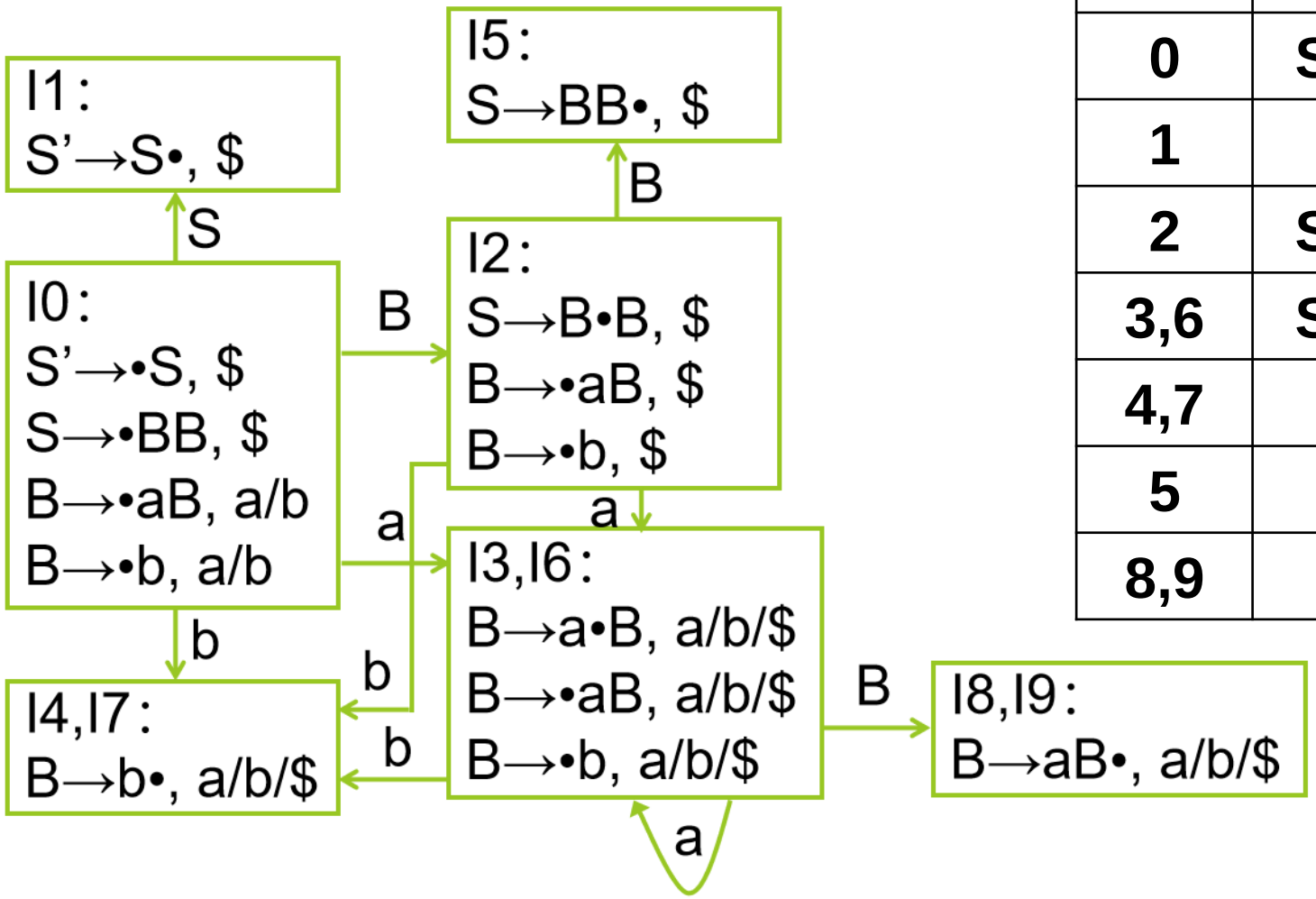
- 构造LALR(1)分析的DFA（合并同心集后）



5. LALR(1)分析

- 构造LALR(1)预测分析表

(0)S'→S (1)S→BB (2)B→aB (3)B→b



状态	ACTION			GOTO	
	a	b	$	S	B
0	$S_{3,6}$	$S_{4,7}$		1	2
1			acc		
2	$S_{3,6}$	$S_{4,7}$			5
3,6	$S_{3,6}$	$S_{4,7}$			8,9
4,7	r_3	r_3	r_3		
5			r_1		
8,9	r_2	r_2	r_2		

5. LALR(1)分析

- 利用LALR(1)分析对某些错误发现的时间可能会产生推迟，但错误的出现位置仍是准确的

(0)S'→S (1)S→BB (2)B→aB (3)B→b

对输入串ab$

步骤	状态栈	符号栈	输入串	ACTION	GOTO
1	0	$	ab$	$S_{3,6}$	
2	0(3,6)	$a	b$	$S_{4,7}$	
3	0(3,6)(4,7)	$ab	$	r_3	8,9
4	0(3,6)(8,9)	$aB	$	r_2	2
5	02	$B	$	ERROR	

状态	ACTION			GOTO	
	a	b	$	S	B
0	$S_{3,6}$	$S_{4,7}$		1	2
1			acc		
2	$S_{3,6}$	$S_{4,7}$			5
3,6	$S_{3,6}$	$S_{4,7}$			8,9
4,7	r_3	r_3	r_3		
5			r_1		
8,9	r_2	r_2	r_2		

第5步才发现错误!

5. LALR(1)分析

- 若利用LR(1)分析表进行分析：

(0)S'→S (1)S→BB (2)B→aB (3)B→b

对输入串ab$

步骤	状态栈	符号栈	输入串	ACTION	GOTO
1	0	$	ab$	S_3	
2	03	$a	b$	S_4	
3	034	$ab	$	ERROR	

第3步即发现错误！

原因：LALR(1)分析合并同心集后，向前搜索符集合扩大了，因此推迟发现错误

状态	ACTION			GOTO	
	a	b	$	S	B
0	S_3	S_4		1	2
1			acc		
2	S_6	S_7			5
3	S_3	S_4			8
4	r_3	r_3			
5			r_1		
6	S_6	S_7			9
7			r_3		
8	r_2	r_2			
9			r_2		

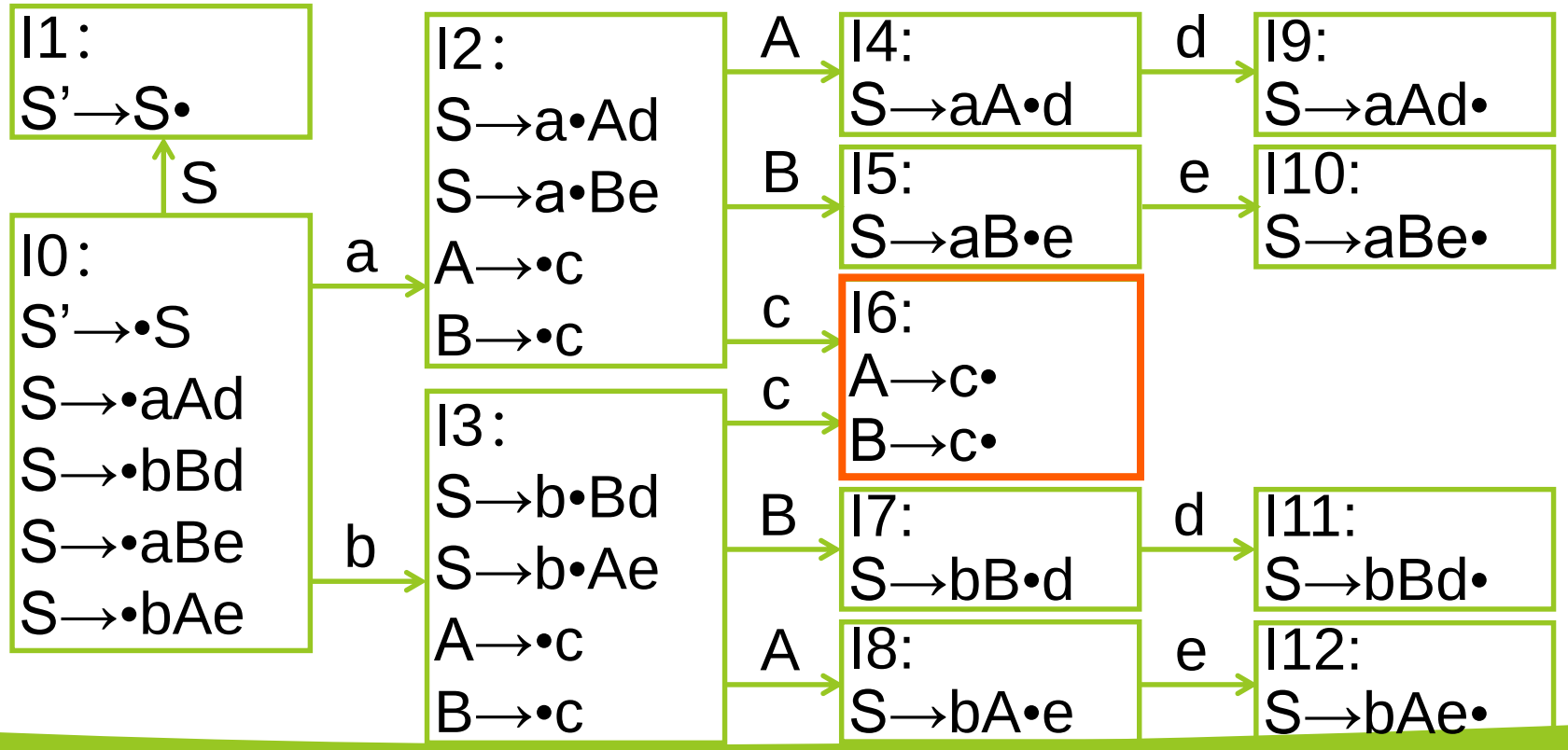
随堂练习（10）

- 请验证以下文法是LR(0)文法或SLR(1)文法或LR(1)文法或LALR(1)文法

文法G'[S']:

(0)S'→S (1)S→aAd (2)S→bBd (3)S→aBe (4)S→bAe (5)A→c (6)B→c

1. 验证是否为LR(0)文法：



存在归约-归约冲突
不是LR(0)文法

随堂练习（10）

- 请验证以下文法是LR(0)文法或SLR(1)文法或LR(1)文法或LALR(1)文法

文法G’[S’]:

(0)S’→S (1)S→aAd (2)S→bBd (3)S→aBe (4)S→bAe (5)A→c (6)B→c

2. 验证是否为SLR(1)文法：

I6:
A→c•
B→c•

FOLLOW(A)={d,e}
FOLLOW(B)={d,e}

FOLLOW(A)={c,d}与FOLLOW(B)={d,e}交集不为空，SLR(1)仍无法解决冲突，不是SLR(1)文法。

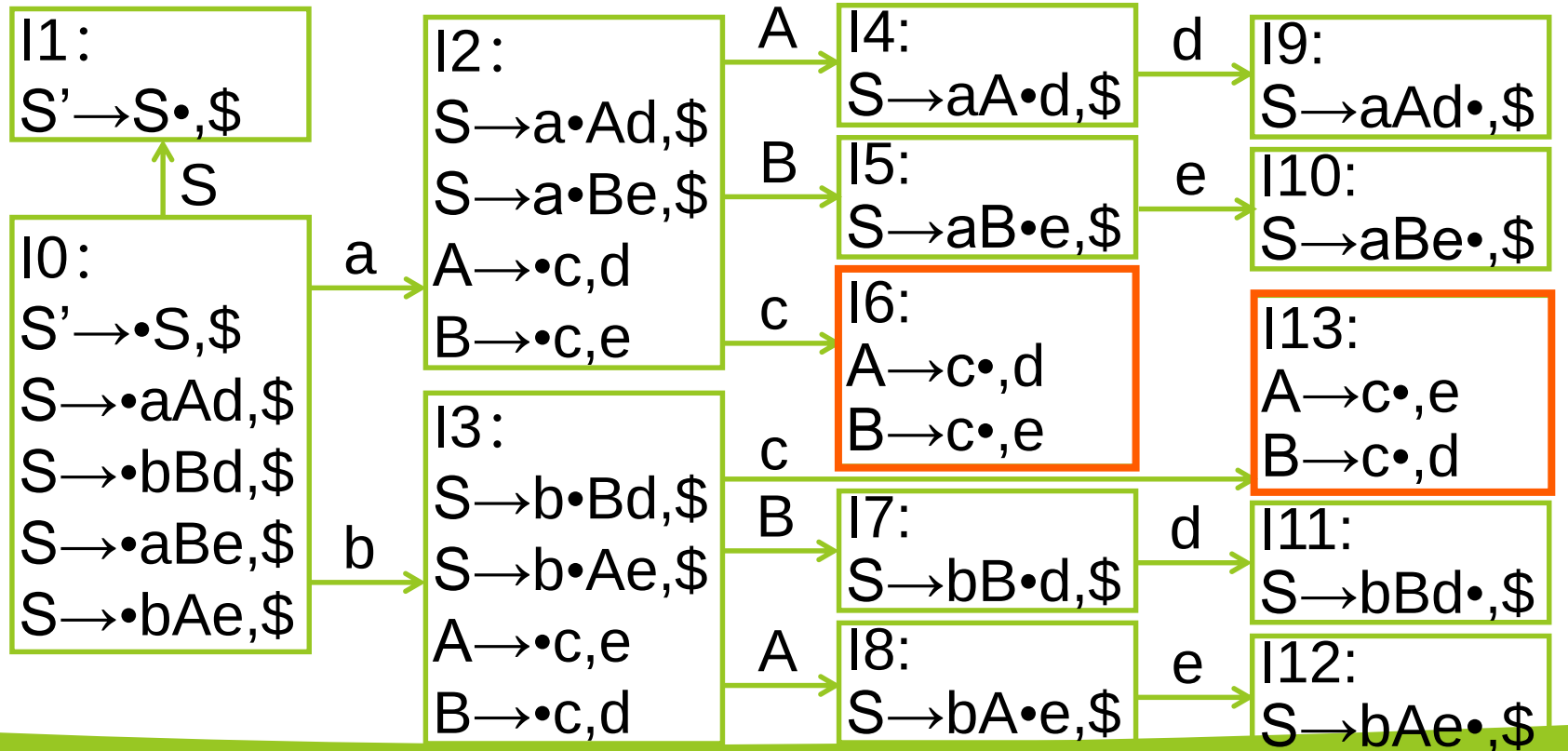
随堂练习（10）

- 请验证以下文法是LR(0)文法或SLR(1)文法或LR(1)文法或LALR(1)文法

文法G'[S']:

(0)S'→S (1)S→aAd (2)S→bBd (3)S→aBe (4)S→bAe (5)A→c (6)B→c

3. 验证是否为LR(1)文法：



归约-归约冲突得以解决，是LR(1)文法

随堂练习（10）

- 请验证以下文法是LR(0)文法或SLR(1)文法或LR(1)文法或LALR(1)文法

文法G'[S']:

(0)S'→S (1)S→aAd (2)S→bBd (3)S→aBe (4)S→bAe (5)A→c (6)B→c

4. 可再进一步验证是否为LALR(1)文法：

I6:	I13:	合并	I6,13:
A→c•,d	A→c•,e	→	A→c•,d/e
B→c•,e	B→c•,d		B→c•,e/d

合并同心集后出现归约-归约冲突，不是LALR(1)文法

结论：该文法是LR(1)文法，非LR(0)、SLR(1)和LALR(1)文法。

5. LALR(1)分析

- LALR(1)分析
  - LALR(1)是LR(1)的优化版本，对LR(1)项目集规范族**合并同心集**，若合并同心集后不产生新的冲突，则为LALR(1)项目集
  - LALR(1)分析可以大大减少项目集（状态）的数目。
  - LALR(1)方法是介于SLR(1)和LR(1)之间的一种方法，即其功能比LR(0)和SLR(1)强，比LR(1)弱。
  - 它具有LR(0)和SLR(1)的状态数少的优点和LR(1)的适用范围广的优点。
  - 能处理大多数编程语言语法
  - Yacc/Bison等工具默认使用它

LR(0)/SLR(1)/LALR(1)/LR(1)小结

- LR(**0**):
  - **无需向前查看**任何输入字符，仅依赖当前状态即可归约;
  - 分析表中一整行均为r_j;
  - 如果存在冲突，即，同一个项目集中既有归约项目，又有移进项目，或存在多条归约项目，则可尝试采用SLR(1)方法解决;

状态	ACTION					GOTO		
	a	b	c	d	$	E	A	B
0	S_2	S_3				1		
1					acc			
2			S_5	S_6			4	
3			S_8	S_9				7
4	r_1	r_1	r_1	r_1	r_1			
5			S_5	S_6			10	

I_3 $S \rightarrow rD\bullet$
$D \rightarrow D\bullet,i$

移进-归约冲突

LR(0)/SLR(1)/LALR(1)/LR(1)小结

- SLR(1)：
  - 在LR(0)的基础上，其他不变，仅当**当前输入字符∈FOLLOW(A)时才归约**；
  - 分析表中只有FOLLOW(A)中字符所在的列填写r_j；比LR(0)更精确；
  - **当移进符号集和归约符号集无交集时，可解决LR(0)冲突**；
  - 否则，仍存在冲突，可尝试采用LR(1)方法解决。

I3：
S→rD•
D→D•,i

FOLLOW(S)={$}

当前输入符为$时，进行归约；
当前输入符为,时，进行移进操作；
冲突得以解决。

I5：
S→ae•c
A→e•

FOLLOW(A)={c,d}

FOLLOW(A)={c,d}与移进符号集{c}交集不为空，SLR(1)仍无法解决冲突。

LR(0)/SLR(1)/LALR(1)/LR(1)小结

- LR(**1**):
  - 在LR(0)的基础上，每个项目**显式**增加一个**向前搜索符号集**[A→α•β, **a**];
  - 该符号集由**FIRST集**计算而得，或由信息传递而得;
    - ✓若有项目[A→α•Bβ，a]属于CLOSURE(I)，B→γ是文法的产生式，β∈V*，**b∈FIRST(βa)**，则[B→•γ, **b**]也属于CLOSURE(I)
  - 分析表中只有向前搜索符号集所在列填写r_j，比SLR(1)更精确;
  - 可解决绝大部分冲突;
  - 但也无法保证解决所有冲突，若仍有冲突，则应考虑LR(k)，k>=2。

I5:
S→ae•c, $
A→e•, d

遇到输入字符c时，移进;
遇到输入字符d时，用产生式A→e归约;
冲突得以解决。

LR(0)/SLR(1)/LALR(1)/LR(1)小结

- LALR(1)：
  - 如果LR(1)的项目集中有同心集，则合并同心集；
  - 若合并后不存在归约-归约冲突，则可进行LALR(1)分析；
  - LALR(1)分析基于合并后的项目集进行；
  - 若合并后有冲突，则该文法只能用LR(1)方法进行分析；
  - 比LR(1)弱，比SLR(1)强，是两者的trade-off。



同心集合并后不包含冲突，是LALR(1)项目集，文法是LALR(1)文法，可以用LALR(1)方法分析

LR(0)/SLR(1)/LALR(1)/LR(1)小结

- 主要区别：**归约的判定方法**
  - LR(**0**)**不看**下一个字符即可判定归约；
  - SLR(**1**)**看**下一个字符，但把所有B→γ•的向前搜索字符集均定义为Follow(B)，即把符号栈顶上的句柄γ归约为B的条件是：向前搜索字符为Follow(B)中的元素；
  - LR(**1**)也是**看**下一个字符，但**归约不同位置上的B时，采用不同的向前搜索字符集，**每个项由心与向前搜索符组成，**搜索符长度为1；**由于LR(1)方法对于归约条件的判定比SLR(1)更精确，可大大减少移入/归约冲突；
  - LALR(1): 对LR(1)项目集规范族合并同心集，减少状态个数。

LR(0)/SLR(1)/LALR(1)/LR(1)小结

- 评价
  - LR(0)分析表局限性大，但其构造方法是其他构造方法的基础；
  - SLR(1)分析表虽然不是对所有文法都存在，但这种分析表较易实现又极有使用价值；
  - LR(1)分析表的分析能力最强，能适用于一大类文法，但是，实现代价过高（表过大）；
  - LALR(1)分析表的能力介于SLR(1)和LR(1)之间，实现效率较高。

LR(0)/SLR(1)/LALR(1)/LR(1)小结

方法	核心思想	向前看符号	分析能力	冲突情况	适用场景
LR(0)	基于LR(0)项，无向前看符号，仅依赖当前状态决定动作	**无**	最弱，只能处理极简单的无冲突文法	极易出现移进-归约冲突	仅用于教学，实际工程极少使用（因冲突太多）
SLR(1)	在LR(0)基础上，用FOLLOW集解决冲突，仅归约时检查FOLLOW符号	**FOLLOW集**	比LR(0)强，但仍有限	比LR(0)少，但仍有部分冲突	早期简单解析器，现较少使用（因LALR(1)更优）
LALR(1)	合并LR(1)的同心项集	**局部向前看符号**	比SLR(1)强，接近LR(1)	可能因合并状态引入延迟冲突	**最常用（在分析能力和状态数间取得平衡）**，是Yacc/Bison默认的方法
LR(1)	每个项精确记录不同的向前看符号，不合并任何状态	**精确向前看符号**	**最强**，能处理所有确定性上下文无关文法	几乎没有冲突，除非文法本身有二义性	需要最强分析能力时使用(但状态数大，解析表可能爆炸，一般仅用于理论研究)

LR(0)/SLR(1)/LALR(1)/LR(1)小结

- 思考：句柄哪去了？LR分析的过程中，好像感受不到句柄的存在？
  - LR分析中，句柄仍是核心概念
  - 但它是**隐式存在**的：句柄的识别是通过状态栈和符号栈的动态组合完成的，而非显式标记
  - LR分析器通过DFA跟踪可能的句柄，当你看到“移进-归约”动作时，本质是DFA在识别句柄的边界
  - 当LR分析器按产生式A→β执行“归约”动作时，**栈顶的符号串β就是句柄**
  - LR分析表（ACTION/GOTO）预计算了所有可能的句柄识别路径
  - 用户无需手动识别句柄

6. 二义性文法的LR分析

- 对二义性文法的LR分析

  ① 消除二义性后分析

  ② 直接分析，但须添加额外的文法限制（如优先级或结合性等）

  – 例：文法$G[E]_1$: E→E+E|E*E|(E)|i

  – 消除二义性之后的等价文法为：$G[E]_2$: E→E+T|T，T→T*F|F，F→(E)|i

6. 二义性文法的LR分析

① 对无二义性的$G[E]_2$: E→E+T|T，T→T*F|F，F→(E)|i进行LR分析:

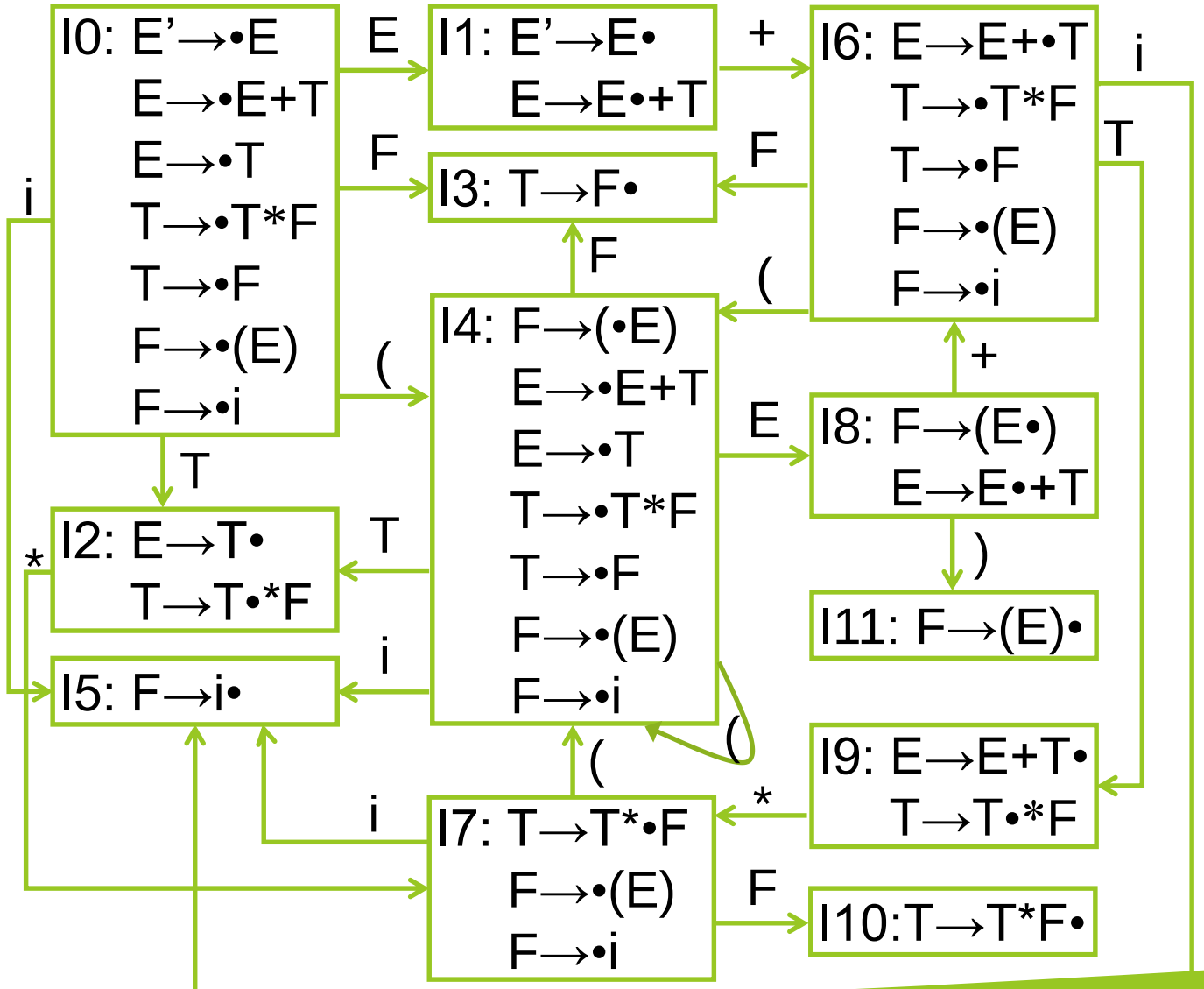
1. 拓广文法:

(0)E'→E (1)E→E+T (2)E→T

(3)T→T*F (4)T→F (5)F→(E)

(6)F→i

2. 构造LR(0)项目集及DFA



6. 二义性文法的LR分析

① 对无二义性的$G[E]_2$: E→E+T|T，T→T*F|F，F→(E)|i进行LR分析：

(0)E’→E (1)E→E+T (2)E→T (3)T→T*F (4)T→F (5)F→(E) (6)F→i

3. 检查是否有冲突

I1：
E’→E•
E→E•+T

FOLLOW(E’)={$}与移进符号{+}无交集，利用SLR(1)方法可解决冲突

I2：
E→T•
T→T•*F

FOLLOW(E)={+,),$}与移进符号{*}无交集，利用SLR(1)方法可解决冲突

I9：
E→E+T•
T→T•*F

FOLLOW(E)={+,),$}与移进符号{*}无交集，利用SLR(1)方法可解决冲突

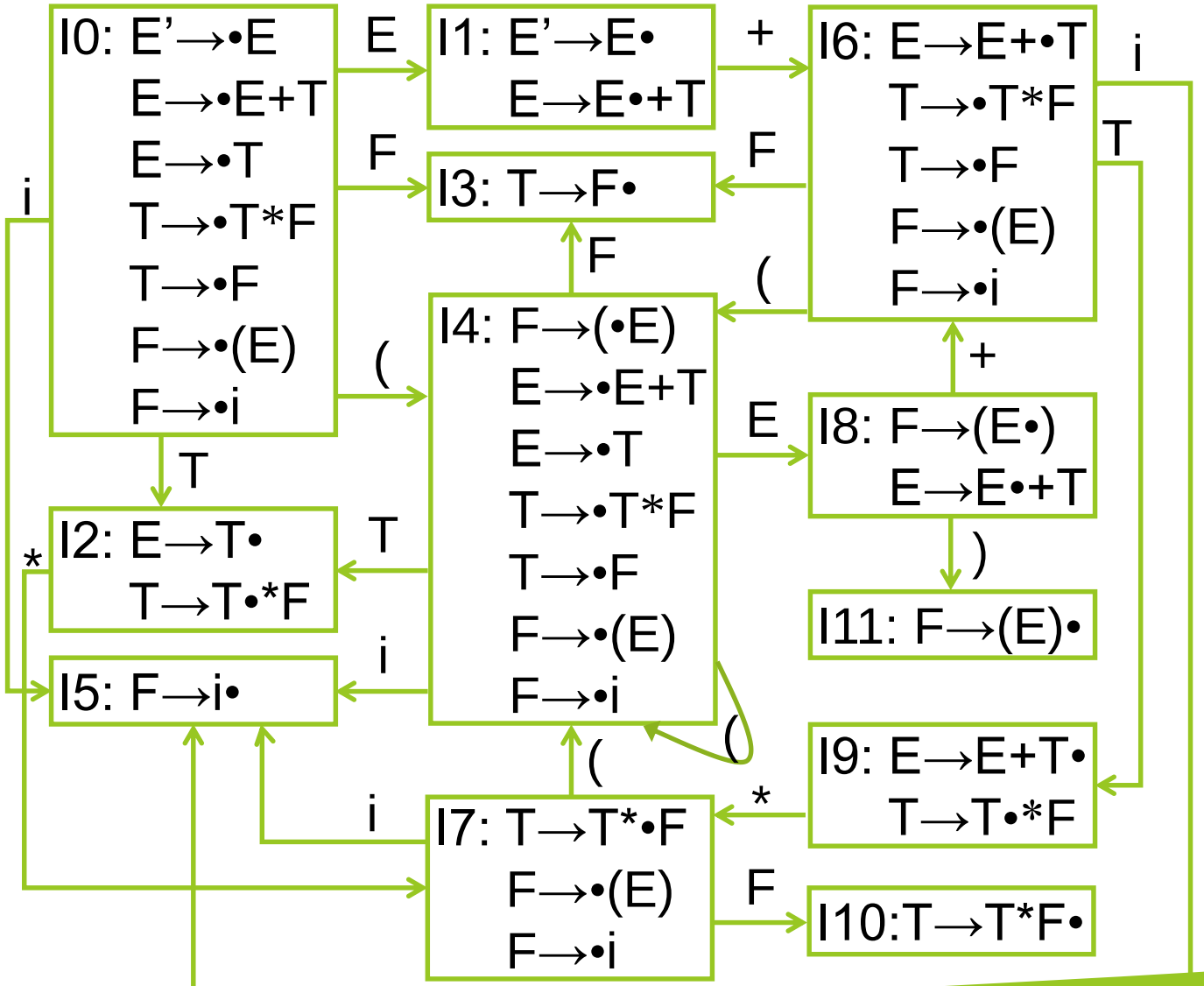
I1、I2、I9状态存在移进-归约冲突

6. 二义性文法的LR分析

(0)E'→E (1)E→E+T (2)E→T (3)T→T*F (4)T→F (5)F→(E) (6)F→i

4. 进行SLR(1)分析

状态	ACTION						GOTO		
	i	+	*	(	)	$	E	T	F
0	S_5			S_4			1	2	3
1		S_6				acc			
2		r_2	S_7		r_2	r_2			
3		r_4	r_4		r_4	r_4			
4	S_5			S_4			8	2	3
5		r_6	r_6		r_6	r_6			
6	S_5			S_4				9	3
7	S_5			S_4					10
8		S_6			S_{11}				
9		r_1	S_7		r_1	r_1			
10		r_3	r_3		r_3	r_3			
11		r_5	r_5		r_5	r_5			



6. 二义性文法的LR分析

(0)E'→E (1)E→E+T (2)E→T (3)T→T*F (4)T→F (5)F→(E) (6)F→i

4. 进行SLR(1)分析　　i+i*i$

状态	ACTION						GOTO		
	i	+	*	(	)	$	E	T	F
0	S_5			S_4			1	2	3
1		S_6				acc			
2		r_2	S_7		r_2	r_2			
3		r_4	r_4		r_4	r_4			
4	S_5			S_4			8	2	3
5		r_6	r_6		r_6	r_6			
6	S_5			S_4				9	3
7	S_5			S_4					10
8		S_6			S_{11}				
9		r_1	S_7		r_1	r_1			
10		r_3	r_3		r_3	r_3			
11		r_5	r_5		r_5	r_5			

步骤	状态栈	符号栈	输入串	ACTION	GOTO
1	0	$	i+i*i$	S_5	
2	05	$i	+i*i$	r_6	3
3	03	$F	+i*i$	r_4	2
4	02	$T	+i*i$	r_2	1
5	01	$E	+i*i$	S_6	
6	016	$E+	i*i$	S_5	
7	0165	$E+i	*i$	r_6	3
8	0163	$E+F	*i$	r_4	9
9	0169	$E+T	*i$	S_7	
10	01697	$E+T*	i$	S_5	
11	016975	$E+T*i	$	r_6	10
12	01697(10)	$E+T*F	$	r_3	9
13	0169	$E+T	$	r_1	
14	01	$E	$	acc	

6. 二义性文法的LR分析

② 直接分析：文法$G[E]_1$: E→E+E|E*E|(E)|i

1. 拓广文法为：(0)E'→E (1)E→E+E (2)E→E*E (3)E→(E) (4)E→i
2. 可以分别验证该文法非LR(0)文法、非SLR(1)文法、亦非LR(1)文法

I7: E→E+E• E→E•+E E→E•*E	I8: E→E*E• E→E•+E E→E•*E

FOLLOW(E)={+,*,),$}与移进符号{+,*}有交集，利用SLR(1)方法无法解决冲突

非SLR(1)文法

I9: E→E+E•, $/+/* E→E•+E, $/+/* E→E•*E, $/+/*

移进和归约的向前搜索符号集有交集，利用LR(1)方法无法解决冲突

非LR(1)文法

3. 但可以对其人为施加限制——利用优先关系和结合性：
   - *的优先级高于+
   - *和+都服从左结合

6. 二义性文法的LR分析

② 直接分析：文法$G[E]_1$: E→E+E|E*E|(E)|i

3. 但可以对其人为施加限制——利用优先关系和结合性：

– *的优先级高于+

– *和+都服从左结合

I7: E→E+E•
E→E•+E
E→E•*E

***比+优先级高，所以遇到*则移进；符号栈$E+E，遇到*，应移进而非归约；**
+服从左结合，所以遇到+则归约。符号栈$E+E，遇到+，应归约。

I8: E→E*E•
E→E•+E
E→E•*E

***比+优先级高，*服从左结合，所以不论遇到+或*都应归约。符号栈$E+E*E，不管遇到+还是*都应归约。**

6. 二义性文法的LR分析

② 直接分析：(0)E'→E (1)E→E+E (2)E→E*E (3)E→(E) (4)E→i

4. 按上述优先关系和结合性构造SLR(1)分析表

状态	ACTION						GOTO
	i	+	*	(	)	$	E
0	S_3			S_2			1
1		S_4	S_5			acc	
2	S_3			S_2			6
3		r_4	r_4		r_4	r_4	
4	S_3			S_2			7
5	S_3			S_2			8
6		S_4	S_5		S_9		
7		r_1	S_5		r_1	r_1	
8		r_2	r_2		r_2	r_2	
9		r_3	r_3		r_3	r_3	

二义性文法SLR(1)分析表

VS.

状态	ACTION						GOTO		
	i	+	*	(	)	$	E	T	F
0	S_5			S_4			1	2	3
1		S_6				acc			
2		r_2	S_7		r_2	r_2			
3		r_4	r_4		r_4	r_4			
4	S_5			S_4			8	2	3
5		r_6	r_6		r_6	r_6			
6	S_5			S_4				9	3
7	S_5			S_4					10
8		S_6			S_{11}				
9		r_1	S_7		r_1	r_1			
10		r_3	r_3		r_3	r_3			
11		r_5	r_5		r_5	r_5			

无二义性文法SLR(1)分析表

6. 二义性文法的LR分析

② 直接分析：(0)E'→E (1)E→E+E (2)E→E*E (3)E→(E) (4)E→i

5. 进行SLR(1)分析　**i+i*i$**

状态	ACTION						GOTO
	i	+	*	(	)	$	E
0	S_3			S_2			1
1		S_4	S_5			acc	
2	S_3			S_2			6
3		r_4	r_4		r_4	r_4	
4	S_3			S_2			7
5	S_3			S_2			8
6		S_4	S_5		S_9		
7		r_1	S_5		r_1	r_1	
8		r_2	r_2		r_2	r_2	
9		r_3	r_3		r_3	r_3	

步骤	状态栈	符号栈	输入串	ACTION	GOTO
1	0	$	i+i*i$	S_3	
2	03	$i	+i*i$	r_4	1
3	01	$E	+i*i$	S_4	
4	014	$E+	i*i$	S_3	
5	0143	$E+i	*i$	r_4	7
6	0147	$E+E	*i$	S_5	
7	01475	$E+E*	i$	S_3	
8	014753	$E+E*i	$	r_4	8
9	014758	$E+E*E	$	r_2	7
10	0147	$E+E	$	r_1	1
11	01	$E	$	acc	

6. 二义性文法的LR分析

② 直接分析：(0)E'→E (1)E→E+E (2)E→E*E (3)E→(E) (4)E→i

5. 进行SLR(1)分析　i+i*i$

步骤	状态栈	符号栈	输入串	ACTION	GOTO
1	0	$	i+i*i$	S_3	
2	03	$i	+i*i$	r_4	1
3	01	$E	+i*i$	S_4	
4	014	$E+	i*i$	S_3	
5	0143	$E+i	*i$	r_4	7
6	0147	$E+E	*i$	S_5	
7	01475	$E+E*	i$	S_3	
8	014753	$E+E*i	$	r_4	8
9	014758	$E+E*E	$	r_2	7
10	0147	$E+E	$	r_1	1
11	01	$E	$	acc	

二义性文法SLR(1)分析

VS.

步骤	状态栈	符号栈	输入串	ACTION	GOTO
1	0	$	i+i*i$	S_5	
2	05	$i	+i*i$	r_6	3
3	03	$F	+i*i$	r_4	2
4	02	$T	+i*i$	r_2	1
5	01	$E	+i*i$	S_6	
6	016	$E+	i*i$	S_5	
7	0165	$E+i	*i$	r_6	3
8	0163	$E+F	*i$	r_4	9
9	0169	$E+T	*i$	S_7	
10	01697	$E+T*	i$	S_5	
11	016975	$E+T*i	$	r_6	10
12	01697(10)	$E+T*F	$	r_3	9
13	0169	$E+T	$	r_1	
14	01	$E	$	acc	

无二义性文法SLR(1)分析

6. 二义性文法的LR分析

- 二义性文法——悬空-else[Dangling-else]：

stmt → **if** *expr* **then** *stmt*

| **if** *expr* **then** *stmt* **else** *stmt*

| **other**

if E1 then if E2 then S1 else S2

- 其对应的无二义性的文法为：

stmt → *matched_stmt* | *open_stmt*

matched_stmt → **if** *expr* **then** ***matched_stmt*** **else** *matched_stmt*

| other

open_stmt → **if** *expr* **then** *stmt*

| **if** *expr* **then** ***matched_stmt*** **else** *open_stmt*

if E1 then if E2 then S1 else S2

6. 二义性文法的LR分析

- 二义性文法——悬空-else[Dangling-else]：

  stmt → **if** *expr* **then** *stmt*

  | **if** *expr* **then** *stmt* **else** *stmt*

  | **other**

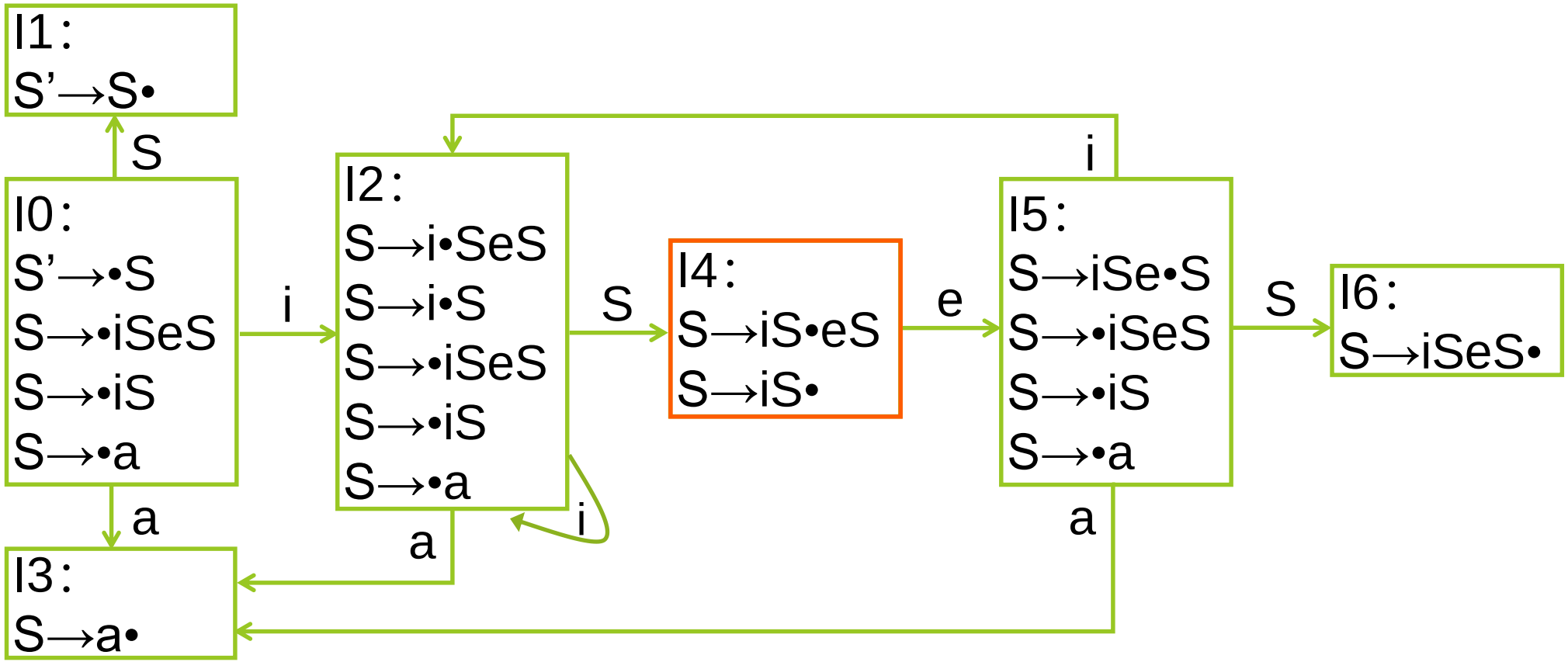
- 假设用**S**表示*stmt*，用**i**表示 **if** *expr* **then**，**e**表示**else**，**a**表示**other**：

  S → iS | iSeS | a

- 拓广为：(0)S’→S (1)S→iSeS (2)S→iS (3)S→a

6. 二义性文法的LR分析

- 对该文法进行LR分析：(0)S’→S (1)S→iSeS (2)S→iS (3)S→a



存在移进-归约冲突；
但FOLLOW(S)={e,$}，因此用SLR(1)无法解决冲突；
可尝试添加人为限制：else总是与离它最近的then配对。

6. 二义性文法的LR分析

- 对该文法进行LR分析：(0)S’→S (1)S→iSeS (2)S→iS (3)S→a

I4:
S→iS•eS
S→iS•

存在移进-归约冲突；
但FOLLOW(S)={e,$}，因此用SLR(1)无法解决冲突；
可尝试添加人为限制：else总是与离它最近的then配对。

假设符号栈中为$**iS**，面临输入符号**e**，恢复成为Dangling-else文法的表达即为：

符号栈中为$**if** *expr* **then** *stmt*，面临输入符号**else**，

此时，按照“**else总是与离它最近的then配对**”的限制条件，应考虑**移进，**

冲突得以解决。

6. 二义性文法的LR分析

- 对该文法进行LR分析：(0)S'→S (1)S→iSeS (2)S→iS (3)S→a
  - 对输入**if *expr* then if *expr* then other else other**$　　**iiaea$**

状态	ACTION				GOTO
	i	e	a	$	S
0	S_2		S_3		1
1				acc	
2	S_2		S_3		4
3		r_3		r_3	
4		S_5		r_2	
5	S_2		S_3		6
6		r_1		r_1	

Dangling-else文法的SLR(1)分析表

步骤	状态栈	符号栈	输入串	ACTION	GOTO
1	0	$	iiaea$	S_2	
2	02	$i	iaea$	S_2	
3	022	$ii	aea$	S_3	
4	0223	$iia	ea$	r_3	4
5	0224	$iiS	ea$	S_5	
6	02245	$iiSe	a$	S_3	
7	022453	$iiSea	$	r_3	6
8	022456	$iiSeS	$	r_1	4
9	024	$iS	$	r_2	1
10	01	$S	$	acc	

Dangling-else文法的SLR(1)分析过程

7. LR分析中的错误恢复[Error Recovery]

- 若能构造出**没有冲突**的LR分析表，则输入的句子中的**所有错误都能被发现（LR分析表中的空白位置）**
- 各种LR分析发现错误的位置可能不一样
  - 在发现错误前需要执行的归约次数：LR(0)>=SLR(1)>=LALR(1)>=LR(1)

- 回顾：错误恢复
  - Parser在Error情况下应该做什么？
    - ✓ **给出Error信息**：Error信息越详尽越好
    - ✓ **从Error中恢复**：能够继续对其余输入进行解析（不要因发现错误而停止语法分析）

7. LR分析中的错误恢复[Error Recovery]

- 回顾：**恐慌模式[Panic Mode]**
  - 是一种错误恢复策略
  - 恐慌：遇到危险先跑再说，不尝试精确修复错误，直接跳过
  - 目标：让parser尽快回到可以继续分析的状态，而不是纠结于错误的细节
  - 基本思想：当parser遇到一个无法处理的错误时，采取“紧急避险”的方式，简单粗暴地**忽略/跳过**输入中的一些符号，直到输入中出现一些特定的符号

7. LR分析中的错误恢复[Error Recovery]

- **恐慌模式[Panic Mode]**
  - 发现错误：当前状态面临当前输入符没有合法动作（分析表对应位置空白）
  - 解决方式：
    - ✓ 从栈顶往下扫描
    - ✓ 直到发现某个状态S，它有一个对应于某个非终结符A的GOTO目标
    - ✓ 丢弃零个或多个输入符号，直到发现一个可能合法地跟在A之后的符号a为止
    - ✓ 将GOTO(S，A)压入栈中
    - ✓ 继续进行正常的语法分析

(0)E'→E (1)E→E+E (2)E→E*E (3)E→(E) (4)E→i

状态	ACTION						GOTO
	i	+	*	(	)	$	E
0	S_3			S_2			1
1		S_4	S_5			acc	
2	S_3			S_2			6
3		r_4	r_4		r_4	r_4	
4	S_3			S_2			7
5	S_3			S_2			8
6		S_4	S_5		S_9		
7		r_1	S_5		r_1	r_1	
8		r_2	r_2		r_2	r_2	
9		r_3	r_3		r_3	r_3	

假设当前状态栈为01478，输入字符串为i*i+i$

7. LR分析中的错误恢复[Error Recovery]

- **短语级别的错误恢复**
  - 检查LR分析表中的**每个报错条目**
  - 根据语言的使用方法来决定程序员所犯的何种错误最有可能引起这个语法错误
  - 构造出适当的恢复过程：给出具体的错误处理例程
  - 在分析表的ACTION表中**每个空条目中填写一个指向错误处理例程的指针**

7. LR分析中的错误恢复[Error Recovery]

- 短语级别的错误恢复
  - 例：文法G[E]: E→E+E|E*E|(E)|i

状态	ACTION						GOTO
	i	+	*	(	)	$	E
0	S_3			S_2			1
1		S_4	S_5			acc	
2	S_3			S_2			6
3		r_4	r_4		r_4	r_4	
4	S_3			S_2			7
5	S_3			S_2			8
6		S_4	S_5		S_9		
7		r_1	S_5		r_1	r_1	
8		r_2	r_2		r_2	r_2	
9		r_3	r_3		r_3	r_3	

SLR(1)分析表

状态	ACTION						GOTO
	i	+	*	(	)	$	E
0	S_3	e_1	e_1	S_2	e_2	e_1	1
1	e_3	S_4	S_5	e_3	e_2	acc	
2	S_3	e_1	e_1	S_2	e_2	e_1	6
3	r_4	r_4	r_4	r_4	r_4	r_4	
4	S_3	e_1	e_1	S_2	e_2	e_1	7
5	S_3	e_1	e_1	S_2	e_2	e_1	8
6	e_3	S_4	S_5	e_3	S_9	e_4	
7	r_1	r_1	S_5	r_1	r_1	r_1	
8	r_2	r_2	r_2	r_2	r_2	r_2	
9	r_3	r_3	r_3	r_3	r_3	r_3	

带错误恢复例程的SLR(1)分析表

7. LR分析中的错误恢复[Error Recovery]

- 短语级别的错误恢复
  - 例：文法G[E]: E→E+E|E*E|(E)|i
    - ✓ r_j：对那些在某些输入上执行特定归约动作的状态，将报错条目替换为归约动作，延后报错的时间；
    - ✓ e_1：在状态0,2,4,5上，这些状态原本期望读入一个运算分量的第一个符号(i或左括号)，但实际却读入+或*或$，此时：将状态0,2,4,5在i上的移进目标，即**状态3压入状态栈，符号i压入符号栈**，并报错：**缺少运算分量**；

状态	ACTION						GOTO
	i	+	*	(	)	$	E
0	S_3	e_1	e_1	S_2	e_2	e_1	1
1	e_3	S_4	S_5	e_3	e_2	acc	
2	S_3	e_1	e_1	S_2	e_2	e_1	6
3	r_4	r_4	r_4	r_4	r_4	r_4	
4	S_3	e_1	e_1	S_2	e_2	e_1	7
5	S_3	e_1	e_1	S_2	e_2	e_1	8
6	e_3	S_4	S_5	e_3	S_9	e_4	
7	r_1	r_1	S_5	r_1	r_1	r_1	
8	r_2	r_2	r_2	r_2	r_2	r_2	
9	r_3	r_3	r_3	r_3	r_3	r_3	

带错误恢复例程的SLR(1)分析表

7. LR分析中的错误恢复[Error Recovery]

- 短语级别的错误恢复
  - 例：文法G[E]: E→E+E|E*E|(E)|i
    - ✓ e_2：在状态0,1,2,4,5上，遇到右括号，此时：**从输入中删除右括号**，并报错：**不匹配的右括号**；
    - ✓ e_3：在状态1,6上，期待读入一个运算符却发现了一个i或左括号，此时：将状态1,6在+上的移进目标，即**状态4压入状态栈，符号+移进符号栈**，并报错：**缺少运算符**；
    - ✓ e_4：在状态6上发现输入是结束符$，此时：将状态6在右括号上的移进目标，即**状态9压入状态栈中，右括号压入符号栈**，并报错：**缺少右括号**。

状态	ACTION						GOTO
	i	+	*	(	)	$	E
0	S_3	e_1	e_1	S_2	e_2	e_1	1
1	e_3	S_4	S_5	e_3	e_2	acc	
2	S_3	e_1	e_1	S_2	e_2	e_1	6
3	r_4	r_4	r_4	r_4	r_4	r_4	
4	S_3	e_1	e_1	S_2	e_2	e_1	7
5	S_3	e_1	e_1	S_2	e_2	e_1	8
6	e_3	S_4	S_5	e_3	S_9	e_4	
7	r_1	r_1	S_5	r_1	r_1	r_1	
8	r_2	r_2	r_2	r_2	r_2	r_2	
9	r_3	r_3	r_3	r_3	r_3	r_3	

带错误恢复例程的SLR(1)分析表

7. LR分析中的错误恢复[Error Recovery]

- 短语级别的错误恢复
  - 例：文法G[E]: E→E+E|E*E|(E)|i

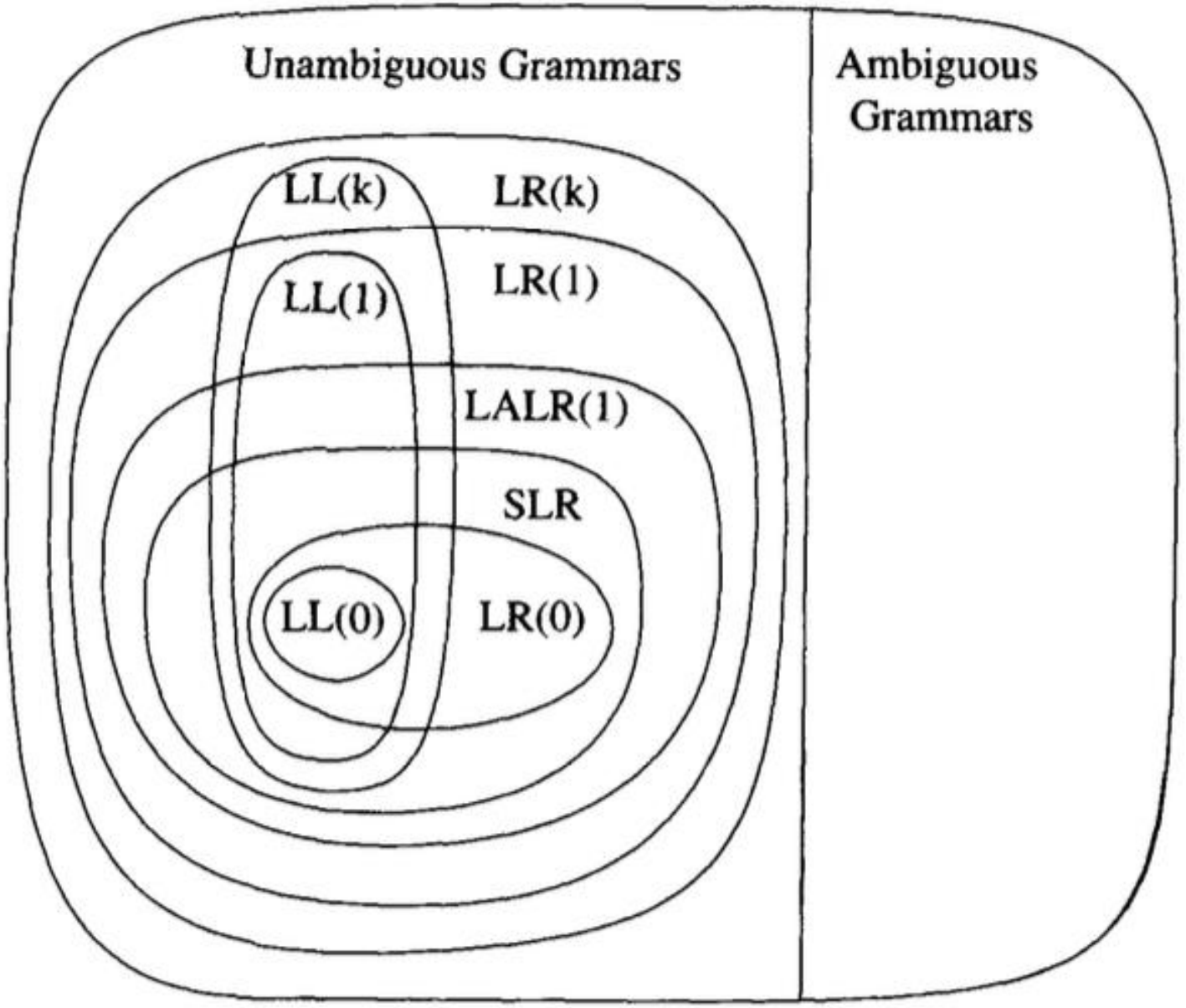
步骤	状态栈	符号栈	输入串	ACTION	GOTO
1	0	$	i+)$	S_3	
2	03	$i	+)$	r_4	1
3	01	$E	+)$	S_4	
4	014	$E+	)$	e_2:不匹配的右括号，从输入中删除右括号	
5	014	$E+	$	e_1：缺少运算分量，将状态3压入状态栈，符号i压入符号栈	
6	0143	$E+i	$	r_4	7
7	0147	$E+E	$	r_1	1
8	01	$E	$	end	

状态	ACTION						GOTO
	i	+	*	(	)	$	E
0	S_3	e_1	e_1	S_2	e_2	e_1	1
1	e_3	S_4	S_5	e_3	e_2	acc	
2	S_3	e_1	e_1	S_2	e_2	e_1	6
3	r_4	r_4	r_4	r_4	r_4	r_4	
4	S_3	e_1	e_1	S_2	e_2	e_1	7
5	S_3	e_1	e_1	S_2	e_2	e_1	8
6	e_3	S_4	S_5	e_3	S_9	e_4	
7	r_1	r_1	S_5	r_1	r_1	r_1	
8	r_2	r_2	r_2	r_2	r_2	r_2	
9	r_3	r_3	r_3	r_3	r_3	r_3	

带错误恢复例程的SLR(1)分析表

总结

- 自顶向下语法分析
  - 递归下降预测分析
  - LL(1)分析
- 自底向上语法分析[移进-归约]
  - 算法优先分析
  - LR分析
    - ✓ LR(0)
    - ✓ SLR(1)
    - ✓ LR(1)
    - ✓ LALR(1)



总结

- 重点掌握
  - 进行LL(1)分析
    - ✓ 消除左公因子、左递归、二义性等情况
    - ✓ 求FIRST集、FOLLOW集
    - ✓ 构造LL(1)分析表
    - ✓ 进行LL(1)分析
  - 进行LR分析——LR(0)、SLR(1)、LR(1)、LALR(1)
    - ✓ 构建项目集、CLOUSRE函数、GOTO函数、DFA
    - ✓ 构建LR分析表
    - ✓ 进行LR分析
    - ✓ 能分辨出一个文法是属于哪种LR文法

第四章课后作业（1）

- 对文法G[S]：S→(L)|a，L→L,S|S，构造预测分析表（**含恐慌模式**），并对输入符号串(a,(a,a))、(a,)和(a,a分别进行LL(1)预测分析，写出分析的全过程（表格形式）。
- 提交要求：
  - 文件命名：学号-姓名-第四章作业(1)；
  - 文件格式：.pdf文件；
  - 手写版、电子版均可；若为手写版，则拍照后转成pdf提交，但**须注意将照片旋转为正常角度，且去除照片中的多余信息**；电子版如word等转成pdf提交；
  - 提交到超算习堂（第四章作业(1)）处；
  - 提交ddl：**4月6日晚上12:00**；
  - **重要提示：不得抄袭！**

第四章课后作业（2）

- 证明文法G[S]：S→Ca|bCe|De|bDa，C→d，D→d是LR(1)而不是LALR(1)的，构造LR(1)分析表，并对输入字符串bda$进行LR(1)分析，写出分析的全过程（表格形式）。
- 提交要求：
  - 文件命名：学号-姓名-第四章作业(2)；
  - 文件格式：.pdf文件；
  - 手写版、电子版均可；若为手写版，则拍照后转成pdf提交，但**须注意将照片旋转为正常角度，且去除照片中的多余信息**；电子版如word等转成pdf提交；
  - 提交到超算习堂（第四章作业(2)）处；
  - 提交ddl：**4月22日晚上12:00**；
  - **重要提示：不得抄袭！**